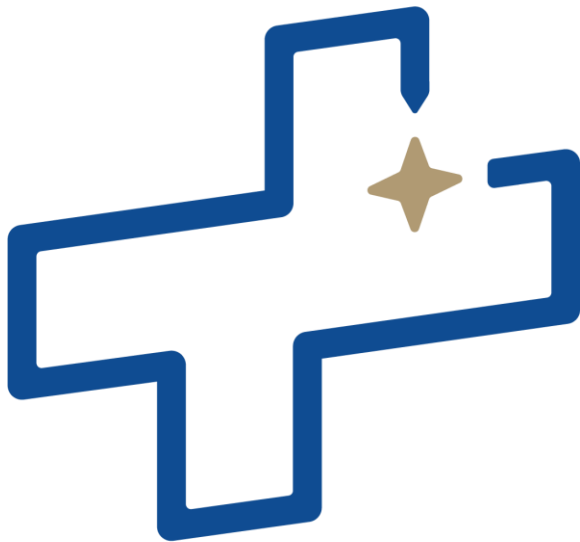




CUBESPACE

User Manual CubeADCS Gen. 2

DOCUMENT NUMBER
VERSION
DATE
PREPARED BY
REVIEWED BY
APPROVED BY
DISTRIBUTION LIST

CS-DEV.UM.CA-01
1.13
25/08/2025
DEVELOPMENT ENGINEERING TEAM
HS, LV, FL, AS, NR
N. Roets, V. Dajee
External



Revision History

VERSION	AUTHORS	DATE	DESCRIPTION
1.00	C. Leibbrandt	31/03/2022	Published – first formal release
1.01	C. Leibbrandt	25/04/2022	Added description on how to reprogram the Bootloader
1.02	C. Leibbrandt, C. Groenewald	20/05/2022	Bootloader allows for auto discovery, update document to reflect same and remove all major functions and features descriptions (now in Product Description)
1.03	C. Groenewald	10/10/2022	Expanded ADCS descriptions
1.04	C. Leibbrandt, L. Visagie, C. Groenewald,	01/12/2022	Updated to reflect latest information
1.05	F. Louw	23/01/2023	Updated Appendix to reflect latest CubeSupport application version and added additional tests
1.06	C. Leibbrandt, O. Gries, F. Louw	13/12/2023	Added CubeProduct upgrade functionality Additional discussion of PC104 and Backplane variants Additional detail of I2C, RS422 and RS485 communications Added brief discussion of Base-Bootloader Moved detail discussion from Section 5 to Commissioning Manual ([RD5]) Added health check details for CubeStar, GNSS, and several third-party products. Improved magnetorquer testing procedure. Other minor diverse improvements
1.07	F. Louw	16/05/2024	Adjusted the recommended health-check supply voltage to the more commonly used value of 12V. Added clarification for testing CubeWheels of size CW0500 or larger in Sections 6.5 and 6.17 Adjusted values of theoretical predicted Magnetorquer currents in Table 20
1.08	M. du Plessis J. Miller	19/09/2024	Updated CubeWheel health check to single wheel speed of 2000 rpm Added CW0500 parameters
1.09	M. du Plessis F. Louw	21/10/2024	Added additional Control Modes to Table 9. Updated Section 6.12 to send correct time for GNSS Health Check
1.10	K. Malan M. Luiz	12/10/2024	Added CubeAuriga health check steps Note regarding GYR ranges added to Table 15
1.11	F. Louw	30/05/2025	Extended GYR1 range, and updated note in Table 15 Corrected CubeSense Earth expected current consumption in Table 36 Changed order of tests during CubeAuriga health checks Removed temporary setting of Integration time multiplier during CubeStar health checks Added CW5000 specifications
1.12	J. Miller	14/07/2025	Added CW1200 specifications. Applied company template.
1.13	J. Miller F. Louw	23/07/2025	Added details of test setups using V_{Bat} bypass for CW0500 and larger. Updated Section 6.5.1 with power-on sequence during test and V_{Bat} Bypass boards Added example command in GNSS Section 6.12.1.



Reference Documents

The following documents are referenced in this document.

[RD1]	CS-DEV.PD.CA-01	CubeADCS Product Description Ver.1.03 or later
[RD2]	CS-DEV.ICD.CA-01	CubeADCS ICD Ver.1.04 or later
[RD3]	CS-DEV.AMNL.BL-01	Bootloader Application Manual, Ver 5.00 or later
[RD4]	CS-DEV.FRM.CA-01	CP Firmware Reference Manual, Ver 6.01
[RD5]	CS-DEV.CMNL.CA-01	CubeADCS Commissioning and Operations Manual, Ver.1.02 or later
[RD6]	N/A	common-framework/common-framework-1.xml (distributed with software package)
[RD7]	N/A	cube-computer-5/cube-computer-common-3.xml (distributed with software package)
[RD8]	N/A	cube-computer-5/control-program-8.xml (distributed with software package)



List of Acronyms/Abbreviations

ACP	ADCS Control Program
ADCS	Attitude Determination and Control System
CAN	Controller Area Network
COTS	Commercial Off The Shelf
CSS	Coarse Sun Sensor
CVCM	Collected Volatile Condensable Materials
DUT	Device Under Test
EDAC	Error Detection and Correction
EHS	Earth Horizon Sensor
EM	Engineering Model
EMC	Electromagnetic Compatibility
EMI	Electromagnetic Interference
ESD	Electrostatic Discharge
FDIR	Fault Detection, Isolation, and Recovery
FM	Flight Model
FSS	Fine Sun Sensor
GID	Global Identification
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
GYR	Gyroscope
HIL	Hardware-in-the-loop
HSS	Horizon Sensor System
I2C	Inter-Integrated Circuit
ID	Identification
LTDN	Local Time of Descending Node
LEO	Low Earth Orbit
MCU	Microcontroller Unit
MEMS	Microelectromechanical System
MTM	Magnetometer
MTQ	Magnetorquer
NDA	Non-Disclosure Agreement
OBC	On-board Computer
PCB	Printed Circuit Board



PPS	Pulse Per Second
RTC	Real-Time Clock
RWA	Reaction Wheel Assembly
RW	Reaction Wheel
SBC	Satellite Body Coordinate
SOFIA	Software Framework for Integrated ADCS
SPI	Serial Peripheral Interface
SRAM	Static Random-Access Memory
SSP	Sub-Satellite Point
STR	Star Tracker
TC	Telecommand
TCTLM	Telecommand and Telemetry (protocol)
TID	Total Ionizing Dose
TLM	Telemetry
TML	Total Mass Loss
UART	Universal Asynchronous Receiver/Transmitter



Table of Contents

1. Introduction	12
1.1 Scope.....	12
2. Handling	13
3. Getting Started	14
3.1 CubeADCS Variants.....	14
3.2 Ground Support Equipment (GSE)	15
3.3 Connecting to the CubeADCS	16
3.3.1 Power setup.....	16
3.3.2 Sensors and Actuators Connection Guide	18
3.3.3 Communication setup.....	24
3.3.4 Communicating with the CubeADCS using CubeSupport PCB.....	29
3.4 Brief Base-Bootloader description	34
4. Functions and Features	38
4.1 Identification	38
4.2 Configuration.....	38
4.3 Power management.....	39
4.4 Bootloader.....	40
4.5 Auto-Discovery.....	40
4.5.1 Diagnostics.....	40
4.5.2 Timing	42
4.6 Programming and Updating of CubeADCS.....	43
4.7 Updating of CubeADCS configuration and Control Program firmware through Bootloader	44
4.8 Updating a connected CubeProduct's configuration and control application firmware via CubeComputer.....	44
4.8.1 Uploading files.....	45
4.8.2 Requesting file information.....	45
4.8.3 Performing a CubeProduct upgrade	46
4.8.4 CubeProduct upgrades with ForcePort	46
4.8.5 Control mode and power handling during a CubeProduct upgrade	46
4.9 Recovery on reset	47
5. ADCS attitude determination and control.....	48
6. Appendix: Health Check/Acceptance test	51
6.1 Connection to the unit with CubeSupport software.....	52
6.2 CubeComputer health check.....	52
6.3 Auto-Discovery results check	57



6.4 CubeTorquer health check	58
6.5 CubeWheel health check	61
6.5.1 Preparing for testing CW0500 and larger.....	62
6.5.2 Common health-check steps for all CubeWheel variants.....	65
6.6 CubeSense Sun health check	67
6.7 CubeMag health check	70
6.8 CubeSense Earth health check	71
6.9 CubeStar health check	73
6.10 CubeAuriga health check	76
6.10.1 Preparation	76
6.10.2 Health Check.....	76
6.10.3 Uploading the Optical Head Configuration binary file	80
6.11 CSS health check.....	81
6.12 GNSS interface health check	82
6.12.1 GNSS data update via telecommand.....	83
6.12.2 GNSS data update via UART interface.....	85
6.13 PPS health check.....	91
6.14 Third-party Reaction Wheel health check	94
6.15 PST3S Star Tracker health check.....	99
6.16 LITEF uFORS Fibre Optic Gyro Health check	105
6.17 Operation at high and low ends of expected VBAT range	108
6.18 Known issues.....	112
7. Appendix: CubeProduct Upgrades via CubeSupport.....	113

Table of Tables

Table 1: Typical GSE items in CubeADCS package	16
Table 2: I2C Communications Check: C232HM-DDHSL-0 connection to CubeSupport PCB	26
Table 3: RS422 Communications Check: Connection to CubeSupport PCB.....	28
Table 4: Configuration messages (matching TCs and TLMs) for ADCS configuration settings.	39
Table 5: Port Diagnostics – Global Errors.....	41
Table 6: Port Diagnostics – Individual Port Errors	41
Table 7: Boot reset scenarios.....	47
Table 8: Estimation mode summary	48
Table 9: Control mode summary	49
Table 10: User and CubeADCS Details.....	51
Table 11: CubeComputer Health Check: Expected Current Consumption	52
Table 12: CubeComputer Health Check: Bootloader configuration.....	53



Table 13: CubeComputer Health Check: Communication interfaces of the Bootloader	53
Table 14: CubeComputer Health Check: Communication interfaces of the Control Program	54
Table 15: CubeComputer Health Check: Expected Results	54
Table 16: Auto-Discovery result.....	58
Table 17: CubeTorquer Health Check: Record of magnetorquers DC resistance	59
Table 18: CubeTorquer Health Check: Open-Loop Command.....	59
Table 19: CubeTorquer Health Check: Voltage on 5V rail.....	59
Table 20: CubeTorquer Health Check: Expected Current Consumption	60
Table 21: CubeTorquer Health Check: Polarity	61
Table 22: CubeWheel Health Check: Power On	65
Table 23: CubeWheel Health Check: Expected (additional) Current Consumption	65
Table 24: CubeWheel Health Check: Command Wheel Speed.....	66
Table 25: CubeWheel Health Check: Expected Actual speeds.....	66
Table 26: CubeWheel Health Check: Expected (additional) Current Consumption	66
Table 27: CubeSense Sun Health Check: Power On.....	67
Table 28: CubeSense Sun Health Check: Expected Current Consumption	67
Table 29: CubeSense Sun Health Check: Adjusting settings to temporary values for ground testing	68
Table 30: CubeSense Sun Health Check: Expected Results	69
Table 31: CubeSense Sun Health Check: Image download	70
Table 32: CubeMag Health Check: Power On.....	70
Table 33: CubeMag Health Check: Expected Current Consumption	70
Table 34: CubeMag Health Check: Expected Results	71
Table 35: CubeSense Earth Health Check: Power On.....	71
Table 36: CubeSense Earth Health Check: Expected Current Consumption	72
Table 37: CubeSense Earth Health Check: Telemetry	72
Table 38: CubeSense Earth Health Check: Image download.....	73
Table 39: CubeStar Health Check: Power On	73
Table 40: CubeStar Health Check: Expected Current Consumption	73
Table 41: CubeStar Health Check: Expected Results.....	74
Table 42: CubeStar Health Check: Image download.....	75
Table 43: CubeAuriga Health Check: Power On	76
Table 44: CubeAuriga Health Check: Expected Current Consumption	76
Table 45: Optical Head Calibration binary Param IDs.....	78
Table 46: CubeAuriga Test results	79
Table 47: CubeAuriga Health Telemetry: Expected Results.....	79
Table 48: Raw CubeAuriga Telemetry expected values	80
Table 49: CSS Health Check: Expected Results	82
Table 50: GNSS Update over telecommand check: Expected Results.....	84
Table 51: GNSS interface testing: UART data	87
Table 52: GNSS Interface Health Check: Expected Results	90



Table 53: GNSS PPS Health Check: Power On	92
Table 54: GNSS PPS Health Check: Output header pin voltage level check.....	93
Table 55: CubeNode-NSSRWL Health Check: CubeNode identifier result	96
Table 56: Reaction Wheel Health Check: Power On	96
Table 57: Reaction Wheel Health Check: Command Wheel Speed.....	97
Table 58: Reaction Wheel Health Check: Expected Actual speed.....	97
Table 59: CubeNode-PST3S Health Check: CubeNode identifier result.....	100
Table 60: PST3S Star Tracker Health Check: Power On	101
Table 61: PST3S Star Tracker Health Check: Expected Current Consumption	101
Table 62: PST3S Star Tracker Health Check: Expected Results without PST3S Star Tracker connected.	101
Table 63: PST3S Star Tracker Health Check: Expected Results with PST3S Star Tracker connected.	103
Table 64: PST3S Star Tracker Health Check: Checks passed.....	105
Table 65: uFORS Gyros Health Check: CubeNode Quad identifier result	107
Table 66: uFORS Gyros Health Check: Power On	107
Table 67: uFORS Gyros Health Check: Expected Current Consumption	107
Table 68: uFORS Gyros Health Check: Expected rates.....	108
Table 69: VBAT extremes for different product configurations	109
Table 70: Extremes to be used for VBAT test.....	109
Table 71: Operation against VBAT extremes.....	109

Table of Figures

Figure 1: PC104 variant of CubeADCS.....	14
Figure 2: Backplane variant of CubeADCS.....	15
Figure 3: “VBAT On”, “BOOT” and “Enable” switches in the OFF position (down)	17
Figure 4: Supply and Ground wires of the supplied DC power cable	17
Figure 5: CubeSupport PCB with the DC cable connected, the “VBAT On” switch in the ON (up) position, and the LEDs turned on to indicate power-on condition.....	18
Figure 6: Connection Port Designators on CubeConnect (PC104 variant, rear view)	19
Figure 7: Connection Port Designators on CubeConnect (PC104 variant, top view)	19
Figure 8: Connection Port Designators on CubeConnect (Backplane variant, top view)	19
Figure 9: PPS Port on CubeConnect (PC104 variant, rear view).....	20
Figure 10: PPS Ports on CubeConnect (Backplane variant, top view)	20
Figure 11: Course Sun Sensor manufactured by CubeSpace.....	21
Figure 12: Course Sun Sensor with harness.....	21
Figure 13: Course Sun Sensor connector harness (one of two).....	21
Figure 14: CSS Port Designators on CubeConnect (PC104 variant, rear view)	22
Figure 15: CSS Port Designators on CubeConnect (Backplane variant, top view).....	22
Figure 16: Magnetorquers and harness.....	23
Figure 17: Magnetorquers port on CubeConnect (PC104 variant, top view).....	23
Figure 18: Magnetorquers port on CubeConnect (Backplane variant, top view)	23



Figure 19: CubeSupport PCB “UART” and “CAN2 CC” headers indicated.....	24
Figure 20: CubeSupport PCB with UART and PCAN-USB connections made.	25
Figure 21: CubeSupport PCB connected to CubeADCS core (PC104 variant)	25
Figure 22: CubeSupport PCB connected to CubeADCS core (Backplane variant).....	26
Figure 23: CubeSupport PCB I2C Pins.	27
Figure 24: FTDI cable plugged into CubeSupport PCB I2C Pins.	27
Figure 25: High-level schematic of cable used for RS422 testing (not supplied)	28
Figure 26: CubeSupport PCB RS422 Pins.....	28
Figure 27: Connection dropdown	29
Figure 28: CAN interface selected	30
Figure 29: Default CAN settings.....	30
Figure 30: Selecting the CubeSat Space Protocol checkbox.....	30
Figure 31: CubeSupport connected to CubeADCS firmware (Control-Program).....	31
Figure 32: Cancelling address scanning once CubeADCS has been found	31
Figure 33: Folded sections of the CubeSupport CubeADCS interface.....	32
Figure 34: Connection dropdown	32
Figure 35: UART interface selected	33
Figure 36: Default UART settings	33
Figure 37: Selecting I2C interface	33
Figure 38: I2C default settings	34
Figure 39: CubeComputer Base Bootloader perspective	35
Figure 40: Refreshing the file list.....	35
Figure 41: Jumping to the Control Program	36
Figure 42: Base Bootloader configuration settings	36
Figure 43: Refresh icon	57
Figure 44: Updated list of expected nodes	58
Figure 45: Providing V_{Bat} and GND directly to CubeDoor, bypassing CubeSupport PCB, for testing CubeSpace Reaction Wheels CW0500 and larger	62
Figure 46: Single CubeWheel plugged into the PC104 variant of CubeADCS Core with a Top-Mount V_{Bat} Bypass board	63
Figure 47: Single CubeWheel plugged into the Backplane variant of CubeADCS Core with a V_{Bat} Bypass board.....	63
Figure 48: Setup of two independent Power Supply channels. Channel 1 = 12V for CubeSupport, Channel 2 = 28V for V_{Bat}	64
Figure 49: Accessing the Passthrough tab from the drop-down menu.....	68
Figure 50: Example of captured CubeSense Sun image as pointed at a mobile phone flashlight.....	70
Figure 51: Example of captured CubeSense Earth image as pointed at a roof light.	73
Figure 52: An image taken with CubeStar.	75
Figure 53: Confirming the correct calibration binary uploaded to the CubeAuriga.	77
Figure 54: Setting the correct Magic Number to perform the SRAM self-test.	78
Figure 55: SRAM self-test result.	78



Figure 56: Setting the correct Magic Number to perform the OH self-test.....	78
Figure 57: Auriga optical head self-test result.	79
Figure 58: File list showing the three required files.	81
Figure 59: Reconfiguring the default jump target.....	81
Figure 60: Diagram of GNSS UART interface test.	85
Figure 61: GNSS UART test setup.....	86
Figure 62: GND points on the CubeSupport PCB.	86
Figure 63: Python code for sending GNSS data.	88
Figure 64: Creating a binary file using HxD.	88
Figure 65: Sending the created binary file using Terminal by Br@y++.	89
Figure 66: Expected GNSS retrieved data.....	89
Figure 67: CubeSupport PCB “PPS_L_In” header pins indicated. Either of these can be used.....	91
Figure 68: CubeSupport PCB “PPS_CC” header pins indicated. Either of these can be used.	91
Figure 69: Pinout of the PPS output header, PC104 variant.....	92
Figure 70: Pinout of the PPS output headers, Backplane variant.....	93
Figure 71: High-level diagram of CubeADCS, CubeNodes, Power Switch Board, and reaction wheels	94
Figure 72: CubeADCS, CubeNode, Power Switch Board, and T2 reaction wheel	95
Figure 73: TY-Space Star Tracker connected to the CubeADCS via CubeNode.....	99
Figure 74: TY-Space Star Pattern Simulator.....	103
Figure 75: TY-Space Star Pattern Simulator mounted on the TY-Space Star Tracker	103
Figure 76: Three uFORS Gyros connected to the CubeADCS via CubeNode Quad	106
Figure 77: File Transfer / Upgrades UI.....	113
Figure 78: File Load Option	113



1. Introduction

This document serves as a general introduction and manual to operating the CubeADCS.

A CubeADCS can be configured in various ways with the system having a specific selection of sensors and actuators to accommodate different mission ADCS objectives. This document is applicable to all CubeADCS configurations in general. Certain sections are only applicable to a CubeADCS that contains the relevant sensors or actuators. Therefore, the user's discretion is required to determine whether a specific section is relevant to the CubeADCS under investigation.

For custom CubeADCS systems the user should refer to a User Specific CubeADCS addendum provided alongside this generic User Manual that describes all specifics applicable to the user that differ from the information provided in this "generic" User Manual.

1.1 Scope

This document assumes the user has already read [RD1] and [RD2], typically supplied to a client / user before the User Manual (this document) is provided.

This document is to be used in combination with the referenced documents to: (a) unpack, assemble, integrate and (b) test (conduct a CubeADCS health-check) after receipt of the new CubeProducts that make up the ADCS system.

This document should be studied in full before the (new) user starts working with the CubeADCS unit. Ensure that all instructions and intended use cases are understood before any action is taken.

First time users are then encouraged to perform actions in the applicable¹ sections of chapter 3 "Getting Started" and then to perform the health-check test procedure as described in Section 6.

As an overview, this User Manual addresses handling precautions applicable to the unit in chapter 2.

A "Getting started" chapter is provided in chapter 3 which guides the user to unpack the unit while subsection 3.3 details interfacing to the unit, addressing the necessary power supply and communication harnesses and establishing initial communication with the unit. Once the user has completed the "Getting started" chapter, the CubeADCS is correctly set-up for the Health Check test.

Chapter 4 describes a select subset of functions and features offered by CubeADCS focussed on guiding the user to be able to successfully assemble and integrate the CubeADCS for a complete standalone Health Check before fitment into the supersystem / satellite.

CubeSpace is continually innovating on designs and device capabilities. Since the firmware for CubeADCS is always improving, CubeSpace will release updated software from time to time. Chapter 4.5.1 explains how to reprogram such an update to CubeADCS.

Chapter 5 describes features of the CubeADCS specific to attitude estimation (determination) and control.

Chapter 6 provides a health check sequence that establishes that the CubeADCS is "alive and well" after being delivered to the user. The steps to perform this test are explained and tables are provided with expected results and space for the user to capture the actual results obtained.

¹ Applicable sections intend to indicate that the user only needs to refer to the sections for the sensors and/or actuators as per the ADCS system under investigation's configuration (i.e. what was ordered). Refer to the User Specific CubeADCS addendum for all non-standard / generic / client specific additional information.



2. Handling



Anti-static: The unit contains several static sensitive components. Therefore, the appropriate electrostatic discharge (ESD) protection measures must be implemented. The unit must never be handled without proper grounding.



Cleanliness: It is recommended that the unit is handled in a clean environment. Therefore, an appropriate laminar flow workbench or cleanroom of ISO class 8 or better should be used.



Moisture: The unit should be kept free of moisture and liquids. These could have corrosive effects on the electronics and electronic joints, leading to degradation and loss of reliability of the circuits.



Shock

The unit must be handled with care. Dropping or bumping the unit should be avoided altogether.



Camera lens cleanliness: The camera lenses should be kept clean and free of dirt that may obstruct the images captured by the camera. Dust should be removed with a microfibre cloth. The lens may be cleaned using ethanol and appropriate lens cleaning equipment if required. Avoid unnecessary cleaning of the lens.



Camera lens covers: Some optics are fitted with dust caps which should be removed before flight.



Magnetometer measurements will be distorted if a workbench containing ferrous parts, such as iron; steel; stainless steel; etc, are used. Working on such a surface will have a dramatic effect on sensor measurements and may cause observed results to differ significantly from reference results.



Housings: The aluminium housing of the different components should NOT be tampered with. Tampering with the housings may damage the components inside. No attempt should be made to loosen or remove the fasteners that secure the housings.



3. Getting Started

This section describes what is required from the user's Lab and what support equipment is provided by CubeSpace to be able to start using the CubeADCS.

The following items are required before the user can get started with unpacking and setup of the CubeADCS:

- Multi-meter or oscilloscope
- Computer with an open USB port (running Windows 7 or later)
- Power Supply which can supply a voltage range of 10 V to 17.6 V at a current limit set to 1.5A.
Note that this is sufficient to test CubeADCS and most sensors, but some configurations of reaction wheels might require a higher voltage and current supply, this is discussed later in this document.

3.1 CubeADCS Variants

CubeSpace currently has two main variants of the CubeADCS as standard offerings. This document will reference some aspects like headers, which are different between these two variants, and so they are briefly discussed below.

The first main variant is shown in Figure 1 and is suitable for integration in a PC104-type stack. Figure 1 shows a variant where all sensors and actuators are mounted external to the CubeADCS, but as per [RD2], CubeSpace also offers options where CubeWheels and CubeTorquers are mounted internal to the CubeADCS mechanical enclosure. These options are not shown.

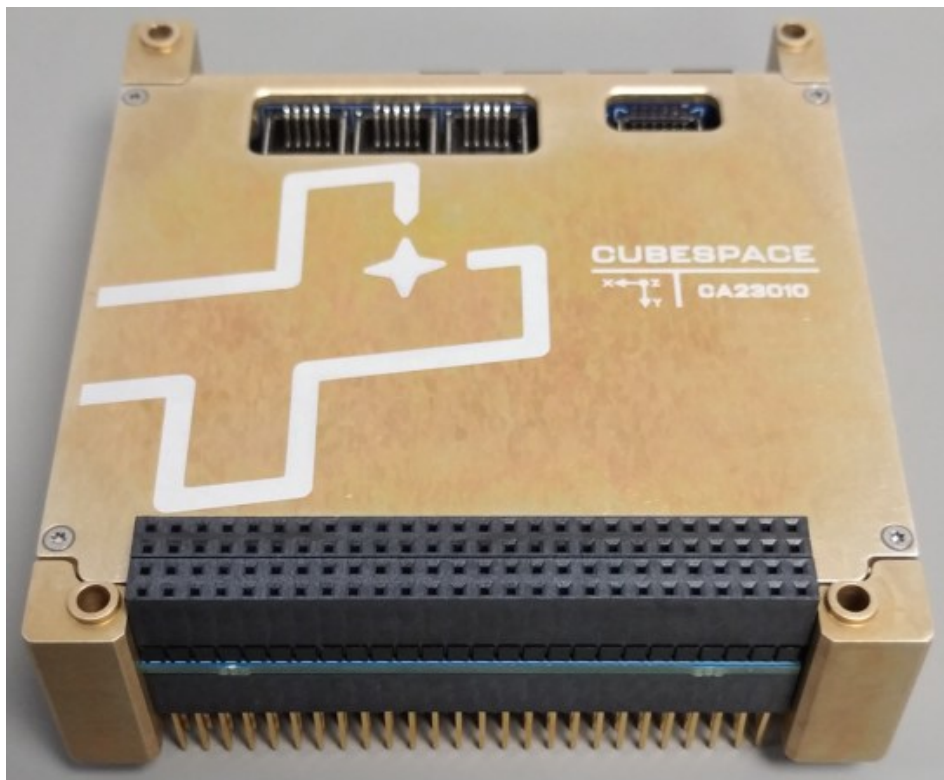


Figure 1: PC104 variant of CubeADCS

The second main variant is shown in Figure 2 and is suitable if a different form factor is required. The differences between these variants are purely of a mechanical nature.

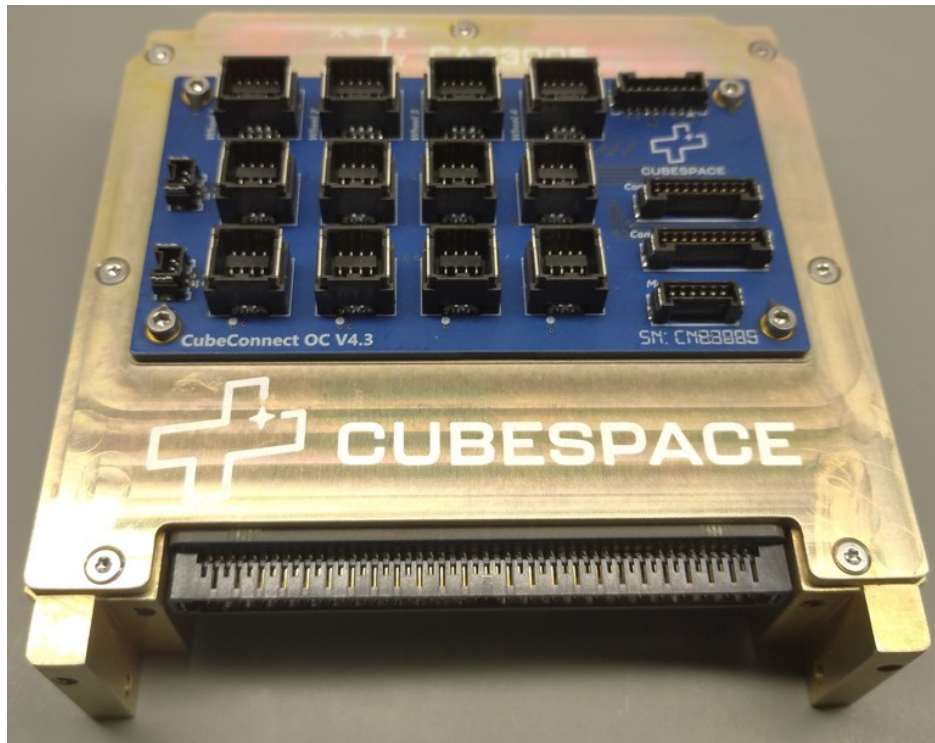


Figure 2: Backplane variant of CubeADCS

3.2 Ground Support Equipment (GSE)

The CubeADCS hardware is packaged in anti-static bags inside a protective shipment case. Remove the unit in a clean environment using gloves. Use a sturdy workbench that is covered with a clean anti-static mat. Any person working on the unit must be grounded with an anti-static wristband.

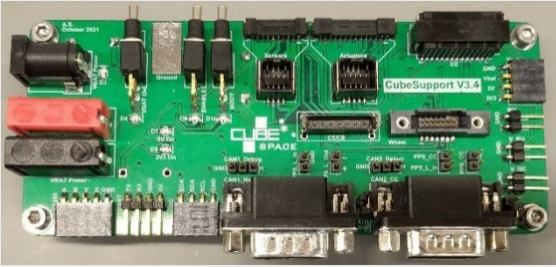
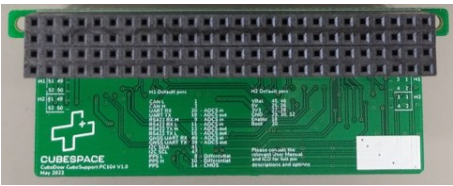
In addition to the CubeADCS itself, the package will contain some or all the following GSE:

- DC Power Cable
- CubeSupport PCB
- PCAN-USB (optional)
 - The latest drivers can be downloaded from:
 - <https://www.peak-system.com/Drivers.523.0.html>
- UART-to-USB cable (optional)
 - The drivers are automatically included in modern versions of Windows, however the latest drivers can be downloaded from:
 - <https://ftdichip.com/drivers/vcp-drivers/>
- CubeDoor-CubeSupport PC104 (for PC104 variants of CubeADCS)
- Flash drive

These GSE items are shown in Table 1. Remove all items from their anti-static bags and place them on the bench.



Table 1: Typical GSE items in CubeADCS package

GSE Item	Image
DC Power Cable	
CubeSupport PCB	
PCAN-USB: CAN Interface for USB	
UART to Serial USB cable	
CubeDoor-CubeSupport PC104	

3.3 Connecting to the CubeADCS

3.3.1 Power setup

Before connecting the bench power supply to the CubeSupport PCB, set the DC power supply voltage to 10.0 V – 16.0 V (**the recommended voltage to set for the health check is 12.0 V**, to exercise the system at the highest current draw). Set the current limit to at least 1.5A. Keep cables between the power supply and the CubeADCS short and thick to reduce series resistance.



A current limit set too low or a high series resistance in the power connection can cause the system to experience voltage dips when drawing large currents. These voltage dips may lead to unexpected behaviour and resets.



If the bundle includes actuators(CW5000 and larger) requiring a 28 V supply, additional power setup is required. This is discussed further in Section 6.

Before connecting the power source, ensure that all toggle switches on the CubeSupport PCB are in the OFF (down) position. (The toggle switches control the power to CubeADCS along with the Enable and Boot signals).

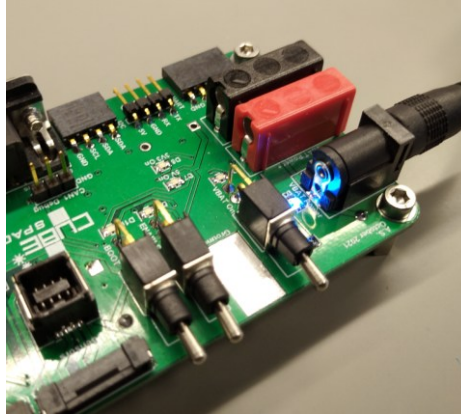


Figure 3: “VBAT On”, “BOOT” and “Enable” switches in the OFF position (down)



Note that the **Enable signal** might not be used for CubeADCS, depending on the order configuration. This signal might not be available on the CubeDoor or the CubeConnect and CubeADCS will therefore be enabled (powered) when the VBAT On switch is in the ON position, irrespective of the state of the Enable switch.

Similarly, the Boot signal might also not be used for CubeADCS, depending on the order configuration. In this case, the CubeADCS will never enter the low-level ROM bootloader embedded in the MCU, but the pre-loaded CubeSpace software Base-Bootloader (see Section 4.4) can be used to perform software updates.

The DC power cable has one black wire and another wire with a white dashed line on it. The wire with the white dashed line is the supply voltage, and the other wire is Ground, as shown in Figure 4:



Figure 4: Supply and Ground wires of the supplied DC power cable

Connect these two wires to the bench power supply. The other side of the cable must be plugged into the “VBAT Power” mating connector on the CubeSupport PCB.



With the DC cable connected to the CubeSupport PCB and the power supply switched on, the “VBAT Power” LED will turn on to indicate that power is present. The “VBAT on” switch can now be pushed to the ON position (up). The “VBAT On”, “5V On” and “3V On” LEDs on the CubeSupport PCB will turn on, as shown in Figure 5, indicating that power is present and that 3.3V is regulated on the PCB. If at this stage all four LEDs are not turned on, please contact CubeSpace.

Turn off the “VBAT on” switch by pushing it to the OFF (down) position and observe that the LED’s turn off (except the “VBAT Power” LED).

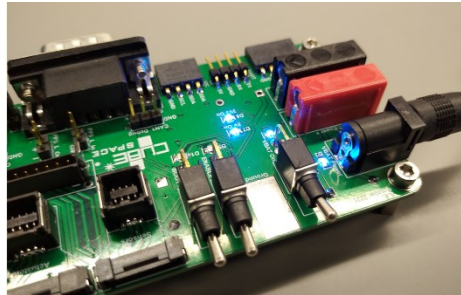


Figure 5: CubeSupport PCB with the DC cable connected, the “VBAT On” switch in the ON (up) position, and the LEDs turned on to indicate power-on condition.

The “VBAT On” switch can now be used along with the Enable switch (if the Enable line is connected to the CubeADCS) to disable and enable the CubeADCS.

3.3.2 Sensors and Actuators Connection Guide

The user can now go ahead with the connection of all the sensors and actuators as described in either this section **or the user specific CubeADCS addendum if supplied.**

3.3.2.1 External sensors and actuators

External sensors and actuators are connected to the CubeADCS via the CubeConnect board. The “Auto-Discovery” feature (as explained in section 4.5) allows the user to freely connect any sensor to any sensor port (ranging from “Sensor 1” to “Sensor 8”) and any actuator to any actuator port (ranging from “Actuator 1” to “Actuator 4”) on the CubeConnect board. In some cases, a sensor harness might be fitted with a connector for an actuator port, or vice versa, this does not change any behaviour to the system except where a product is able to plug in.

Exceptions are the sensors and actuators that are not connected to these ports (i.e., which have their own dedicated ports), and these are discussed in section 3.3.2.3 below.

Figure 6, Figure 7, and Figure 8 below shows the CubeConnect board for the two main CubeADCS variants, where the designator of each connection port can be seen. In the rest of this section “Sensor <x>” and “Actuator <x>” refers to these designators, unless stated otherwise.

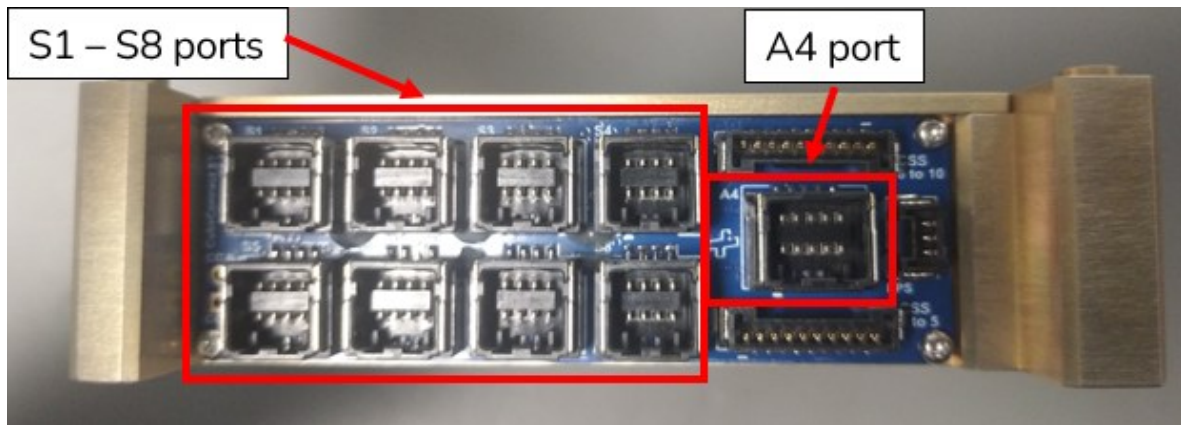


Figure 6: Connection Port Designators on CubeConnect (PC104 variant, rear view)

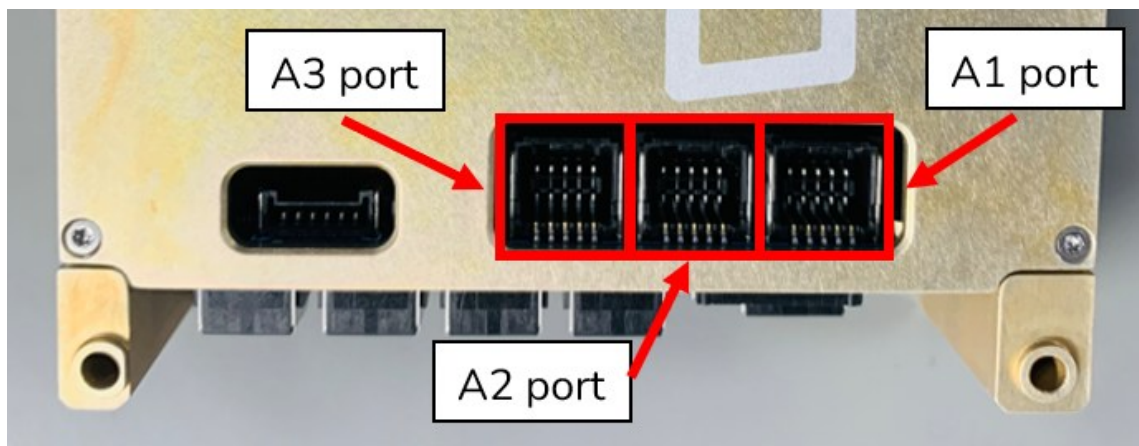


Figure 7: Connection Port Designators on CubeConnect (PC104 variant, top view)

Note that for PC104 variant of CubeADCS containing internal CubeWheels and CubeTorquers, the ports shown in Figure 7 will not be visible externally, as they will be used for connecting to these internal actuators.

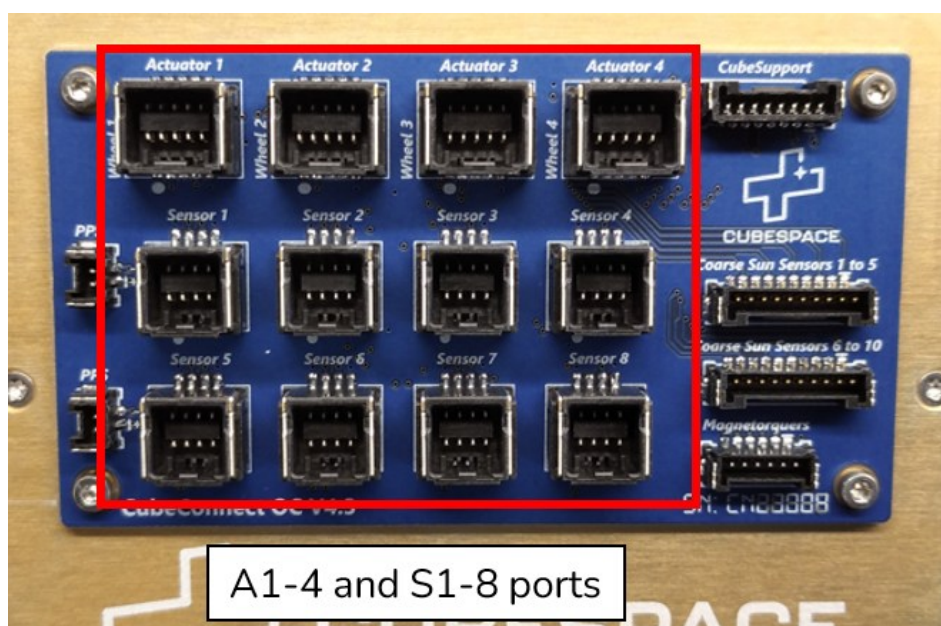


Figure 8: Connection Port Designators on CubeConnect (Backplane variant, top view)



3.3.2.2 Pulse Per Second (PPS) connectors

Some third-party sensors might require a PPS input. This signal is supplied to the CubeADCS by a GNSS module on the client satellite, and in turn passed on to these third-party sensors via a connected CubeNode. The harness required for this PPS input to the CubeNode will be manufactured accordingly and need to be plugged in to one of the connectors shown in Figure 9 and Figure 10.

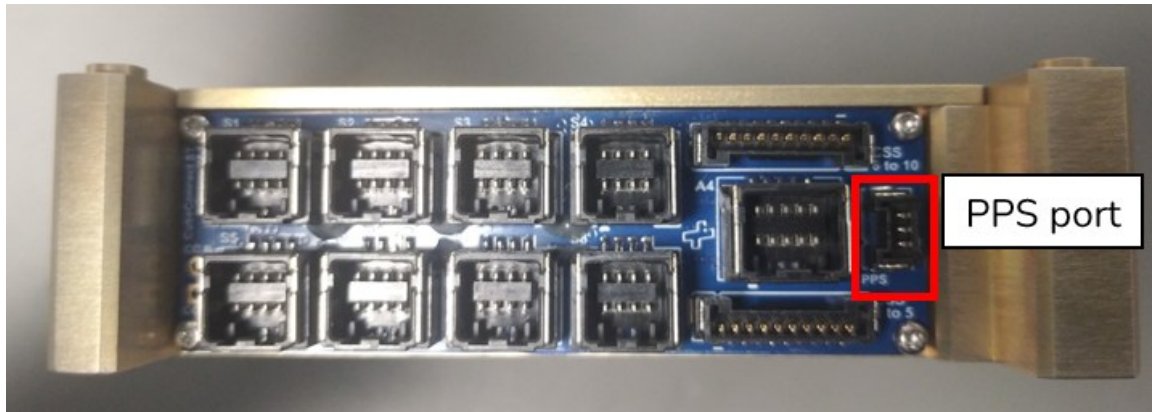


Figure 9: PPS Port on CubeConnect (PC104 variant, rear view)



Figure 10: PPS Ports on CubeConnect (Backplane variant, top view)

For the Backplane variant shown in Figure 10, the two PPS ports are interchangeable due to them being connected in parallel. The supplied CubeNode harness, if carrying a PPS plug, can be connected to either port. In this case, it will be easiest to plug the specific CubeNode into the Sensor 1 port, or Sensor 5 port since they are the closest to the PPS ports.

3.3.2.3 Sensors and actuators with dedicated connectors

3.3.2.3.1 Coarse Sun Sensors (CSS)

Coarse Sun Sensors are normally supplied by CubeSpace but can also be wired by the client. Figure 11 shows a CubeSpace-manufactured Coarse Sun Sensor, with a blue wire connected to the cathode and a white wire connected to the anode. The supplied Coarse Sun Sensor photodiodes are SLCD-61N8 from Advanced Photonix.



Figure 11: Course Sun Sensor manufactured by CubeSpace.

This section assumes that 10 sensors will be used.

The client typically can place each CSS on any facet of the satellite to optimally expose the Coarse Sun Sensors to the sun, given the satellite's typical mission modes. To allow for this freedom, each CSS and its 2-wire harness is supplied independently. A short 50 mm harness is included, with a Molex Pico-lock connector to simplify assembly into the satellite. This connector is polarized and prevents the CSS from being plugged in incorrectly.



Figure 12: Course Sun Sensor with harness.

Two CSS connector harnesses are supplied as shown in Figure 13, suitable to plug into the "Coarse Sun Sensors 1 to 5" and the "Coarse Sun Sensors 6 to 10" headers, shown in Figure 14 and Figure 15. The length of these harnesses must be specified by the client.



Figure 13: Course Sun Sensor connector harness (one of two).

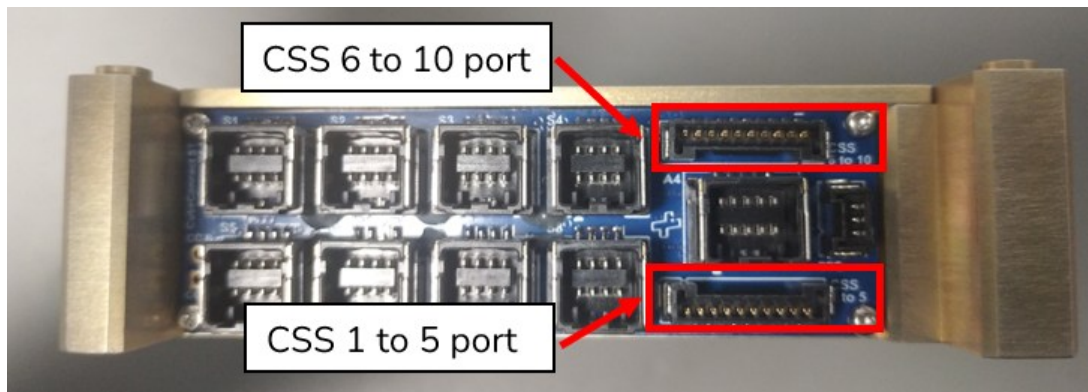


Figure 14: CSS Port Designators on CubeConnect (PC104 variant, rear view)

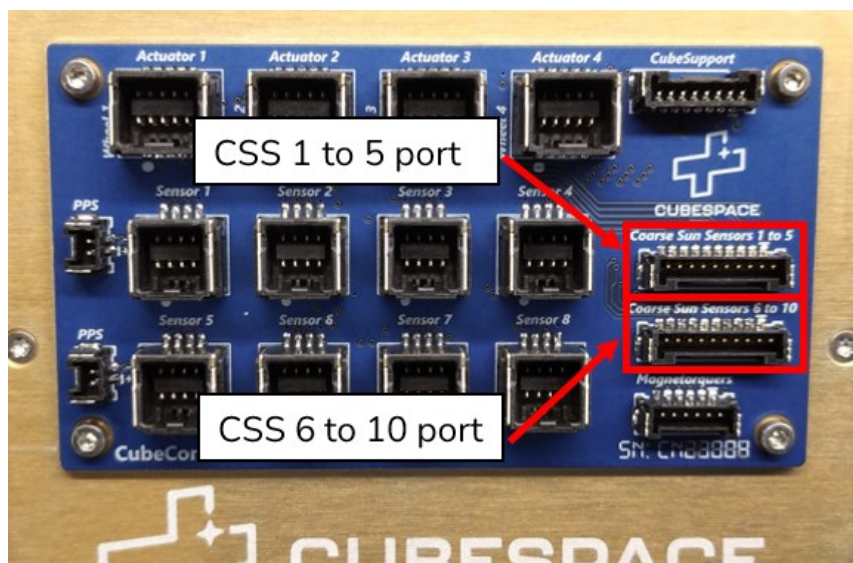


Figure 15: CSS Port Designators on CubeConnect (Backplane variant, top view)

Note: Each sensor will be automatically given an ID based on the position of the sensor in the array and the port it is connected to:

- Sensors connected to “Coarse Sun Sensors 1-5” port will be labelled CSS0 to CSS4 in telemetry exchanges.
- Sensors connected to “Coarse Sun Sensors 6-10” port will be labelled CSS5 to CSS9 in telemetry exchanges.
- The anode of each Coarse Sun Sensor is connected to the Ground plane of the CubeComputer.

Clients can select to have CubeSpace supply ten Course Sun Sensors including required harnessing, or to rather implement the course sun sensing photodiodes on their own. An example would be if solar panels with built-in course sun sensor photodiodes are used. CubeSpace can in this case supply open-ended Course Sun Sensor harnesses which need to be soldered to the client-implemented photodiodes during satellite assembly. The harnesses will be the same as in Figure 13 but without the white Pico-lock connectors.

3.3.2.3.2 Magnetorquers

The magnetorquer set is supplied with a standard 300mm long harness as shown in Figure 16. This harness plugs into the Magnetorquers header as shown in Figure 17 and Figure 18.



This section assumes that 3 magnetorquers will be used. In some configurations, two magnetorquers (in parallel) per axis might be used, in which case the supplied harness will be manufactured to accommodate this.

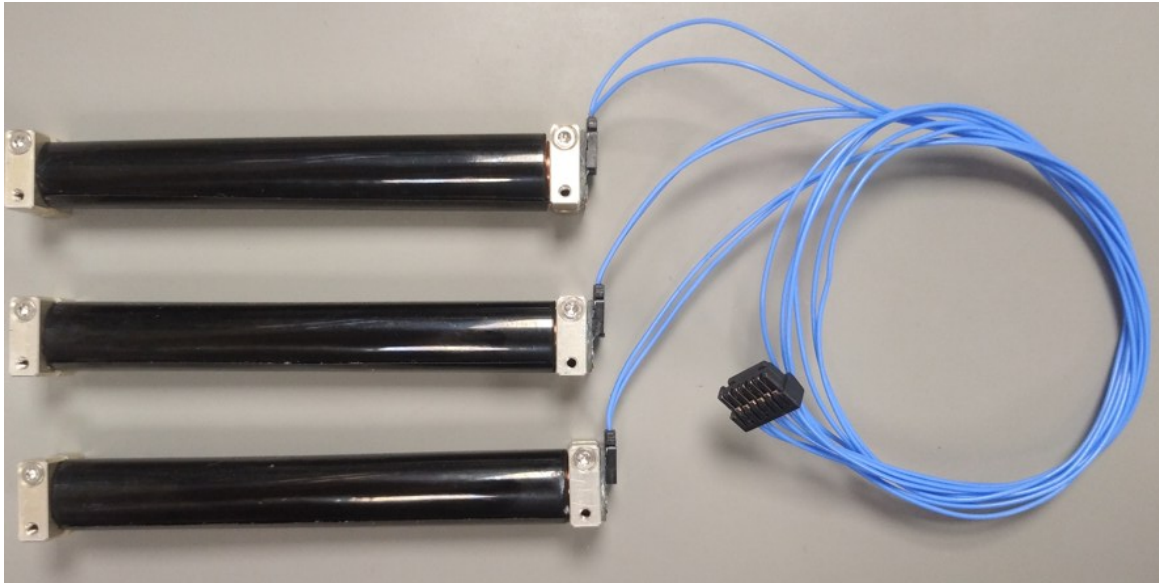


Figure 16: Magnetorquers and harness

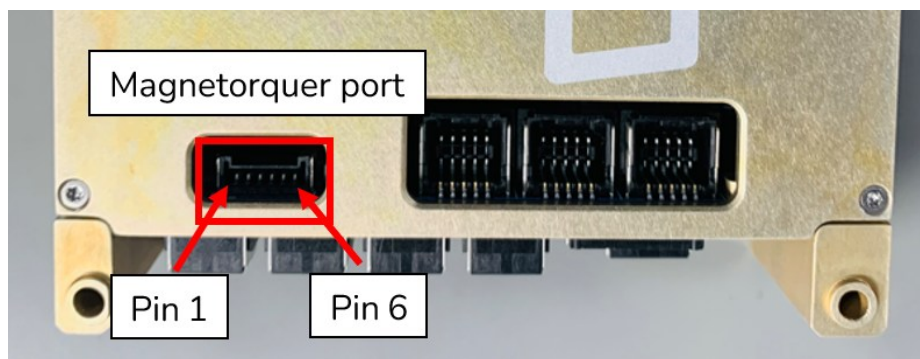


Figure 17: Magnetorquers port on CubeConnect (PC104 variant, top view)

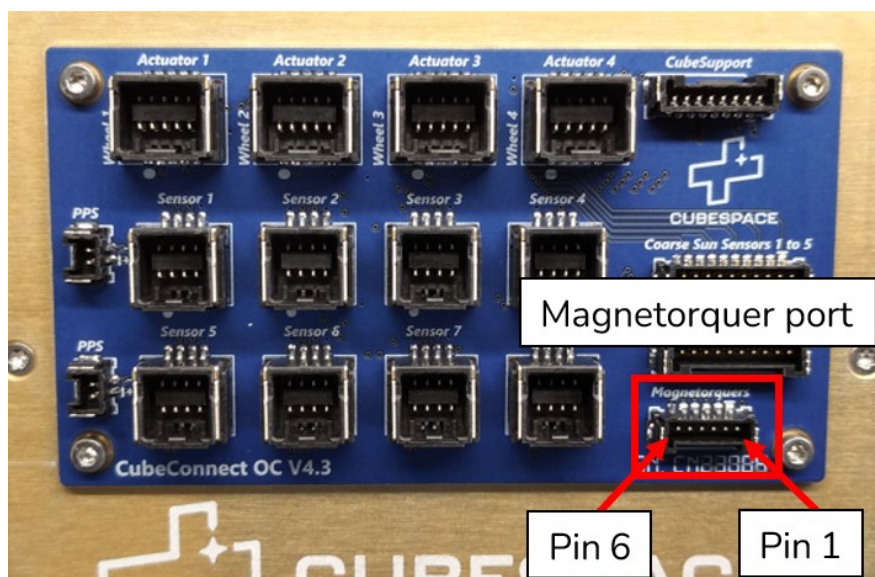


Figure 18: Magnetorquers port on CubeConnect (Backplane variant, top view)



Note: Each magnetorquer will be automatically given an ID based on the position of the magnetorquer in the array and the pins it is connected to:

- The Magnetorquer connected to pins 1 and 2 will be labelled MTQ0 in telemetry exchanges.
- The Magnetorquer connected to pins 3 and 4 will be labelled MTQ1 in telemetry exchanges.
- The Magnetorquer connected to pins 5 and 6 will be labelled MTQ2 in telemetry exchanges.

3.3.3 Communication setup

3.3.3.1 CAN and UART

CubeSupport PCB breaks out the communication interfaces to user-friendly headers. Although I2C and RS485/RS422 is also supported, the two preferred communication interfaces on the CubeADCS are CAN and UART. These channels are made available on the “CAN2 CC” and “UART” headers respectively, as indicated in Figure 19. A PC can interface with the CubeADCS through these channels on the CubeSupport PCB.

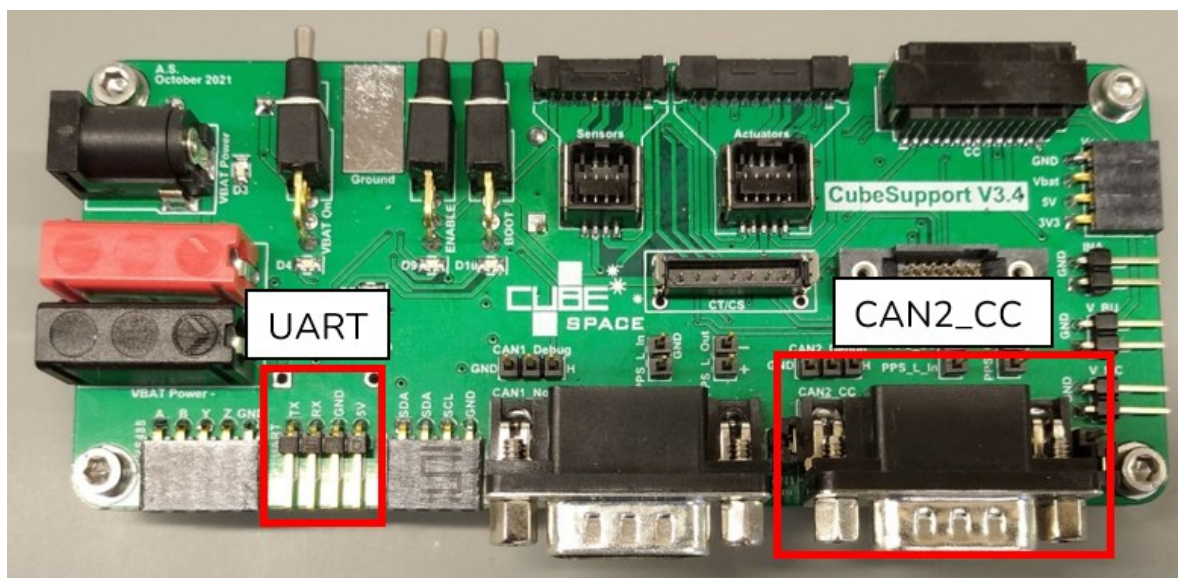


Figure 19: CubeSupport PCB “UART” and “CAN2 CC” headers indicated.

The PCAN-USB cable to connect to the “CAN2 CC” header and the FTDI USB-to-UART cable to connect to the “UART” header are supplied as part of the Ground Support Equipment. These two cables allow the PC to interface via the UART- and/or CAN communication lines.



Note that the UART to Serial FTDI USB cable is terminated in a three-pin female header containing the GND-, Tx- and Rx connections while the UART header on the CubeSupport PCB is a four-pin male header that contains an additional 5V pin that is not used. Ensure that this cable is correctly connected to the CubeSupport PCB, as indicated in Figure 20, to avoid any possible damage.

Connect the applicable communication interface/s as show in Figure 20.

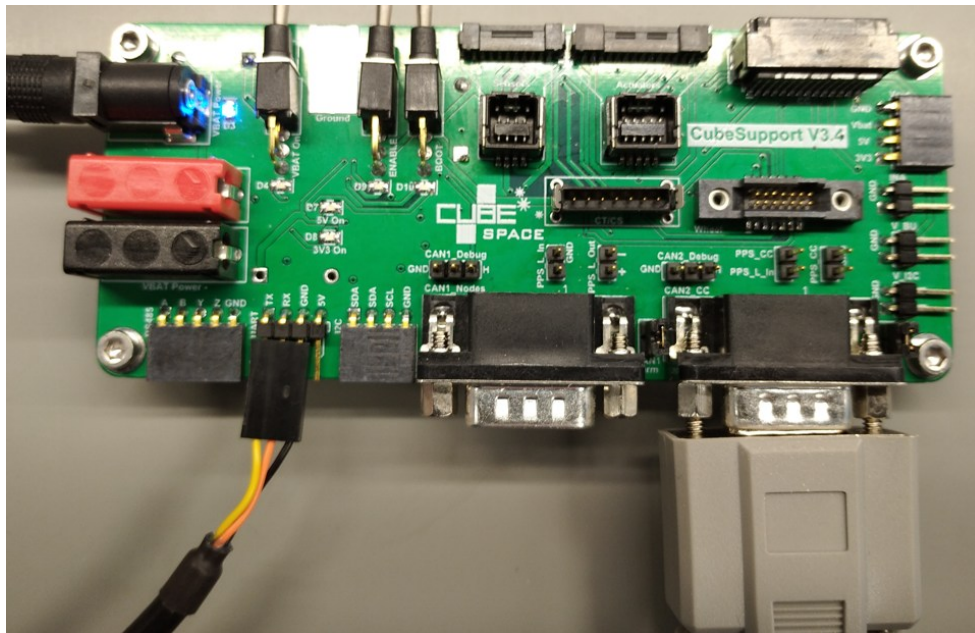


Figure 20: CubeSupport PCB with UART and PCAN-USB connections made.

To connect a PC104 variant of the CubeADCS to the CubeSupport PCB, the supplied “CubeDoor-CubeSupport PC104” PCB and included harness is used, as shown in Figure 21.

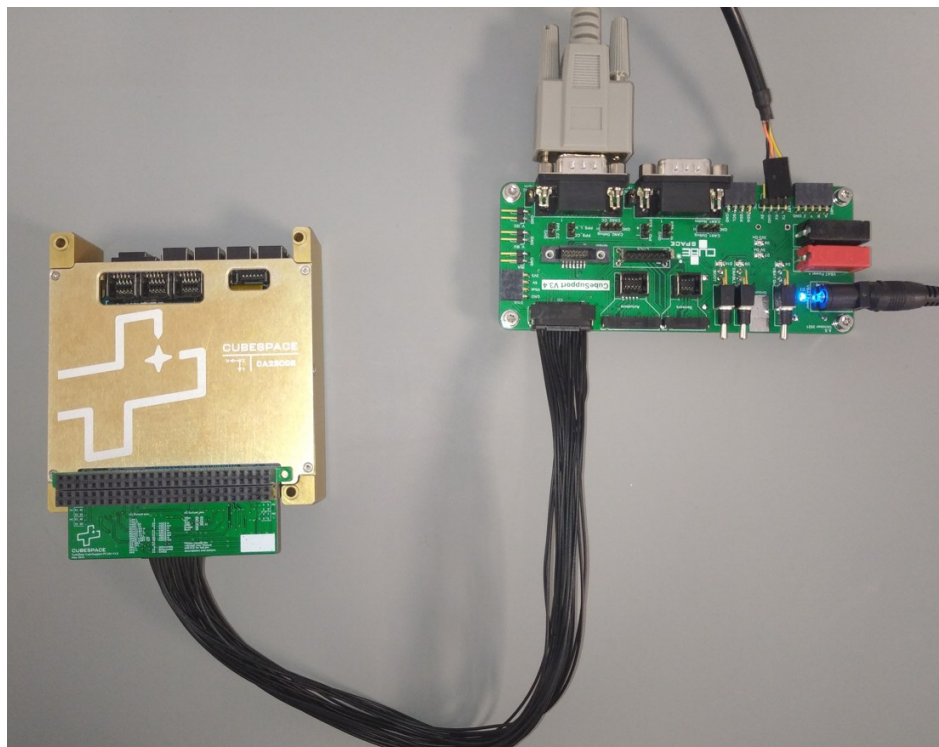


Figure 21: CubeSupport PCB connected to CubeADCS core (PC104 variant)

To connect a Backplane variant of the CubeADCS to the CubeSupport PCB, simply use the supplied “CubeDoor-CubeSupport” harness, as shown in Figure 22.

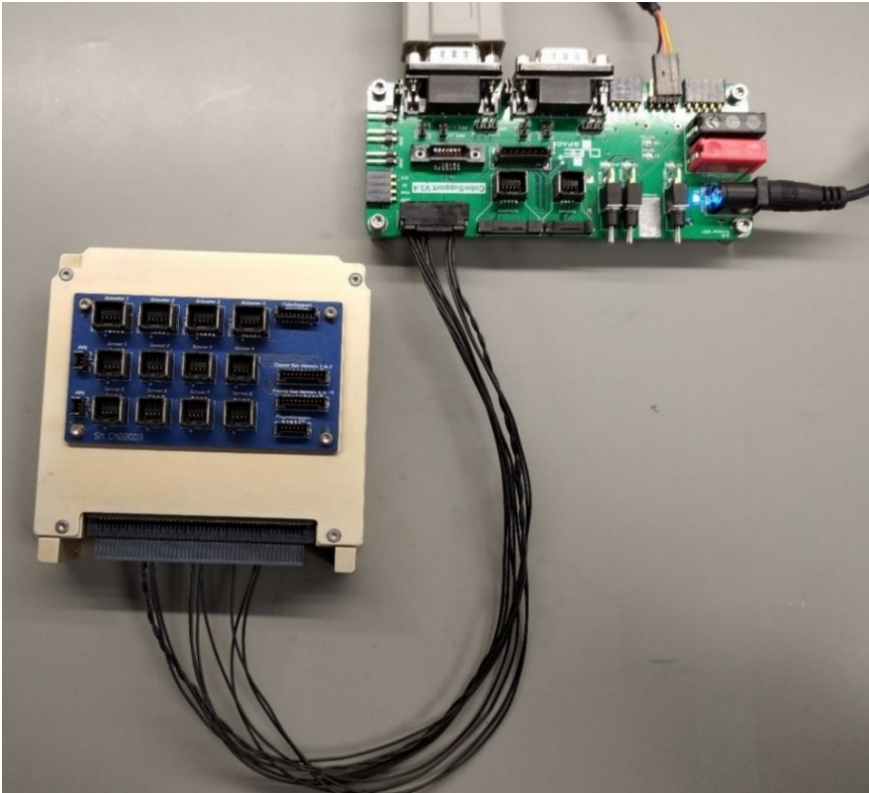


Figure 22: CubeSupport PCB connected to CubeADCS core (Backplane variant)

3.3.3.2 I2C

Both CubeComputer and CubeSupport PCB hardware support I2C communication. Therefore, if a client wishes to use I2C rather than, or in addition to, the main CAN and UART communication channels, the option is there to do so. The client can wire the I2C Clock, Data, and GND signals from a master device (such as an OBC) into the CubeSupport PCB's I2C header, shown in Figure 23.

During CubeADCS assembly, the I2C interface is tested with a custom cable, not supplied as part of a CubeSpace GSE package. The cable used is C232HM-DDHSL-0, manufactured by FTDI. The connections that are made between the FTDI cable and the CubeSupport PCB's I2C header, shown in Figure 23 and Figure 24, is summarized in Table 2 below. The CubeADCS makes use of Clock Stretching functionality.

Table 2: I2C Communications Check: C232HM-DDHSL-0 connection to CubeSupport PCB

Field	Pin Number	CubeSupport PCB connection	C232HM-DDHSL-0 Description
Green	4	SDA	SDA Input
Yellow	3	SDA	SDA Output
Orange	2	SCL	SCL Output
Purple	7	SCL	Clock Stretch Functionality Input
Black	10	GND	Ground

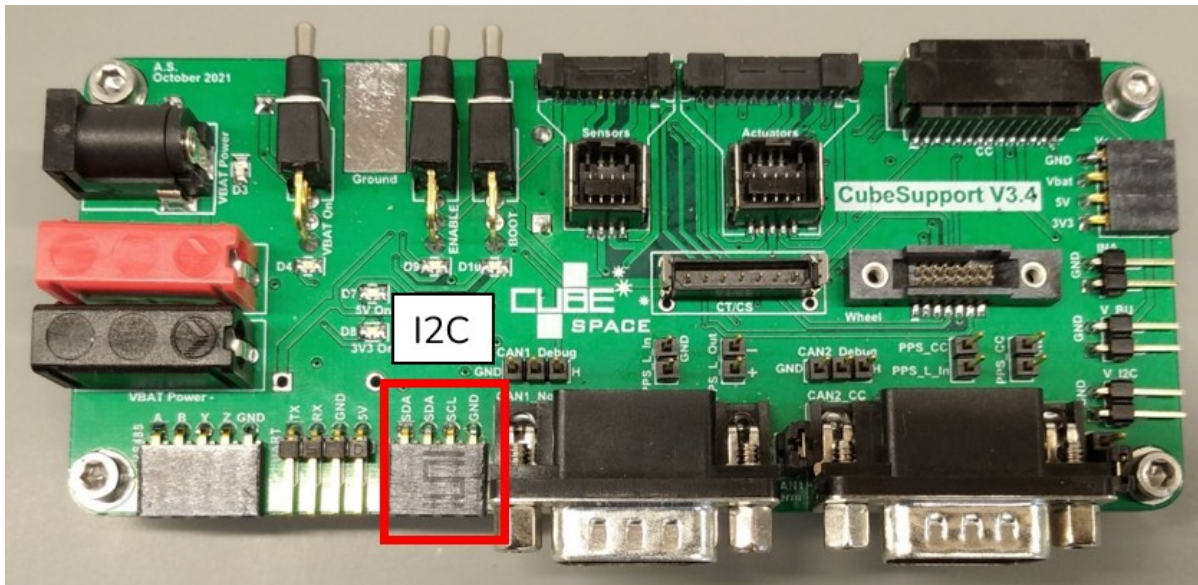


Figure 23: CubeSupport PCB I2C Pins.

Figure 24 shows how the orange and purple wires of the FTDI cable can be shorted together to share the SCL pin on the CubeSupport PCB. Note the solder bridges between the top and bottom rows of header pins.

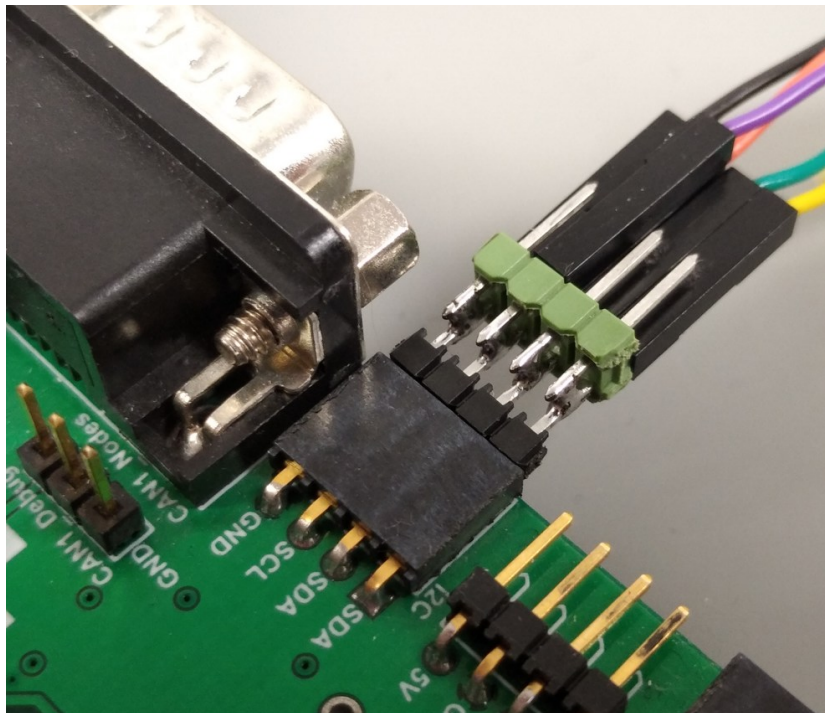


Figure 24: FTDI cable plugged into CubeSupport PCB I2C Pins.

3.3.3.3 RS422

The RS422 interface to the CubeADCS is tested with a custom cable, not supplied as part of a CubeSpace GSE package. This custom cable is essentially the same as the USB-to-UART FTDI cable from Table 1, with two 5 V-supplied MAX485 transceivers connected to translate the differential voltage levels (a high-level schematic is shown in Figure 25). Also, the [USB-COM422-PLUS1](#), manufactured by FTDI can be used as a standalone RS422 interface.

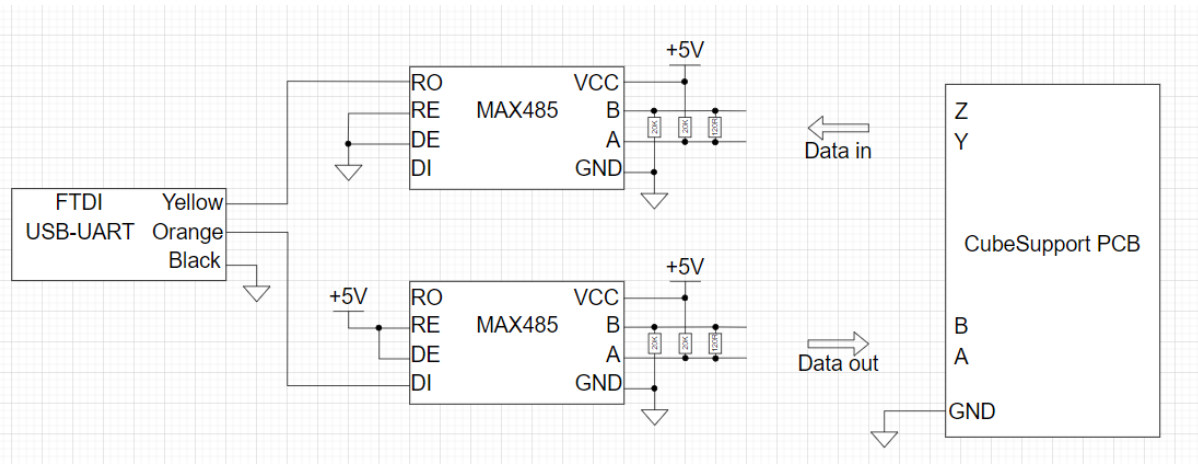


Figure 25: High-level schematic of cable used for RS422 testing (not supplied)

Figure 26 and Table 3 shows the RS422 connections between the CubeADCS and CubeSupport PCB.

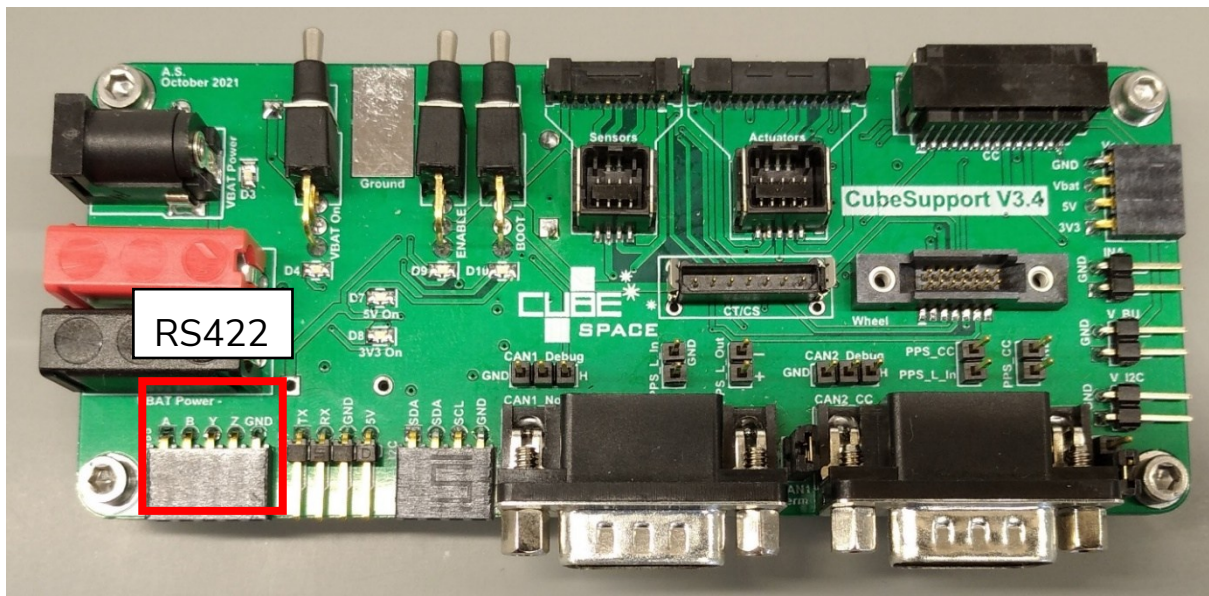


Figure 26: CubeSupport PCB RS422 Pins

Table 3: RS422 Communications Check: Connection to CubeSupport PCB

CubeSupport PCB pin	Description
A	RXD+ (CubeADCS input)
B	RXD- (CubeADCS input)
Y	TXD+ (CubeADCS output)
Z	TXD- (CubeADCS output)
GND	Ground

3.3.3.4 RS485

Instead of a four-wire, full-duplex RS422 interface, clients can select to rather use a two-wire, half-duplex RS485 interface. The connection between the CubeADCS and CubeSupport PCB will be the same as in Figure 26 and Table 3, expect that the A and B pins act as bidirectional input and output pins. Note that data transmission and reception on these pins cannot happen simultaneously, and the flow control of the



transceivers shown in Figure 25 needs to be managed with the RE and DE pins. The Y and Z pins are not connected on a CubeADCS configured for RS485 communication.

The RS485 interface to the CubeADCS is tested with a custom cable, not supplied as part of a CubeSpace GSE package. The cable used is USB-RS485-WE-1800-BT, manufactured by FTDI.

3.3.4 Communicating with the CubeADCS using CubeSupport PCB

Following the steps described earlier in Section 3.3, the CubeADCS unit should now be setup in such a way to allow the user to be able to power the unit and establish communication connection.

This section describes how the user can communicate with the CubeADCS from a PC using the *CubeSupport* application, a Windows-based software application also supplied with the CubeADCS. The *CubeSupport* application can be opened by running the executable file located in the software release bundle (see Section 4.6) under <...\sw-bundle-vxxx\tools\cube-support\cubesupport.exe>.

3.3.4.1 Powering on the CubeADCS, running of the bootloader, and starting the Control-Program

CubeSpace employs a software bootloader that runs when the CubeADCS is first powered on. This bootloader is discussed briefly in Section 3.4 and more in-depth in Section 4.4.

Establishing a connection over one of the CubeADCS's communication interfaces require that the user make the required hardware connections, and then power the CubeADCS on by toggling the "VBAT On" and "Enable" toggle switches of the CubeSupport PCB to the "On" positions. The CubeSupport's "Boot" switch must remain in the "Off" position otherwise the CubeADCS MCU will enter its embedded low-level ROM bootloader.

After power-on, the CubeADCS will run the software bootloader for a (configurable) pre-set period, normally 5 seconds, before jumping to the Control Program. If the *CubeSupport* application establishes a communication connection during this 5-second period, the CubeADCS will not jump to the Control-Program but instead keep running the bootloader, allowing for software updates to be performed as discussed in Section 4.6.

3.3.4.2 CAN Connection

To establish communication over CAN, follow the steps below:

- 1) Power on the CubeADCS by turning on the power supply, and ensure the "Boot" toggle switch on the CubeSupport PCB is in the "Off" position.
- 2) Toggle the "VBAT On" and "Enable" toggle switches on the CubeSupport PCB to the "On" position.
- 3) Wait for at least **5 seconds** after powering on the unit before continuing to allow the bootloader to jump to the Control Program.
- 4) Click on the dropdown arrow next to the  icon (top left) in the CubeSupport application and select PCAN Interface, as shown in Figure 27 and Figure 28. (There should be no need to change the default connection settings shown in Figure 29).

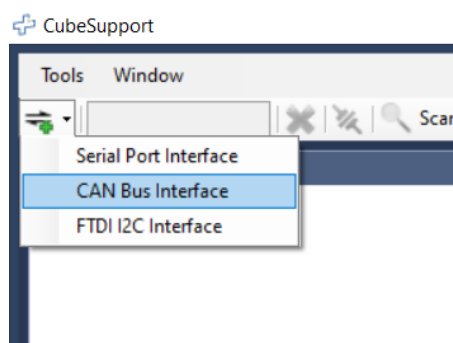


Figure 27: Connection dropdown

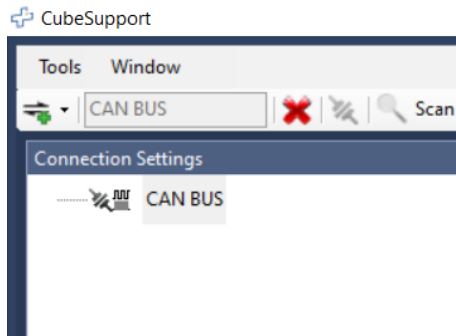


Figure 28: CAN interface selected

CAN Device	USB 1	
Maximum Burst (packets)	1	
Inter Burst Delay (ms)	1	
☒ High Resolution Timing		
☐ Use Cubesat Space Protocol (CSP)		
CubeSupport Address	240	
Usol Event Dst Port - CSP	58	
Usol TLM Dst Port - CSP	59	
Usol Event Dst Addr - CubeSpace	240	
Usol TLM Dst Addr - CubeSpace	241	

Figure 29: Default CAN settings

5) If applicable, ensure that the CubeSat Space Protocol is selected as shown in Figure 30:

CAN Device	USB 1	
Maximum Burst (packets)	1	
Inter Burst Delay (ms)	1	
☒ High Resolution Timing		
☒ Use Cubesat Space Protocol (CSP)		
CubeSupport Address	30	
Usol Event Dst Port - CSP	58	
Usol TLM Dst Port - CSP	59	
Usol Event Dst Addr - CubeSpace	240	
Usol TLM Dst Addr - CubeSpace	241	

Figure 30: Selecting the CubeSat Space Protocol checkbox

6) Click the connect button . The CubeSupport application will automatically attempt a connection to the CubeADCS by requesting the "Identification" telemetry packet. When the CubeADCS responds,



the CubeSupport application will recognise that it is connected to a CubeADCS and will display the appropriate interface, as shown in Figure 31.

- 7) Once the above step has succeeded, the communication between the CubeSupport application and the CubeADCS can be considered as successfully established.

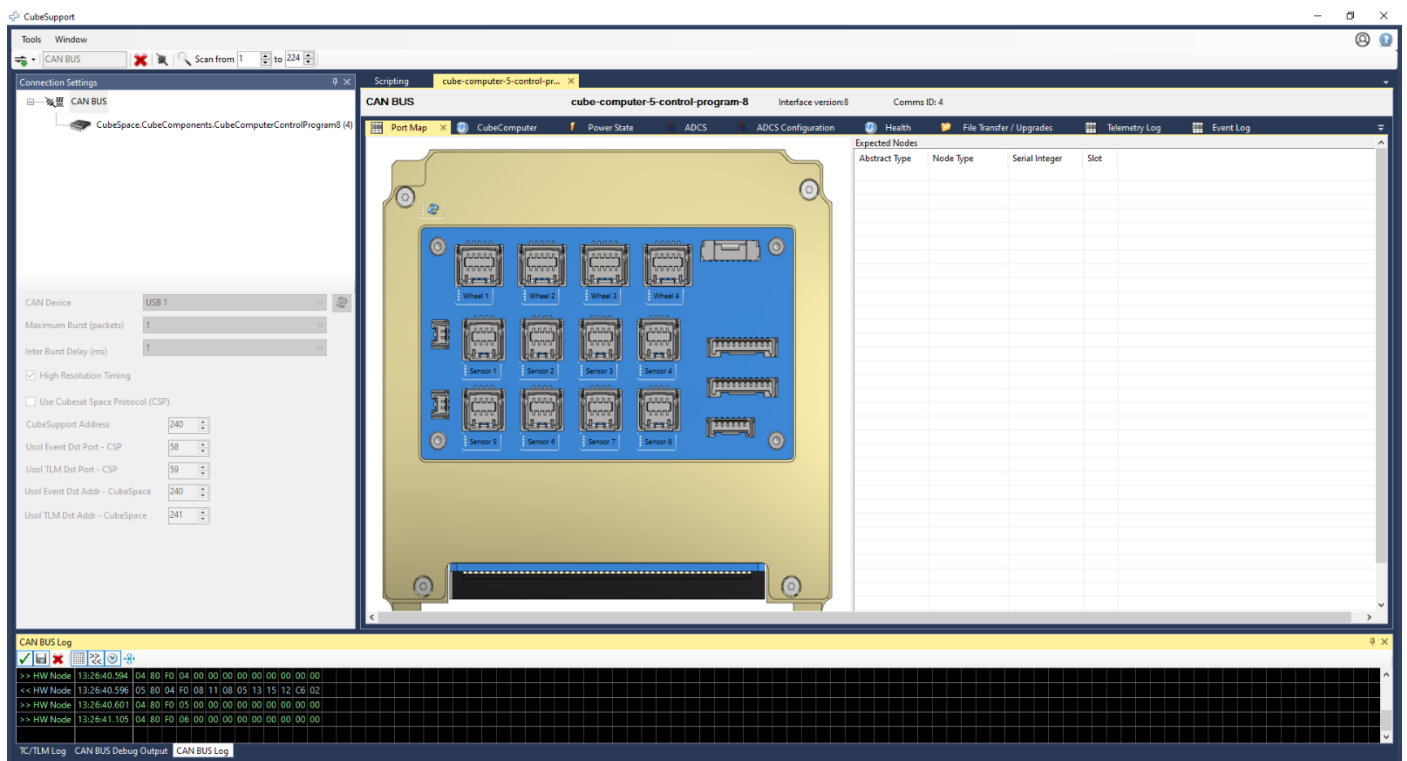


Figure 31: CubeSupport connected to CubeADCS firmware (Control-Program)

TIP: Once the CubeADCS has been “found”, click Cancel to stop CubeSupport from scanning the entire address range (see Figure 32). Alternatively, before connecting, set the scan range from 1 to 10:

Scan from 1 to 10 (Assuming that the CubeADCS has been configured with a CAN address between 1 and 10, which will be the case by default).

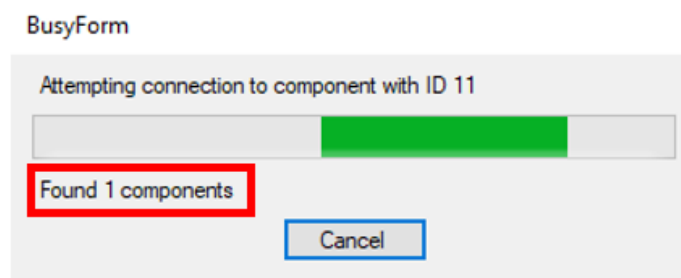


Figure 32: Cancelling address scanning once CubeADCS has been found

TIP: If CubeSupport displays the bootloader interface, it means the CubeADCS is still running the bootloader. Click “Disconnect” and then reset or power cycle the device (and wait 5 seconds before connecting again).

TIP: Figure 31 displays the Port Map tab. In most other tabs, fold the sections of the interface for easier navigation, as shown in Figure 33 below.

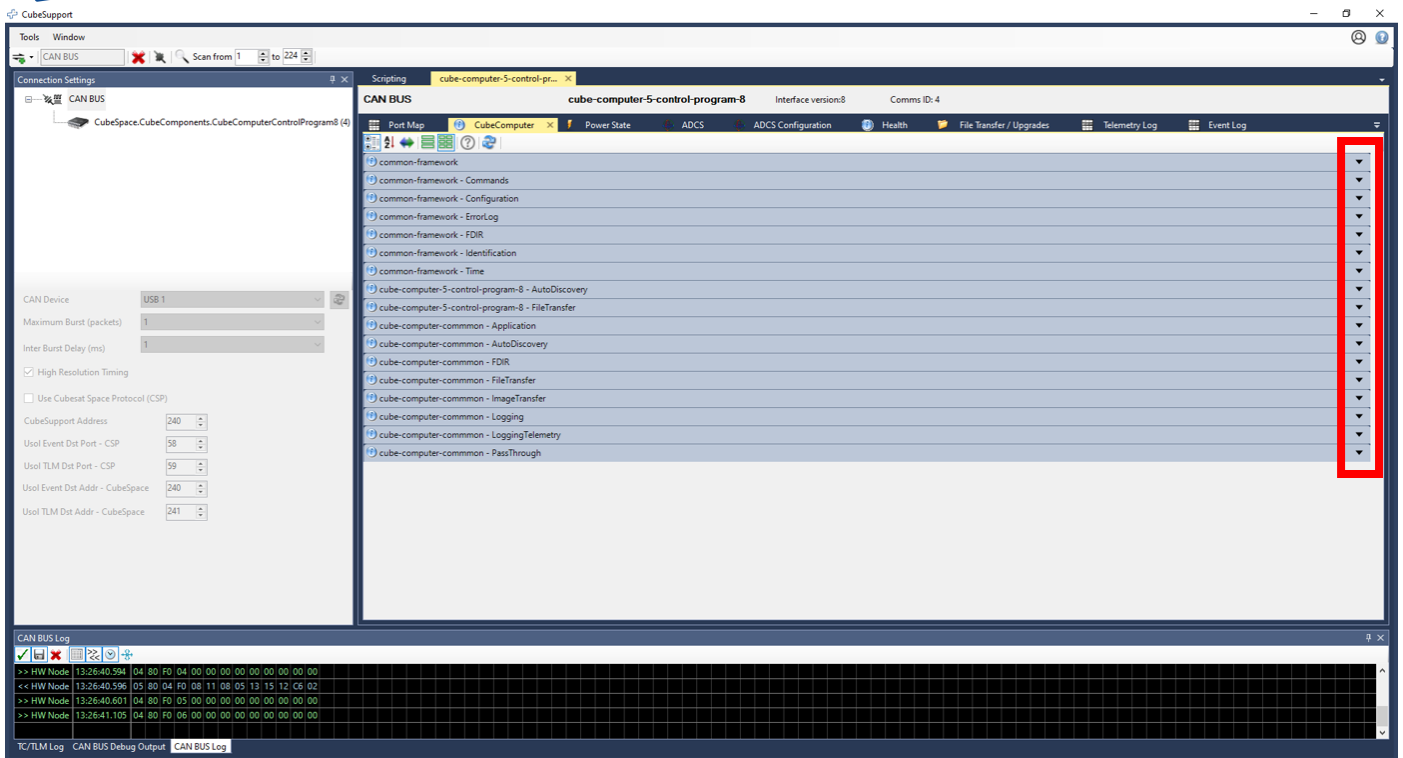


Figure 33: Folded sections of the CubeSupport CubeADCS interface

3.3.4.3 UART

To establish communication over UART, follow the steps below:

- 1) Power on the CubeADCS by turning on the power supply, and ensure the “Boot” toggle switch on the CubeSupport PCB is in the “Off” position.
- 2) Toggle the “VBAT On” and “Enable” toggle switches on the CubeSupport PCB to the “On” position.
- 3) Wait for at least **5 seconds** after powering on the unit before continuing to allow the bootloader to jump to the Control Program.
- 4) Click on the dropdown arrow next to the  icon (top left) in the CubeSupport application and select UART Interface, as shown in Figure 34 and Figure 35. Ensure the correct Serial Device is used (for UART communications, it is the supplied TTL232R-3V3 device). There should be no need to change the default connection settings shown in Figure 36, unless the client specifically requested a different UART configuration.

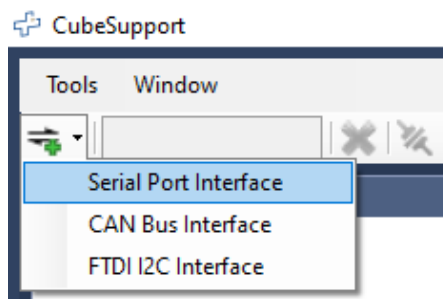


Figure 34: Connection dropdown

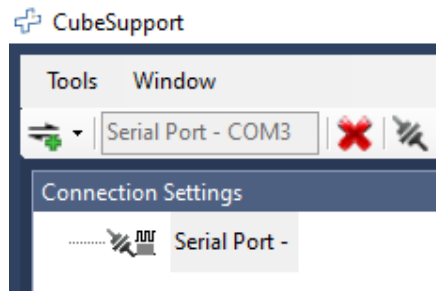


Figure 35: UART interface selected

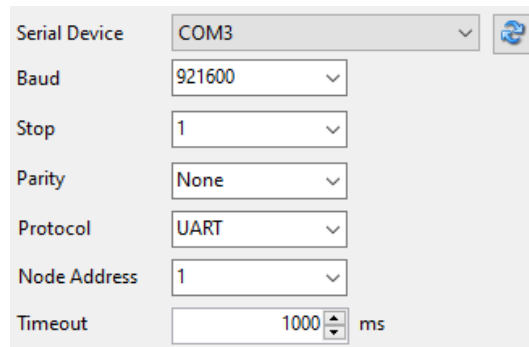


Figure 36: Default UART settings

- 5) Click the connect button . The CubeSupport application will automatically attempt a connection to the CubeADCS by requesting the “Identification” telemetry packet. When the CubeADCS responds, the CubeSupport application will recognise that it is connected to a CubeADCS and will display the appropriate interface, as shown in Figure 31.
- 6) Once the above step has succeeded, the communication between the CubeSupport application and the CubeADCS can be considered as successfully established.

3.3.4.4 I2C

The following steps can be followed to establish an I2C connection.

- 1) Make the connections shown in Table 2 and plug the I2C cable into the test PC’s USB port.
- 2) Power on the CubeADCS by turning on the power supply, and ensure the “Boot” toggle switch on the CubeSupport PCB is in the “Off” position.
- 3) Toggle the “VBAT On” and “Enable” toggle switches on the CubeSupport PCB to the “On” position.
- 4) Wait for at least **5 seconds** after powering on the unit before continuing to allow the bootloader to jump to the Control Program.
- 5) From the dropdown menu in the CubeSupport application, select “FTDI I2C Interface” as shown in Figure 37.

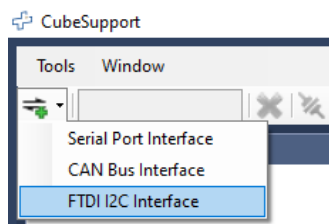


Figure 37: Selecting I2C interface

- 6) Ensure that the default settings are used, and that the correct Serial Device is selected. Figure 38 shows the default settings:

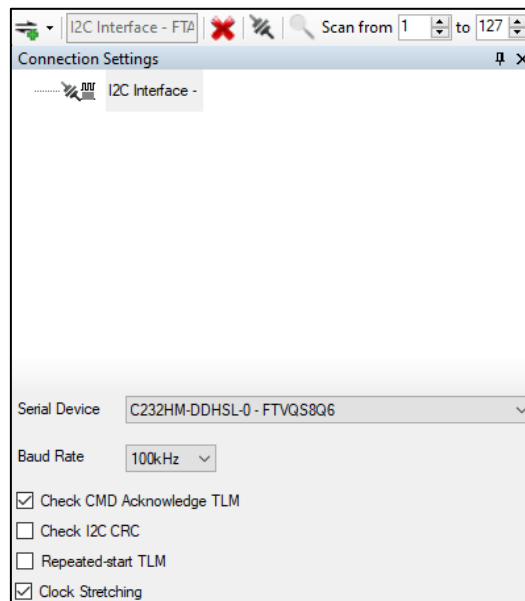


Figure 38: I2C default settings

- 6.1) Scan from: 1 to 127
- 6.2) Serial Device: C232M-DDHSL-0
- 6.3) Baud Rate: 100 kHz
- 6.4) Check CMD Acknowledge TLM: Checked
- 6.5) Check I2C CRC: Not checked
- 6.6) Repeated-start TLM: Not checked
- 6.7) Clock-Stretching: Checked
- 7) Click the connect button . The CubeSupport application will automatically attempt a connection to the CubeADCS by requesting the “Identification” telemetry packet. When the CubeADCS responds, the CubeSupport application will recognise that it is connected to a CubeADCS and will display the appropriate interface, as shown in Figure 31.
- 8) Once the above step has succeeded, the communication between the CubeSupport application and the CubeADCS can be considered as successfully established.

3.3.4.5 RS422/RS485

After connecting the RS422 or RS485 ground support cable as discussed in Sections 3.3.3.3 and 3.3.3.4, a connection can be established by following the exact same steps as for a UART connection, discussed in Section 3.3.4.3. The correct Serial Device setting still needs to be selected.

3.4 Brief Base-Bootloader description

After a reset, the CubeComputer Base Bootloader runs for a configurable period of 5 seconds. If a communications connection is established within this period using the CubeSupport application, it will automatically halt the bootloader and prevent the Control Program from running. The CubeSupport perspective for the bootloader is shown below.

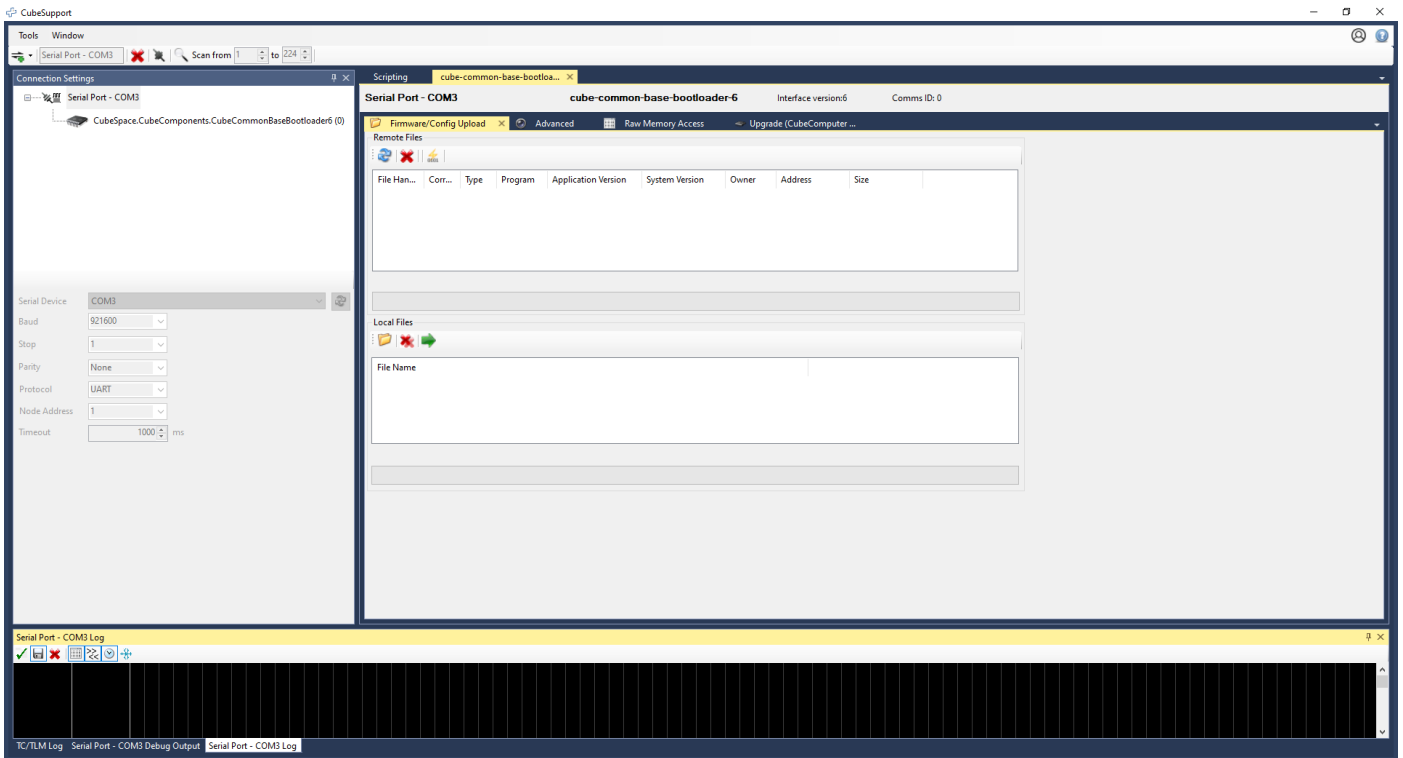


Figure 39: CubeComputer Base Bootloader perspective

To jump to the Control Program, the user can first refresh the list of applications in the CubeComputer's embedded flash memory using the “Request” icon .

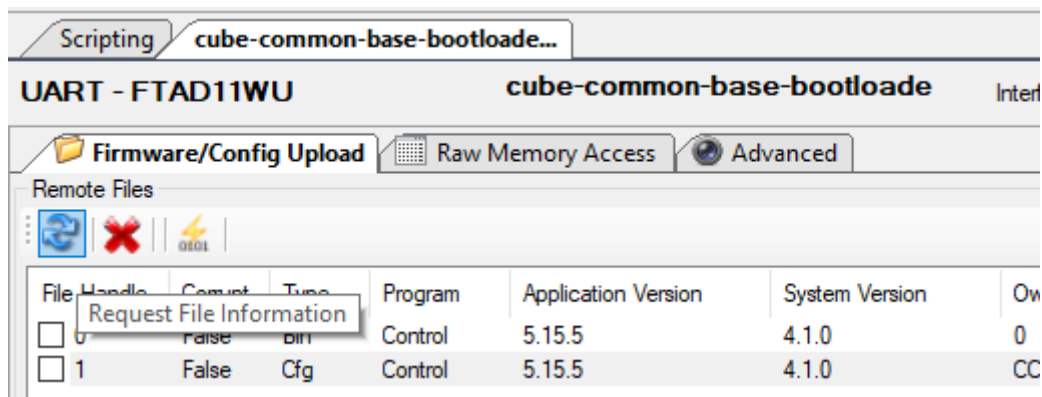


Figure 40: Refreshing the file list

Select the correct “Bin” file to run (there is usually only one Bin file along with a configuration file), and then click the “Jump To Application” icon .

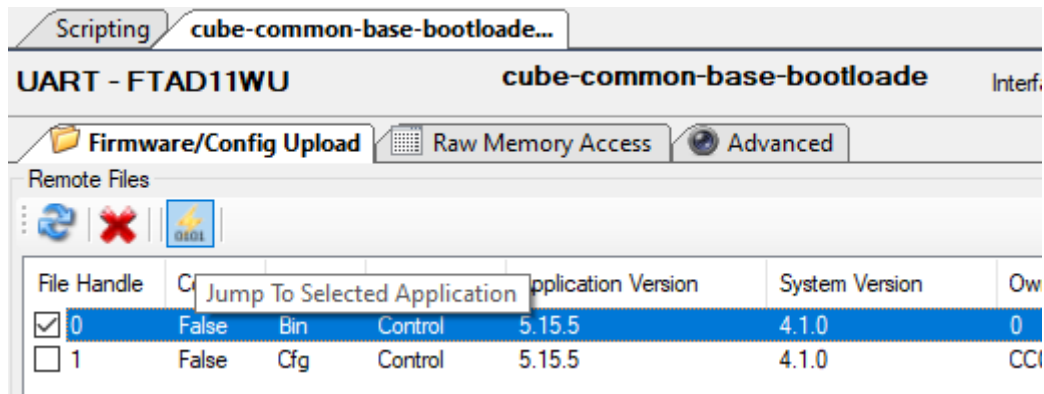


Figure 41: Jumping to the Control Program

The Advanced tab of the Base Bootloader also contains some configurable settings applicable to the bootloader itself, shown in Figure 42. The user can update these settings, like the CAN address or CSP functionality of the CubeADCS. When updated configuration settings are set, the Bootloader performs a software reset, and reestablishment of the communication interface might be required, using the new settings.



Note that for the CubeADCS and CubeComputer, the CAN1 interface is the internal CAN bus, used to communicate to connected sensor and actuator products. Please do not change the CAN1 settings.

CAN2 is the interface to the client OBC. The CAN2 address and CSP settings can be altered as required by the client.

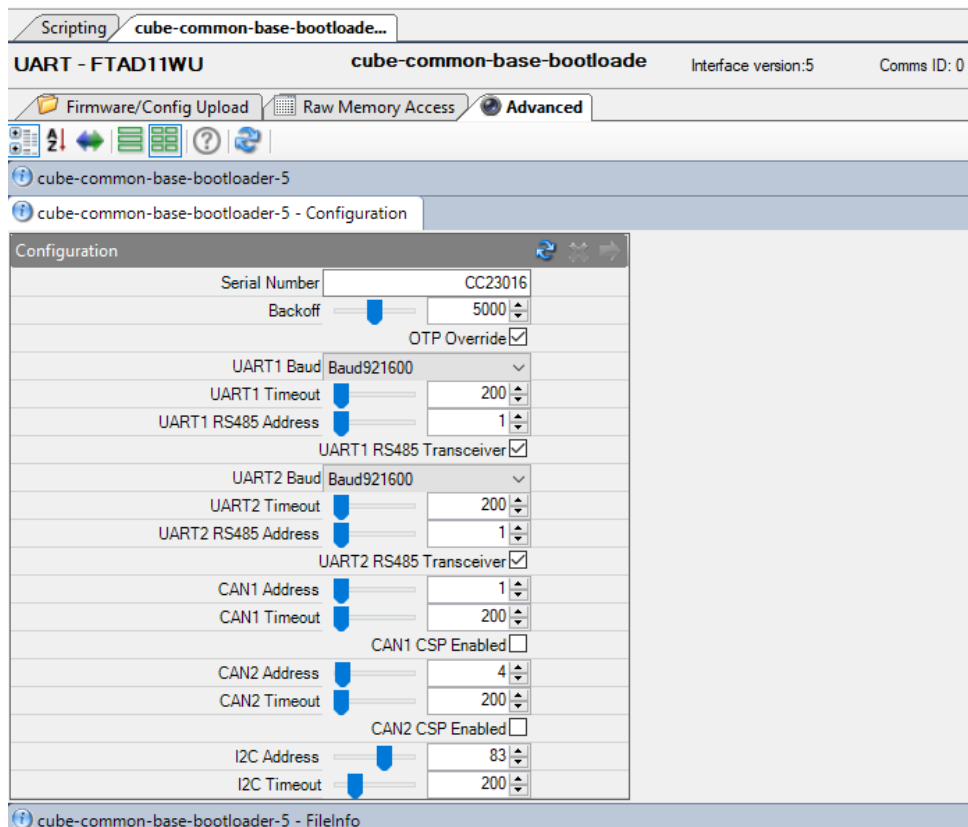


Figure 42: Base Bootloader configuration settings



By default, the Control Program inherits the communication settings from the Base Bootloader, allowing the user to easily update settings like the I2C address or UART baudrate.



Once the user has physically worked through this **“Getting started”** section, the user is requested to proceed with the system health check as defined in section 6.



4. Functions and Features

This section describes a select subset of functions and features offered by CubeADCS focussed on guiding the user to be able to successfully assemble and integrate the CubeADCS for a complete standalone Health Check before fitment into the supersystem / satellite.

For a summary list of all functions and features offered by the CubeADCS, the user is referred to the CubeADCS Product Description document (see [RD1]).

For a full list and description of all functions and features offered by the CubeADCS and for CubeADCS commissioning purposes, the user is referred to the CubeADCS Commissioning Manual document (see [RD5]).

Note that in some cases, specific Telecommands (TC) and Telemetry Requests (TLM) are included in the text following below. Such TC or TLM messages are highlighted for the user by the following style “Telecommand Name”. Telecommand and telemetry names are identified by their “Code Names” in the API documentation, with an optional friendly name in brackets, for instance: Version (Firmware Version). The code name identifies the particular telecommand or telemetry message in the API definitions [RD6], [RD7] and [RD8]. The same information is also available in the generated HTML documentation tables, distributed as part of the software package.

4.1 Identification

Each CubeProduct, including the CubeADCS, is programmed with a serial number, and details about its firmware.

The serial number of a CubeProduct is important as each CubeProduct may have specific calibration values associated with it, i.e. for a specific serial number. Further, knowledge of which CubeProducts and their associated serial numbers are utilised in a specific CubeADCS bundle, allows CubeSpace to accurately maintain configuration records for the CubeADCS, in turn allowing configuration specific support to the client when needed.

Similarly, knowledge of the firmware (versions) programmed onto a CubeProduct is important so as to allow correlation with the versions of firmware available in the software release packages, and to confirm firmware updates when done.

To check these details about a CubeADCS, the Identification, Identification2, SerialNumber and Version TLM can be requested. The content of these TLM frames can be referenced in [RD6].

4.2 Configuration

The CubeADCS makes use of configuration files to define the behaviour and setup for the firmware programs. This includes settings that define the hardware specific options, enabling specific communications interfaces, changing protocol, baud rates, which CubeProducts to expect as part of the Auto-Discovery feature etc.

Similarly, each CubeProduct also makes use of configuration files to define the behaviour of the respective CubeProduct.

Configuration files are supplied as part of software releases. The configuration files are binary coded versions of configuration settings and are not meant to be viewed or modified by users. For more information on uploading and changing configuration files, please consult reference [RD3].

The ADCS configuration also includes default values for a subset of settings that may be changed via telecommand. These commands also have matching “get” telemetry requests, to make it easier to modify single values in the larger command message (i.e., read config telemetry, modify a specific parameter only, and send command with updated config).



The CubeADCS configuration messages are listed in Table 4.

Table 4: Configuration messages (matching TCs and TLMs) for ADCS configuration settings.

Command/telemetry name	Content
ConfigDefaultModes (Default mode configuration)	Power-on default control mode and operational state
ConfigMount (Mounting configuration)	Mounting directions and mounting angles that define the mounting transforms between sensor/actuator coordinate frames and Spacecraft Body Coordinate (SBC) frame
ConfigMag0OrbitCal (Main magnetometer in-orbit calibration configuration)	In-flight calibration values for magnetometer identified as Mag0
ConfigMag1OrbitCal (Redundant magnetometer in-orbit calibration configuration)	In-flight calibration values for magnetometer identified as Mag1
ConfigAdcsController (ADCS controller configuration)	Controller gains and parameters
ConfigAdcsEstimator (ADCS estimator configuration)	Estimator system noise and sensor noise covariances Flags to control sensor use in estimators
ConfigAdcsSatellite (ADCS satellite configuration)	Satellite moments of inertia and products of inertia Vector directions to influence controller pointing behaviour
ConfigOrbitSatParams (Satellite orbit parameter configuration)	Orbital elements for satellite orbit
ConfigOrbitTargetParams (Target satellite orbit parameter configuration)	Orbital elements for target satellite orbit
ConfigNodeSelection (Node selection configuration)	Selection flags to control which sensors are to be used, if multiple sensors of the same type are present
ConfigActuatorRwl (Wheel configuration)	Wheel hardware configuration (wheel sizes and parameters, and mounting selection – pyramid vs. three orthogonal wheels)
ConfigActuatorMtg (Magnetorquer configuration)	Magnetorquer hardware configuration (torquer strength and maximum on-time)

Changes that are made to the configuration via telecommand are only temporary – until the CubeADCS is reset or power cycled. To persist config changes to non-volatile memory, a ConfigPersist telecommand must be sent.

4.3 Power management

The ADCS Control Program (CP) running on the CubeComputer, controls power to the nodes (including the sensors and actuators they are connected to) using several enable lines, connected to the CubeComputer MCU. The enabled state of CubeSense, CubeWheel(s), CubeNode, CubeStar and the CubeMag nodes are changed via the CP by sending a PowerState telecommand to the CP (see [RD4] and [RD7]).

The power selection for each node is a PowerSelect enumeration value and is usually PowerOff (0) or PowerOn (1).



4.4 Bootloader

CubeSpace provides its own in-house developed software bootloader application, called Base-Bootloader, for each CubeProduct which executes before the Control Program. Both the Base-Bootloader and Control Program reside in the internal flash memory of each CubeProduct's MCU, including that of the CubeADCS. The Base-Bootloader has slightly expanded functionality compared to the MCU's embedded low-level ROM bootloader, which is not used under normal circumstances except to initially program the MCU with the Base-Bootloader during CubeProduct assembly. However, clients can still select to have access to this ROM bootloader by way of the regular UART pins in addition to a BOOT pin on the CubeADCS's OBC interface. It is recommended that clients connect this BOOT pin and UART pins to the satellite OBC in case the base-bootloader somehow gets corrupted in orbit.

The Base-Bootloader is used in favour of the previous, now-deprecated, CubeSpace software bootloader, called Flash-Bootloader, in order to improve bootloader simplicity and robustness. The Base-Bootloader is simply used to update the Control Program, configuration files, and embedded option bytes of the CubeComputer itself. The CubeComputer Control-Program has the additional complex functionality to update the bootloaders and flight software of the CubeProducts connected to the CubeADCS.

This bootloader interface is also supported by the CubeSpace's CubeSupport application.

The Identification telemetry frame can be used to identify whether the Base-Bootloader or the Control Program is currently running. The ProgramType is a numeric value that is unique to the specific application. The CubeADCS firmware and command and telemetry interface is described in detail in [RD4], the accompanying interface definition files [RD6], [RD7] and [RD8], as well as the HTML documentation tables that are generated from the latter references, distributed as part of the software bundle.

For information about the bootloader application(s), refer to [RD3].

4.5 Auto-Discovery

The ADCS includes an "Auto-Discovery" feature which is implemented as part of the CubeADCS Control-Program. "Auto-Discovery" refers to the CubeADCS's ability to automatically determine which CubeProducts are present, as well as which port each CubeProduct is connected to, without user intervention. This feature allows the user to connect any CubeProduct to the CubeConnect port that best suits their mechanical design, without the need for any changes to the control program firmware configuration. The CubeProducts that are discovered in the system are compared to the CubeProducts that are expected in the ADCS system.

Auto-Discovery consist of two parts: The port validation step and the discovery step. During port validation, the CubeADCS will check if the CubeProducts are still connected to the same ports that they were previously discovered on. Only if port validation fails will Auto-Discovery take place. This two-step procedure happens once when the Control-Program boots. If a CubeProduct is moved to a different port, the CubeADCS must be reset so that Auto-Discovery will be performed.

An Auto-Discovery is successful if the discovered CubeProducts match the expected CubeProducts. The expected CubeProducts are listed as a Control Program configuration item per CubeADCS system. This configuration is managed by CubeSpace.

The BootStatus telemetry can be queried to determine if port validation or Auto-Discovery is in progress.

4.5.1 Diagnostics

All diagnostic data relating to Auto-Discovery is exposed via the PortDiagnostics telemetry. This telemetry contains global errors, as well as errors for each port.

The global errors are used to signal to the user that at least one port has experienced an error. These errors can be checked first to determine if any error has occurred, only then is it required to check the individual port errors.



Table 5: Port Diagnostics – Global Errors

PARAMETER / FIELD	DESCRIPTION	POSSIBLE CAUSES
<Global Error Code>	If this field has a value of 0, no errors occurred and there is no need to check for any other errors. If this field is non-zero, the user must check the global error flags as well as the individual port errors to diagnose the issue.	N/A
<None Expected>	Set if the expected nodes list is empty.	Note: This is not strictly considered an error, as the user may wish to integrate the CubeComputer in to a FlatSat design without any nodes connected.
<None Discovered Error>	Set if zero nodes are discovered.	Nothing is connected.
		All nodes have an incompatible bootloader after upgrading the CubeComputer bootloader and need to be upgraded.
<Expected Match Error>	Set if the discovered nodes do not match the expected nodes.	At least one node has an incompatible bootloader.
		At least one node from the wrong CubeADCS bundle has been connected.
		At least one node from the correct CubeADCS bundle has not been connected.
<Address Assign Error>	Set if at least one node could not be assigned an address	The target node potentially experienced an internal error and could not complete the command. Please perform Auto-Discovery again and report the issue.
<Abstract Type Error>	Set if the expected nodes list contains duplicate abstract types for at least one node.	N/A
<Config Persist Error>	CubeComputer failed to persist the discovered nodes structure.	Write protection is possibly active. Please report to CubeSpace.

Table 6: Port Diagnostics – Individual Port Errors

PARAMETER / FIELD	DESCRIPTION	POSSIBLE CAUSE(S)
<Error Code>	If this field has a value of 0, no errors occurred and there is no need to check for any other errors. If this field is non-zero, the user must check the port error flags as to diagnose the issue.	Note: If the global error code has a value of 0, all port errors should be ignored. This field will be non-zero if there is not a node connected to it, with the “Comms Error” flag set. This does not necessarily imply an Auto-Discovery error and only serves to provide information.
<Unexpected Error>	Set after Auto-Discovery if the node type connected to a port is not listed in the expected nodes.	This node does not belong with this CubeADCS bundle.
<Comms Error>		There is nothing connected to this port.



PARAMETER / FIELD	DESCRIPTION	POSSIBLE CAUSE(S)
	Set during Port Validation and Auto-Discovery if no communication was possible on this port.	The node connected to this port is programmed with an incompatible bootloader.
<Node Type Match Error>	Set after Port Validation if the node type of the node connected to the port no longer matches the node type that was previously discovered on this port.	The node has been removed.
		The node has swapped ports.
<Serial Number Match Error>	Set after Port Validation if the serial number integer of the sensor/wheel connected to the port no longer matches the serial number integer that was previously discovered on this port.	The node has been removed.
		The node has swapped ports.
<Address Assign Error>	Set during Auto-Discovery if the node connected to this port could not be assigned an address.	The node potentially experienced an internal error and could not complete the command. Please perform Auto-Discovery again if the problem persists, report the error.
<Abstract Type Error>	The node connected to this port has an abstract type assigned to it that is also assigned to at least one other node.	N/A

4.5.2 Timing

The duration of Port Validation and Auto-Discovery is dependent on the state of the system and may vary. This section will show the expected durations for each step.

Port Validation Duration

Possible scenarios are listed below:

- a) The expected nodes structure matches the discovered nodes structure and all nodes are still present and healthy (expected program flow):
 -
 - In this scenario, each of the discovered nodes will be queried once to confirm it is still present at the expected port and address. The duration depends on the number of discovered nodes.
 - Each node is allowed 100 milliseconds to boot before the query. The query can be assumed to return immediately.
 - e.g., 8 discovered nodes will result in a duration of 800 milliseconds.
 -
- b) The expected nodes structure does not match the discovered nodes structure:
 -
 - In this scenario, Port Validation will return immediately, as there is no need to query any of the nodes. It is already known that an Auto-Discovery must be performed.
 -
- c) The expected nodes structure matches the discovered nodes' structure but at least one node is no longer present or is faulty:
 -
 - In this scenario, each node will be queried as in a). However, when a missing node is queried, it will not respond. There is a timeout of 5 milliseconds on each query. Leading to an additional 5 milliseconds for each missing/faulty node. An attempt is made to query all discovered nodes regardless of the number of missing/faulty nodes.



Auto-Discovery Duration

Auto-Discovery requires that all possible ports and all possible addresses be scanned. There are 12 possible ports and 255 possible addresses. During Auto-Discovery, each discovered node is assigned an address sequentially in the order it is discovered. This results in a faster Auto-Discovery operation after the first successful one. A newly upgraded node will always have an address of 1. This results in a faster Auto-Discovery operation after a full CubeADCS upgrade.

Possible scenarios are listed below:

- a) The CubeADCS is newly upgraded and running for the first time, and all nodes are connected and healthy (expected program flow):
 - In this scenario, all ports are scanned, each starting from address 1. All nodes will have an address of 1. Therefore, each port with a connected node can be expected to consume only the boot time of the node. Each node is allowed 100 milliseconds to boot.
 - Each port that does not have a node connected will consume the boot time plus a 5-millisecond timeout for each of the queries for all 255 possible addresses.
 - e.g., If 8 nodes are connected, 800 milliseconds will be consumed for the connected nodes. Each of the remaining (4) unconnected ports will consume 1375 milliseconds for boot and address scanning, totalling 5500 milliseconds.
 - The total duration for Auto-Discovery will be 6300 milliseconds.
- b) An Auto-Discovery has previously been performed and all or some nodes have been assigned an address:
 - In this scenario, some or all nodes will have addresses assigned and will no longer have the default address of 1. This results in a slightly longer address scan on ports that have nodes connected. i.e., Referring to a), a node connected to port *Sensor 2* may have been assigned an address of 5, meaning addresses 1-4 will need to be scanned before a response is received on the port. Ports that do not have nodes connected follow the same principle as in a).
- c) There are no nodes connected or all nodes are faulty (worst case):
 - In this scenario, all addresses are scanned for all ports resulting in a maximum duration of 16500 milliseconds.

Note: This list is not exhaustive and relies on the discretion of the user as to the current state of connected nodes.

4.6 Programming and Updating of CubeADCS

The user does not have access to the low level (SWO/JTAG) programming pins of each MCU in the CubeADCS bundle. Instead, all CubeProducts are pre-loaded with the Base-Bootloader that allows reprogramming of its Control Program firmware.

The Base-Bootloader application is programmed in the start position of Flash memory of each CubeProduct's MCU. This Bootloader provides an interface consistent with the control program and allows for easy update of the Control Program and configuration. This interface is also supported by the CubeSpace's CubeSupport application. This interface and update method works for all communication interfaces and no additional- or special hardware connections are required. This makes this method to update the CubeADCS the preferred method in orbit since it can be easily implemented by the OBC.

When to Update

The user will need to reprogram or update the CubeADCS for the following scenarios:

- a) A next version of a control program for a unit / units (Update)
- b) A next version of configuration for a unit / units (Update)
- c) A combination of the above



ROM Bootloader

As discussed in Section 4.4, all CubeProducts also have a manufacturer provided ROM bootloader. This provides an alternative method for updating firmware, although all updates in orbit are normally done via the CubeSpace Base-Bootloader. Details on reprogramming the CubeProduct on the ground through the ROM bootloader is supplied in [RD3].

Software release package

CubeSpace supplies a software release bundle together with the initial hardware delivery of the CubeADCS that contains the latest firmware-, bootloader- and configuration file versions of the CubeADCS units that should correspond to the firmware versions on the units. Subsequent software release bundles are also made available to offer new functions and features, address earlier software issues and so on.

Each software release bundle contains a number of files:

- The firmware-, configuration- and bootloader files can be found under <...\sw-bundle-vxxx\firmware>.
- The CubeProducts are available as subdirectories below it, for example <...\sw-bundle-vxxx\firmware\cube-computer-5>
- A subdirectory containing the control program (FW) and configuration file(s) for the CubeProduct is available, for example <...\sw-bundle-vxxx\firmware\cube-computer-5\control-program>
- A subdirectory with tools like the CubeSupport application.

Each software release package also contains documentation to explain what is contained in the release.

4.7 Updating of CubeADCS configuration and Control Program firmware through Bootloader

To update the configuration of the CubeADCS (CubeComputer) or its application software, the unit must first be reset, or power cycled.

Upon power-on or reset, the bootloader application will run first. The bootloader uses a backoff timer and will listen for incoming commands for **5 seconds** before jumping to (starts executing) the Control Program application. The user must send a Halt (Halt bootloader) command within 5 seconds of power-on or reset, except if a new connection is established through the CubeSupport application within this period, in which case the bootloader will automatically be halted.

The boot process is then halted and the CubeADCS will remain in the bootloader application until a JumpToApp (Jump To Application) or Reset command is received.

The CubeADCS is now ready to be programmed. The Bootloader Application Manual (see [RD3]) provides detailed information for interfacing with the bootloader to update the configuration and control application firmware.

4.8 Updating a connected CubeProduct's configuration and control application firmware via CubeComputer

The CubeProducts connected to the CubeADCS can be updated via the CubeADCS Control Program. Note that this functionality is not available in the bootloader application. Having this functionality in the CubeADCS control application means that it is possible to maintain control while a CubeProduct is being updated, if the CubeProduct being updated is not required for the current control mode.

To update a connected CubeProduct, the firmware and configuration files needed for the update must first be uploaded to the CubeADCS memory using the CubeADCS Control Program. Once the files are uploaded, the user may trigger the CubeADCS to perform the upgrade.



4.8.1 Uploading files

A file upload consists of a setup telecommand followed by the data transfer. The setup telecommand requires that the file metadata be extracted from the CubeSpace (.cs) file to be uploaded and sent as part of the setup telecommand. The data transfer implements the Bulk Data Transfer protocol. For information on CubeSpace files and the Bulk Data Transfer protocol, refer to [RD4].

The FileTransferSetup telecommand is used to initiate a file upload. For file uploads, this telecommand requires two parameters:

1. OpCode: Set to Upload.
2. MetaData: Data block copied from the .cs file to upload.

All other parameters are ignored.

The FileTransferStatus telemetry exposes state and error information for the current transfer. The ErrorCode parameter of this telemetry is the first indicator of an error condition. If ErrorCode is non-zero, this indicates an error, and the user should abort the transfer and inspect the error flags in the telemetry for detailed information on the cause of the error.

The full upload procedure is as follows:

1. Initiate the transfer with FileTransferSetup telecommand.
2. Poll the FileTransferStatus telemetry.
 - a. Until the State parameter changes to Busy.
 - b. Allow for up to 1 second for the state to change.
 - c. The poll backoff should be at least 10 milliseconds.
 - d. Abort if ErrorCode is non-zero, or the state does not transition in time.
3. Use the Bulk Data Transfer protocol to transfer the file data.
4. Request the FileTransferStatus telemetry and confirm that no errors occurred during the transfer.
5. Request file information for the uploaded file(s) to confirm they are present as expected.

Example code is provided as part of libcubeobc (included in the software bundle) for performing file uploads.

4.8.2 Requesting file information

The FileTransferSetup telecommand is used to request file information. For file information, this telecommand requires two parameters:

1. OpCode: Set to Info.
2. File: Set to the file to get information for.

All other parameters are ignored.

The full procedure to request file information is as follows:

1. Initiate the request with FileTransferSetup telecommand.
2. Poll the FileTransferStatus telemetry.
 - a. Until the State parameter changes to Idle.
 - b. Allow for up to 3 seconds for the state to change.
 - c. The poll backoff should be at least 10 milliseconds.
 - d. Abort if ErrorCode is non-zero, or the state does not transition in time.
3. Request the FileInfo telemetry to get the file information.

If the Size parameter of the FileInfo telemetry is 0(zero), the file is either not present or is corrupt (CRC check failed).

Regarding the file handle, note that all CubeProducts of the same type share the same firmware binary file. E.g. all CubeWheels will use the firmware file CubeWheelBin. Configuration files are unique and each CubeProduct will have its own configuration file. The integer suffix of configuration file handles is linked to



the assigned abstract type of the CubeProduct. E.g. The file handle CubeWheelCfg0 is associated with the abstract type RWL0 (If RWL0 is a CubeWheel). Abstract types are assigned as part of the expected nodes configuration item and is managed by CubeSpace. The ExpectedNodes telemetry can be requested to see the assignment.

Example code is provided as part of libcubeobc (included in the software bundle) for requesting file information.

4.8.3 Performing a CubeProduct upgrade

The FileTransferSetup telecommand is used to initiate a CubeProduct upgrade. For upgrades, this telecommand requires the following parameters:

1. OpCode: Set to Upgrade.
2. Node: Set to the node type of the CubeProduct to be upgraded.
3. SerialInt: Set to the integer portion of the serial number of the node to be upgraded. E.g. For CubeMag-Deloyable with serial number "CM23001", this parameter should be 23001.
4. Program: Set to the program to upgrade.
5. ForcePort: Set to PortNone in all cases except for those specified in section [add link].

All other parameters are ignored.

The full upgrade procedure is as follows:

1. Initiate the upgrade with FileTransferSetup telecommand.
2. Poll the FileTransferStatus telemetry.
 - a. Until UpgradeState parameter changes to Upgradeldle.
 - b. Allow for up to 120 seconds for the state to change. i.e., for the upgrade to complete.
 - c. The poll backoff should be at least 100 milliseconds. However, a longer backoff is preferable as it allows more processing time to perform the upgrade.
 - d. Abort if ErrorCode is non-zero, or the state does not transition in time.

Note that when an upgrade is performed, both the firmware binary and configuration of the CubeProduct are both programmed to the CubeProduct. Therefore, both files should be uploaded before performing the upgrade.

Example code is provided as part of libcubeobc (included in the software bundle) for performing CubeProduct upgrades via CubeComputer.

4.8.4 CubeProduct upgrades with ForcePort

Referring to file uploads, the FileTransferSetup telecommand lists a parameter ForcePort. This parameter allows the upgrade process to force an upgrade on the specified port, rather than relying on the result of Auto-Discovery for determining the port that the target node is connected to.

Under normal operating conditions this parameter is not needed and should be set to PortNone. However, if a connected CubeProduct becomes unresponsive and cannot be discovered with Auto-Discovery, this parameter can be used to upgrade the bootloader of the CubeProduct in an attempt to recover it. Note that this requires that the port that the CubeProduct is connected to is known.

4.8.5 Control mode and power handling during a CubeProduct upgrade

When a connected CubeProduct is upgraded, it is required to be running its bootloader application. This means that it is not usable by the CubeADCS for control. In this scenario the selection of the control mode takes priority. i.e., the CubeADCS will not automatically change the control mode to facilitate the upgrade. Rather, if the current control mode is not suitable, the FileTransferStatus telemetry will set the NodeUsedError flag. The power state of the CubeProduct being upgraded is handled automatically by the CubeADCS. When an upgrade is initiated, the current power state of all connected CubeProducts is captured and will be used to restore power once the upgrade is complete. The power state of a CubeProduct



being upgraded cannot be changed while the upgrade is in progress, however, a new power state can be commanded, but will only take effect once the upgrade is complete. The power state of CubeProducts that are not being used for the upgrade can be changed while the upgrade is in progress.

When upgrading the bootloader of a connected CubeProduct, all connected CubeProducts are required to be powered off. Therefore, it is not possible to perform control. This is because the CubeProduct bootloader is upgraded using the manufacturer ROM bootloader, which requires full ownership of the CAN bus.

4.9 Recovery on reset

If an MCU reset or power cycle of the CubeADCS occurs while controlling the attitude of the satellite, four scenarios, found in Table 7, may occur.

Table 7: Boot reset scenarios

Condition at reset/power cycle	Condition at control program start-up (5s later)
Satellite is tumbling randomly. Detumbling controller is active (or not)	Satellite is still tumbling randomly
Satellite is in steady-state Y-Thomson spin – Detumbling controller is active	Satellite will still be in Y-Thomson spin
Satellite is in Y-Momentum control mode – stable attitude	Wheel spinning down, satellite spinning about Y-axis (Y-Thomson spin)
Satellite is in 3-Axis pointing mode	Wheels would have had low bias, so satellite will be in slow random spin
Satellite is executing a slew manoeuvre in 3-axis reaction wheel control	Wheel momentum will transfer back to satellite resulting in random spin

The first scenario should only occur during the commissioning of the satellite. In this case, automatic recovery will most likely not be preferred.

In the second scenario, the satellite remains in a Y-Thomson spin after the reset event. The satellite will remain in this state without active control if there are no significant external disturbances. For a typical start-up time of 5 to 10s, there will be no change in the spinning motion of the satellite.

If a reset occurs while the satellite is in the Y-momentum stabilized mode, the wheel speed will slowly ramp down because of the friction of the wheel bearings. Angular momentum will be transferred back to the satellite and the satellite will eventually transition into a Y-Thomson spin; however, the wheels might still be spinning (ramping down) when the Control Program comes online.

If the satellite is in 3-Axis reaction wheel control mode, it will revert to a slow random tumble motion, depending on wheel speed at the reset. If the ADCS was executing a slew manoeuvre, the wheel speeds might potentially be high, and this wheel momentum will be transferred to the satellite body.



5. ADCS attitude determination and control

This chapter describes features of the CubeADCS specific to attitude estimation (determination) and control. The full attitude state of the satellite consists of:

1. The attitude itself – the transform between the orbit reference coordinate (ORC) frame, and the satellite body coordinate (SBC) frame. This is represented internal to the ADCS as a 4-element quaternion, and direction-cosine matrix (DCM), but for external telemetry and commanding makes use of a 2-1-3 Euler rotation sequence, with the angles defined as pitch, roll and yaw.
2. The orbit referenced (ORC) and inertially referenced (IRC) angular rate vector.

The ADCS requires knowledge of the current attitude in order to control the spacecraft. There are different options to estimate (measure / determine) the attitude, available as estimation mode selections. Not all modes provide the full attitude state, as some modes only measure angular rates. The combination of sensors required also varies by estimation mode.

The various estimation modes are summarized in Table 8.

Table 8: Estimation mode summary

Estimation Mode	DESCRIPTION	Estimated State	Use Case
EstNone (0)	No estimation performed	Nothing	Startup/default mode
EstGyro (1)	Gyro measurements are used directly as the estimated angular rate, without any dynamic bias corrections	Angular rate only	Used for initial rate estimation, and during detumbling
EstMagRkf (2)	Estimates angular rate using successive measurements of the magnetic field vector	Angular rate only	Used to confirm/verify gyro measured angular rates. Can also be used to confirm magnetometer mounting transform.
EstPitchRkf (3)	Estimates angular rate, and pitch angle, from magnetometer measurements only. Requires orbit position and velocity to be accurate.	Angular rate vector and pitch angle only.	Can be used together with Y-spin mode when satellite is only rotating around pitch axis.
EstTriad (4)	Calculate attitude from two selectable modelled/measured vector pairs (selection between FSS, MAG, HSS) and angular rates directly used from IMU (gyro) measurements. Requires orbit position and velocity to be accurate. Cannot output attitude angles if either sensor measurement is invalid.	Angular rate vector and attitude angles	Used only as a test or debugging mode, to check mounting transformations and sensor operation.
EstFullEkf (5)	Extended Kalman Filter (EKF) implementation that uses all available sensor inputs to estimate current attitude (quaternion) and angular rates.	Angular rate vector and attitude angles	Used with 3-axis reaction wheel control, when accurate knowledge of satellite mass properties is available, and the satellite experiences few external disturbances
EstGyroEkf (6)	Extended Kalman Filter (EKF) implementation that uses all available sensor inputs to estimate current attitude (quaternion) and dynamic gyro bias. Angular rate is calculated directly from IMU measurement (with subtracted estimated bias)	Angular rate vector and attitude angles	Used with 3-axis reaction wheel control



The control behaviour that the CubeADCS exerts on the satellite is governed by the control mode selection. Control modes are grouped based on their use of actuators and end effect. The control modes of the CubeADCS are summarized in Table 9.

Table 9: Control mode summary

Control mode group	Control mode	Description
-	0: ConNone	No control
Detumbling and safe-mode magnetic spin control	1: ConBdot	Detumbling Bdot, using single-axis magnetorquer control only
	2: ConYspin	Y-Thomson spin, using magnetorquer control only
	3: ConBdot3	3-axis Bdot detumbling, using magnetorquer control only
	4: ConDetumble	3-axis low power fast (8 Hz) detumbling using magnetorquer control only
	5: ConSunYspin	Sun Y-axis spin detumbling, using magnetorquer control only
	6: ConZspin	Z-Thomson spin, using magnetorquer control only
	7: ConSunZspin	Sun Z-axis spin detumbling, using magnetorquer control only
Gravity-gradient	8: ConGGboom	MTQ control of GG boom & yaw zero control
	9: ConGGsun	MTQ control of GG boom & Z-RW yaw sun pointing control
Y-Momentum	10: ConYwheelInit	Y-Momentum wheel (Initial pitch acquisition)
	11: ConYwheel	Y-Momentum wheel (Steady state)
3-Axis Wheel control	12: ConXYZwheel	3-axis reaction wheel control, to follow externally provided RPY reference (nadir pointing = zero roll, pitch and yaw reference)
	13: ConSunTrack	3-axis reaction wheel sun tracking control
	14: ConTgtTrack	3-axis reaction wheel Earth target tracking control
	15: ConTgtSteer	3-axis reaction wheel Earth target steering control
	16: ConGndTrack	3-axis reaction wheel ground station target tracking control
	17: ConIrcTrack	3-axis reaction wheel inertial target tracking control
	18: ConMoonTrack	3-axis reaction wheel moon tracking control
	19: ConSatTrack	3-axis reaction wheel satellite tracking control
	20: ConYawSun	Nadir pointing and yaw towards the sun
	21: ConYawTarget	Nadir pointing and yaw towards a ground target
	22: ConRollSun	Nadir pointing and roll towards the sun
	23: ConRollTarget	Nadir pointing and roll towards a ground target
	24: ConYawTargetBest	Nadir pointing and yaw best +/-Y body facet towards a ground station
	25: ConFmcTarget	Forward Motion Compensation Target Scan
	26: ConYawSunSpin	RW sun yaw-spin control
	27: ConSunDetumble	RW detumbling to sun-point control
	29: ConSunPayload	RW sun tracking and rotation for payload pointing to Zenith
Wheel management	50: ConStopRW	Controlled stop of reaction wheels
	51: ConHxyzRW	Open loop XYZ wheel momentum command control



Full detail of all the above modes, their use cases during satellite operations, valid combinations, and configurable behaviour, are provided in [RD5].



6. Appendix: Health Check/Acceptance test

This Appendix first requests user contact information and then provides a sequence to perform a “health check” / Acceptance test on the CubeADCS to confirm that it is working as it should.



Please complete this Appendix Acceptance test and return a digital copy of the whole appendix to CubeSpace along with copies of the captured image(s) and test pattern(s).

Please complete Table 10. This table will allow CubeSpace to identify the user using the CubeProduct.

Table 10: User and CubeADCS Details

Information needed	Information supplied
Company/university/institution:	
Contact person:	
Email of contact person	
Satellite name:	
CubeADCS Serial number	
Software package version:	
Person(s) responsible for Health Check testing	
Date Health Check was performed	

Please keep in mind that all required safety measures must be taken when handling CubeADCS. ESD protection must always be in place. Please use gloves when handling CubeADCS and ensure that the user and the surface worked on is properly grounded. Flight Models (FM) must always be handled in a dust-free environment such as a cleanroom.

It is essential to perform the basic CubeADCS health check procedure upon receipt of either hardware or software to ensure the unit is in complete working condition before further testing/integration occurs.

For the sections that follow where the expected current consumption values are given, the current was measured from a bench power supply set to **12.0V**. Unless stated otherwise, the value given as the expected current is the current increase that should be shown by the power supply.



Consult Section 6.5.1 for details on connecting the power supply to the CubeADCS bundle in the case that CubeWheels CW0500 or larger is used.

With the CubeADCS plugged in and the “VBAT On” switch in the “Off” position, the current consumption is ~1 mA due to the “VBAT On” LED on the CubeSupport PCB.

When switching the “VBAT On” and “Enable” switches to the “On” position to power on the CubeADCS, the current consumption on the CubeSupport PCB increases by ~ 19 mA due to additional LEDs on the CubeSupport PCB itself. The current consumption of the CubeComputer’s 3.3 V rail increases by ~90 mA after booting into the Control Program. If the CubeADCS is configured for the 3.3 V rail to be supplied



externally, the bench PSU will show this total current of ~110 mA being drawn, as the CubeADCS will be supplied from the 3.3 V linear regulator on the CubeSupport PCB. If the CubeComputer's onboard switching regulator is used to supply the 3.3 V and 5 V rails, the current being drawn from the bench PSU will be lower.

6.1 Connection to the unit with CubeSupport software

Following the steps described in sections 3.3.1, 3.3.2 and 3.3.4, the CubeADCS unit should be setup in such a way to allow the user to power the unit and communicate with it from a PC using the *CubeSupport* application, a Windows-based software application also supplied with the CubeADCS unit.

The *CubeSupport* application can be opened by running the executable file located in the software release bundle under <...\sw-bundle-vxxx\tools\cube-support\cubesupportui.exe>.

6.2 CubeComputer health check

To perform the health check of CubeComputer, follow the steps below:

- 1) Power on the CubeADCS by turning on the power supply and the "VBAT On" toggle switch on the CubeSupport PCB.

As discussed in Section 3.4, establish a CAN or UART connection **within 5 seconds** of powering on the CubeADCS. This will ensure the CubeADCS is only running the Bootloader.

Note: If this step is not done quickly enough, and the Control Program starts running, a "Reset to Bootloader" procedure can be performed (CubeComputer tab → common-framework - Commands → Reset → ToBootloader). This procedure automatically sends a Soft reset command, immediately followed by a Halt Bootloader command to keep the CubeComputer bootloader from jumping to the Control Program. However, as stated by the pop-up message that follows, the view must be manually refreshed by closing and then reestablishing the communication connection.

- 2) Alternatively, a power cycle can be performed by toggling the VBAT switch, but the communication interface must be disconnected and reconnected within 5 seconds.
- 3) Verify that the current drawn from a bench power supply set to **12.0V** increases by the values shown in Table 11 below and complete the table.
- 4) Note that the actual values might be slightly lower if a Demo Model variant is used, since it will not have a Main IMU fitted.

Table 11: CubeComputer Health Check: Expected Current Consumption

Field	Expected Result	Actual result	Comments	P / F
Maximum inrush current	If 3.3 V is supplied from CubeComputer's onboard switching regulator: ~80 mA Else, if supplied externally from the CubeSupport PCB's linear regulator: ~150mA			
Idle CubeComputer current, after 5-second boot procedure concluded	If 3.3 V is supplied from CubeComputer's onboard switching regulator: ~40 mA Else, if supplied externally: ~80mA		If supplied from onboard regulator: Failed if > 60 mA. If supplied externally: Failed if > 130 mA.	



- 5) Retrieve all the telemetry indicated in Table 12, using the CubeSupport application, by clicking the Refresh button . Fill in Table 12.

Table 12: CubeComputer Health Check: Bootloader configuration

Field	Expected Result	Actual result	Comments	P / F
Advanced tab → cube-common-base-bootloader-5 - Identification → Identification				
Program type identifier	ProgramTypeBootloader		Just record the results.	
Interface version				
Firmware version (Major)				
Firmware version (Minor)				
Advanced tab → cube-common-base-bootloader-5 - Identification → Serial Number				
OTP serial	Record of the two Serial Numbers in the form “CCxxxxx”		These two must match	
Config Serial				
Advanced tab → cube-common-base-bootloader-5 - Configuration → Configuration				
CAN2 Address	Client dependent		The defaults, unless otherwise specified by the client in the Client Mission Overview document, are: CAN2 Address: 4 CAN2 CSP Enabled: Unchecked I2C Address: 83	
CAN2 CSP enabled				
I2C Address				

- 6) Verify that all applicable communication interfaces used by the Bootloader are operational. Establish a communication connection with the CubeADCS for each of the required communication interfaces as selected by the client. Follow the instructions from Section 3.3.4. Fill in the results in Table 13 below.

Table 13: CubeComputer Health Check: Communication interfaces of the Bootloader

Field	Expected Result	Actual result ²	Comments	P / F / NA
CAN connection established	Connection established and retrieved Identification telemetry packets on all interfaces selected by the client.		Consult the specific CubeADCS's configuration sheet to establish which communication interfaces are applicable. For unused interfaces, fill in "N/A".	
UART connection established				
I2C connection established				
RS422 connection established				
RS485 connection established				

- 7) Now the Control Program must be run. The easiest is to follow the quick steps in Section 3.4. Alternatively, toggle the CubeSupport PCB's VBAT switch off and on again, but wait at least **5 seconds** after powering on the unit before continuing, to allow the Bootloader to jump to the Control Program.

² These, and several other results in these tables, can simply be marked with a checkmark to indicate success: ✓



- 8) Verify that all applicable communication interfaces used by the Control Program are operational. Establish a communication connection with the CubeADCS for each of the required communication interfaces as selected by the client. Follow the instructions from Section 3.3.4. Fill in the results in Table 14 below.

Table 14: CubeComputer Health Check: Communication interfaces of the Control Program

Field	Expected Result	Actual result	Comments	P / F / NA
CAN connection established	Connection established and retrieved Identification telemetry packets on all interfaces selected by the client.		Consult the specific CubeADCS's configuration sheet to establish which communication interfaces are applicable. For unused interfaces, fill in "N/A".	
UART connection established				
I2C connection established				
RS422 connection established				
RS485 connection established				

- 1) Find and note the **firmware version** numbers that should be loaded on the CubeComputer by navigating to <...\sw-bundle-vxxx\firmware\cube-computer-5\control-program> in the software release bundle. The file of interest is named "**app-control-program-cube-computer-5-X.Y.Z-freertos-release**", where "X.Y.Z" denotes the firmware version. "X" corresponds to the Major firmware version, "Y" corresponds to the minor firmware version and "Z" denotes the patch version.
- 2) Similarly, the CubeComputer firmware's **interface version** must also be verified. This version number can be found by navigating to <...\sw-bundle-vxxx\libcubeobc\api\docs\cube-computer-5> in the software release bundle. This location contains a file named "**cube-computer-5-control-program-X-api-reference.html**", where "X" denotes the CubeComputer interface version. Note this version.
- 3) Retrieve all the telemetry indicated in Table 15, using the CubeSupport application, by clicking the Refresh button . Fill in Table 15.

Table 15: CubeComputer Health Check: Expected Results

Field	Expected Result	Actual result	Comments	P / F
CubeComputer tab → common-framework - Identification → Identification2				
Node type identifier	NodeTypeCubeComputer			
Interface Version	Verified in the software bundle in step 2)			
Firmware version (Major)	Verified in the software bundle in steps 1) and 2)			
Firmware version (Minor)				
Firmware version (Patch)				
Runtime (seconds)	Incrementing			
Runtime (milliseconds)	0 – 1000			
CubeComputer tab → common-framework - Identification → Version				
Git Branch	Master		Just record the results. Note that the Git Hash in the manifest.txt file is a hexadecimal value, but the Version telemetry returns a decimal value.	
Git Hash	Can be obtained from manifest.txt file			
Hardware version (Minor)	Variable			



Field	Expected Result	Actual result	Comments	P / F
CubeComputer tab → common-framework - Identification → Serial Number				
OTP serial	Record of the two Serial Numbers in the form “CCxxxxx”		These two must match, except for certain configurations, for example a Demo Model, where the Config Serial will be CC00000.	
Config Serial				
CubeComputer tab → common-framework - Time → Current Unix Time				
Current Unix seconds	Incrementing		Set to time of firmware build by default. Can be synced to the PC's system time with the Sync button: 	
Power State tab → cube-computer-common - PowerState				
GYR0 power state GYR1 power state	PowerOn		This switches on the redundant Gyro, which is normally inactive, before continuing with the next test below. Set, then Get to confirm receipt	
Health tab → cube-computer-5-control-program – Health → ACP execution telemetry				
Time since iteration start	0 – 200 ms		Value should vary. The ADCS control loop runs at 5 Hz and the magnetic control loop runs at 1 Hz., so the maximum delay would be 200 ms.	
Current execution point	Varying		Should be mostly ExecPointIdle	
Execution time of node sync	~0 ms		These values could vary depending on sensor and actuator activity	
Execution time of sensor comms	~0 ms			
Execution time of sensor calibration	~1 ms			
Execution time of ADCS estimators	~0 ms			
Execution time of ADCS controllers	~0 ms			
Execution time of wheel comms	~0 ms			
Execution time of ADCS models	~21 ms			
Execution time of ADCS loop sync	~160ms			
Execution time of health service	~3 ms			
Execution time of telemetry loggings	~9 ms			
ADCS loop offset from RTC	~0 ms			
ADCS loop has started flag	Checked, unless Current execution point is ExecPointIdle or ExecPointTlmLog			



Field	Expected Result	Actual result	Comments	P / F
HIL synchronised	Not checked, unless doing Hardware-in-the-loop testing			
Health tab → cube-computer-5-control-program – Health → CubeComputer Health				
MCU Temperature	~25 C		Ambient temperature dependent	
MCU Current	~26 mA			
MCU Voltage internal	~3300 mV			
MCU Supply Voltage	~3300 mV			
5V Supply Voltage	~5000 mV			
Battery Voltage	~11300 mV (After accounting for the voltage drop of the diode on the CubeSupport PCB)			
Hardware Version Voltage	~300 mV, ~500 mV, ~700 mV or ~900 mV		Just note the result. Depends on CubeComputer hardware revision obtained earlier	
SRAM1 Current	< 1 mA			
SRAM2 Current	< 1 mA			
FPGA Current 1v5	~ 0 mA			
FRAM Current	~ 0 mA			
ADC Current	~ 1 mA			
Flash Current	~ 1 mA			
RS485 Current	~1 mA			
CAN Current	~6.5 mA			
Gyro Current	~18 mA		For Demo Model variants (with no main IMU fitted), this is 0 mA	
Redundant Gyro Current	~2 mA			
Gyro Over Voltage	Not checked			
Gyro Under Voltage	Not checked			
Redundant Gyro Over Voltage	Not checked			
Redundant Gyro Under Voltage	Not checked			
ADCS tab → cube-computer-5-control-program – AdcsTelemetrySensorRaw → Raw GYR sensor telemetry				
Timestamp	Incrementing			
GYR0 raw rate X component	-0.5 to +0.5 deg/s		These values must be near zero, only varying slightly, when the CubeADCS is not being rotated. If the CubeADCS is carefully rotated, these values will indicate the measured rate of rotation around the specific axis in degrees per second.	
GYR0 raw rate Y component	-0.5 to +0.5 deg/s			
GYR0 raw rate Z component	-0.5 to +0.5 deg/s			
GYR1 raw rate X component	-3.0 to +3.0 deg/s			



Field	Expected Result	Actual result	Comments	P / F
GYR1 raw rate Y component	-3.0 to +3.0 deg/s		Note that on some configurations, GYRO might not be fitted. In that case, the GYRO results will be 0.	
GYR1 raw rate Z component	-3.0 to +3.0 deg/s		Note: the large range of the redundant GYR1 limits are expected.	
GYR0 valid flag	Checked		For some configurations, GYRO might not be fitted, in which case the Valid Flag will be unchecked.	
GYR1 valid flag	Checked			



If the Interface version, Firmware version (Major) or Firmware version (Minor) differ from the expected results (i.e. do not reflect the latest software versions as per the software bundle), please stop and contact CubeSpace.

CubeSpace ensures that all the latest firmware as per the software release package are in fact programmed on the various CubeADCS units and CubeSpace then conducts the same health check as per this section before shipping to the user. If the user finds discrepancies, the shipped baseline should be considered as not correct.

Once the values in the tables have been confirmed, the CubeComputer is considered to be in working condition. If any result obtained does not match the expected result, please contact CubeSpace.

6.3 Auto-Discovery results check

Auto-Discovery of all connected products is performed when the Control Program first runs. The CubeSupport application does not inform the user of this via unsolicited telemetry, and the CubeSupport view is not automatically updated. The user must refresh the view using the  icon as shown below, after which the list in the Port Map tab will be updated with appropriate information about all expected and connected products.

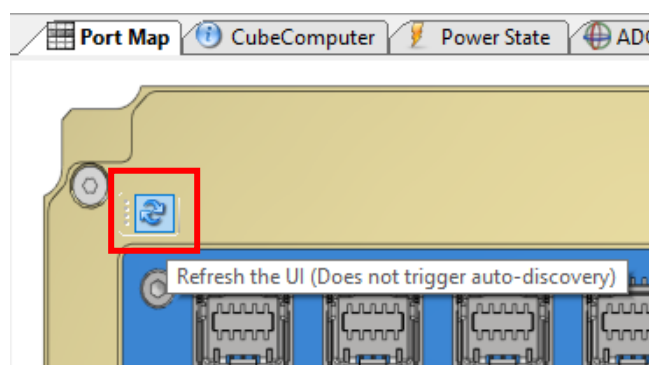


Figure 43: Refresh icon



6) Reconnect the harness to the CubeConnect header indicated in Figure 17 / Figure 18.

Table 17: CubeTorquer Health Check: Record of magnetorquers DC resistance

Field	Expected Result	Actual result	Comments	P / F
Magnetorquer 1 (MTQ0) DC resistance	15 – 200 ohm		These results will vary depending on the magnetorquer type and ambient temperature and is only taken for the record.	
Magnetorquer 2 (MTQ1) DC resistance	15 – 200 ohm			
Magnetorquer 3 (MTQ2) DC resistance	15 – 200 ohm			

Follow the steps below to perform the CubeTorquer health check via CubeComputer:

- 1) Set the duty ratio of the first magnetorquer rod to 75% using the OpenLoopCommandMtg telemetry as shown in the table below.

Table 18: CubeTorquer Health Check: Open-Loop Command

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 - Commands → OpenLoopCommandMtg				
MTQ0 open-loop duty cycle command	750 Note: This is the value for the duty ratio, out of 1000.		The actual duty ratio can vary in most cases, due to control loop latency. The value will be limited in software to be a maximum of 800 milliseconds out of 1000, because the flight software takes magnetometer samples during the remaining time.	
MTQ1 open-loop duty cycle command	0			
MTQ2 open-loop duty cycle command	0			

- 2) Retrieve the measured value of the CubeComputer's 5V rail while the magnetorquer is switched on.
Note: As the On-Off cycle of the magnetorquer driving loop is applied, this value might show very slight variation, depending on the nature of the Power Supply circuitry. Retrieve the value several times and only record the lowest observed value.
- 3) Fill in the lowest observed value in Table 19.
- 4) Repeat Steps 2) and 3) for the remaining two magnetorquers, in turn, and fill in the values in the Table 19.

Table 19: CubeTorquer Health Check: Voltage on 5V rail

Field	Expected Result	Actual result	Comments	P / F
Health tab → cube-computer-5-control-program-8 – Health → CubeComputer Health				
MTQ0: 5V Supply Voltage	~5000 mV Note: Depending on the Power supply circuitry, this value can vary during the On- and Off period of the duty cycle. Record the lowest retrieved value of several measurements.		Lowest retrieved value must be > 4800 mV.	
MTQ1: 5V Supply Voltage				
MTQ2: 5V Supply Voltage				



- 5) Fill in the DC resistance values and 5V rail measurements from Table 17 and Table 19 into Table 20.
- 6) Set the duty ratio of the first magnetorquer (MTQ0) back to 750, as in Table 18. The values of the remaining magnetorquers must remain at 0.
- 7) Retrieve the current measurement and fill in the Positive current peak and average values in Table 20 below. In some specific cases, when drawing more than 450 mA per magnetorquer, high values for the duty cycle setting might saturate the current sensors, in that case lower the setting for the open-loop duty cycle command from the default value of 750 until the measurement is no longer saturated and record the duty cycle used.
- 8) Set the duty ratio of the first magnetorquer (MTQ0) to -750 to drive the torquer in the reverse polarity using the same duty ratio. The values of the remaining magnetorquers must still remain at 0.
- 9) Retrieve the current measurement again and fill in the Negative current peak and average values in Table 20 below.
- 10) Repeat steps 5) to 7) for the remaining magnetorquers (MTQ1 and MTQ2).

Table 20 has some embedded formulas to assist with determining the Error %. Select the contents of the table and update the fields (shortcut key: F9). Verify that the error percentage is below 3%.



The accuracy of the current sensors has independently been verified and is beyond the scope of this document. The checks done in this Health Check are for record keeping.

The validity of the current readings reported by the magnetorquer rod current sensors, could be improved if an accurate multimeter is used in-line with the magnetorquer rods to compare with the readings reported by the CubeComputer.

Normally this multimeter-based measurement is not strictly required, as the accuracy of these readings are usually not mission-critical and merely serves as a point of interest, and therefore theoretical values based on the DC resistance and measured 5V rail would suffice. In this case the error range could be extended beyond 3% due to more unknowns in the system.

Table 20: CubeTorquer Health Check: Expected Current Consumption

Field				Expected Result		Actual result			Comments	P / F
Health tab → cube-computer-5-control-program-8 – Health → Torquer Current measurements										
		Torquer resistance from Table 17 ³	5V supply voltage from Table 17	Peak (mA) ⁴	Avg (mA)	Peak (mA)	Avg (mA)	Error (%) (must be < 3%) ⁵		
MTQ0	Fwd	60	5000	82	61.5			-100	Temperature dependant.	
	Rev							-100		

³ The table adds 1.0 ohm to this value when calculating the expected Peak current, to account for the On-resistance of the H-bridge driving circuitry.

⁴ This value can be substituted by a value measured in-circuit with a multimeter. It's theoretical values is determined by the formula: Actual measured 5V supply voltage, divided by: the sum of the measured torquer resistance plus 1.0 ohm.

⁵ Error between the expected and actual Peak values.



Field				Expected Result			Actual result		Comments	P / F
MTQ1	Fwd	60	5000	82	61.5			-100	Expected Avg values are for Duty Cycle = 75% but can vary by several mA due to control loop latency.	
	Rev							-100		
MTQ2	Fwd	60	5000	82	61.5			-100		
	Rev							-100		

11) Using a small compass or mobile phone compass application, confirm that the polarity of the magnetic field reverses as the forward open-loop duty cycle command of Step 5) and the reverse command of Step 7) is performed and capture the result in Table 21 below.

NOTE: Due to the nature of the interaction of magnetic fields, the CubeTorquer polarity checking needs to be done with only one CubeTorquer at a time. Therefore, only the single CubeTorquer being tested at a time must have its open-loop duty cycle command set to non-zero while the other CubeTorquers not being tested must remain at zero.

Table 21: CubeTorquer Health Check: Polarity

Field	Expected Result	Actual result	Comments	P / F
Where possible, measure polarity of the magnetic field for forward and reverse operation	Magnetic field polarity of each torquer reverses between a positive and negative open-loop duty cycle command.	MTQ0: MTQ1: MTQ2:		

12) Set the open-loop magnetorquer commands to zero when finished.

Once the values in the above tables have been confirmed, the CubeTorquer is considered to be in working condition. If any result obtained does not match the expected result, please contact CubeSpace.

6.5 CubeWheel health check

CubeSpace is adding to the CubeWheel product range, with the CW0500 being the first of the new releases. In addition to the 3.3V digital supply, the motor circuitry of these larger Reaction Wheels must be supplied from a V_{bat} bus.

Section 6.5.1 provides specific steps to be taken when testing CubeWheels CW0500 and larger. If CW0162 or smaller are being tested, then one can proceed directly to Section 6.5.2.



The CubeSupport PCB is not designed to accommodate voltages above 17.6V, being in line with the existing smaller CubeWheel variants. Until the CubeSupport PCB design is revised, testing CW0500 and larger requires additional preparation presented in 6.5.1.



6.5.1 Preparing for testing CW0500 and larger

6.5.1.1 Supplying V_{Bat} and GND directly to CubeDoor

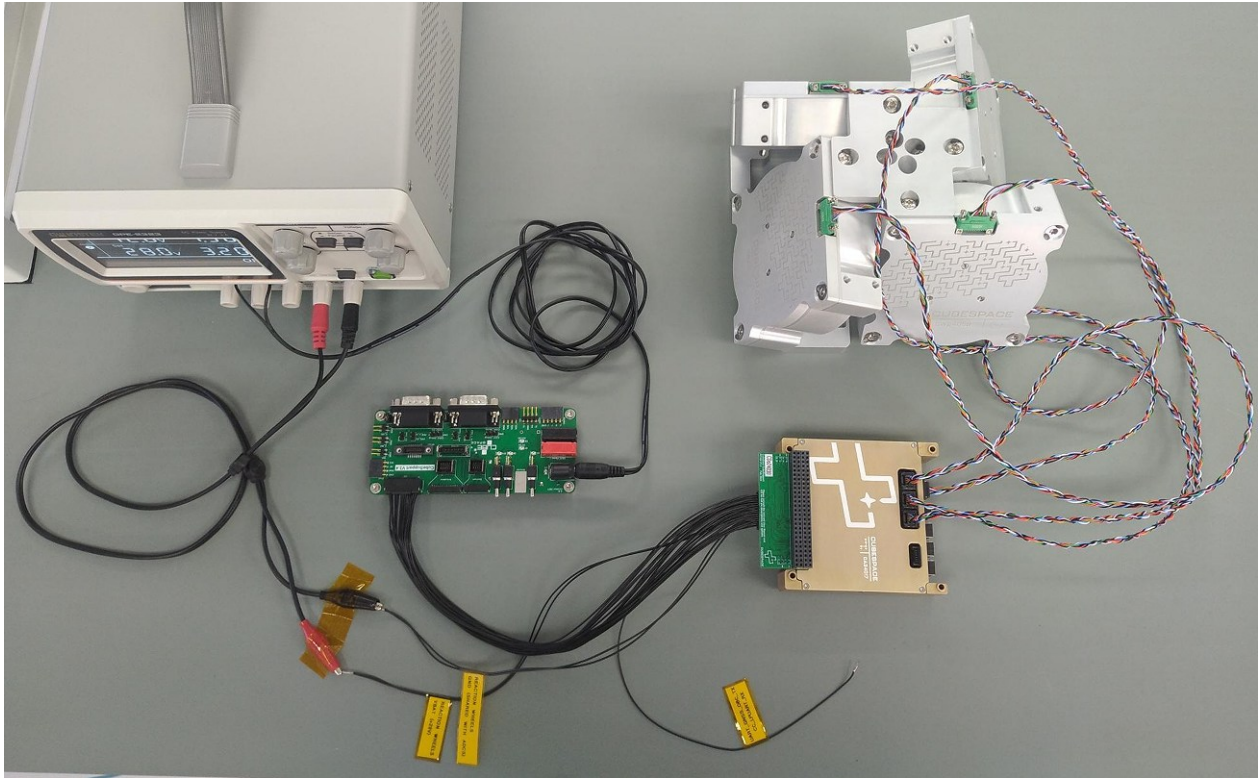


Figure 45: Providing V_{Bat} and GND directly to CubeDoor, bypassing CubeSupport PCB, for testing CubeSpace Reaction Wheels CW0500 and larger

With the shipment of these larger reaction wheels, a modification will be made to the CubeSupport PCB harness to break out the Supply and Return wires for the VBAT rail.

- For bundles containing CubeWheels CW0500 or CW1200, the broken-out wires will be labelled “REACTION WHEELS VBAT (+16V)” and “REACTION WHEELS GND (SHARED WITH ADCS)” respectively. Connect these wires to an independent Power Supply channel set to **16.0V, 3.0A – 3.2A**.
 - Although the 16V battery voltage for these wheels fall within the CubeSupport PCB’s acceptable 17.6V input range, the wires are broken out to assist with the tests discussed in Section 6.17.
- For bundles containing CubeWheels CW2500 or CW5000, the broken-out wires will be labelled “REACTION WHEELS VBAT (+28V)”, and “REACTION WHEELS GND (SHARED WITH ADCS)” respectively. Connect these wires to an independent Power Supply channel set to **28.0V, 3.0A – 3.2A**.

6.5.1.2 Routing V_{Bat} and GND directly to the reaction wheels using the VBat Bypass

To power CW0500 and larger reaction wheels, CubeADCS uses a V_{Bat} Bypass PCB that plugs into the CubeDoor board (in the case of PC104 variants) or completely replaces the CubeDoor PCB (in the case of the Backplane variant). As indicated by the name, the V_{Bat} bypass routes V_{Bat} and GND directly to the reaction wheels, around the CubeADCS Core instead of through it. Please refer to CubeADCS ICD [RD2] for more detail. Appropriate reaction wheel harnesses will be supplied and must be plugged into the V_{Bat} Bypass board as shown in the figures below.

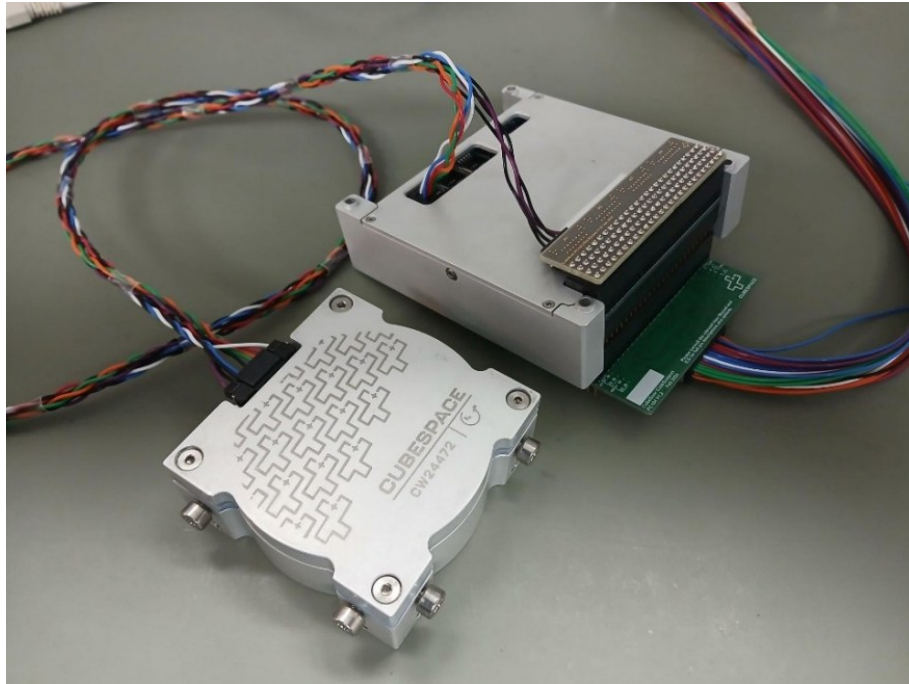


Figure 46: Single CubeWheel plugged into the PC104 variant of CubeADCS Core with a Top-Mount V_{Bat} Bypass board

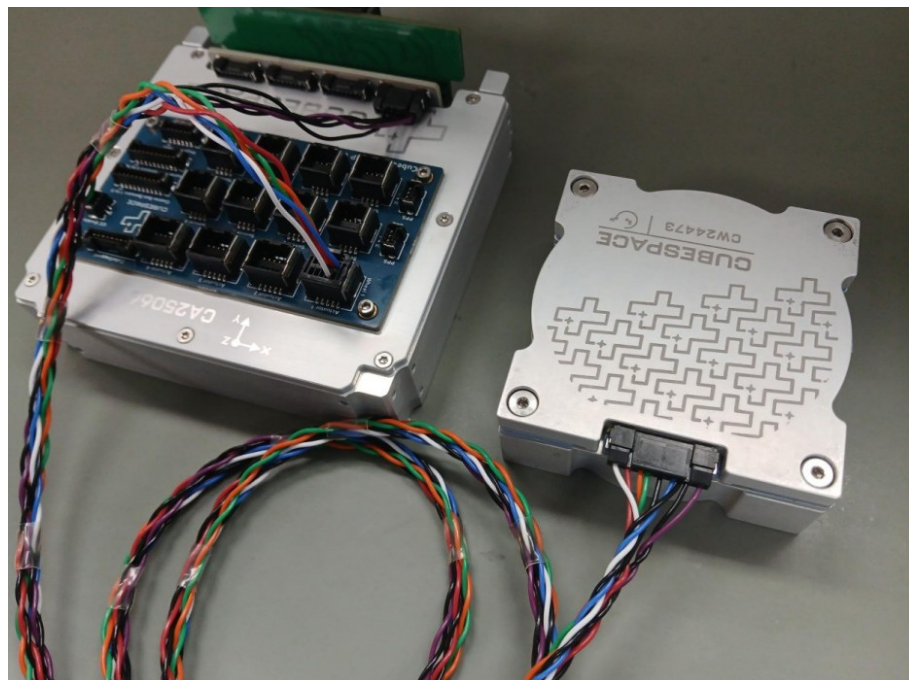


Figure 47: Single CubeWheel plugged into the Backplane variant of CubeADCS Core with a V_{Bat} Bypass board

Note that after a redesign of the CubeADCS Core's internal electronics, this V_{Bat} Bypass board (and alternative reaction wheel harnesses) will not be required anymore.



6.5.1.3 Power-on sequence for setups using the V_{Bat} Bypass PCB with CW0500 or larger reaction wheels

Supply the CubeSupport PCB with an independent Power Supply channel set to **12.0V, 1.5A** as usual.

Do not connect more than 17.6V to the CubeSupport PCB!



Ensure all power is off when plugging in harnesses into the CubeWheel. “Hot plugging” in a harness may damage the component

Note that the two GND wires of the broken-out V_{Bat} Return and the remaining single GND wire in the CubeSupport harness are internally connected on the CubeADCS, so the two Power Supply channels can remain completely independent.

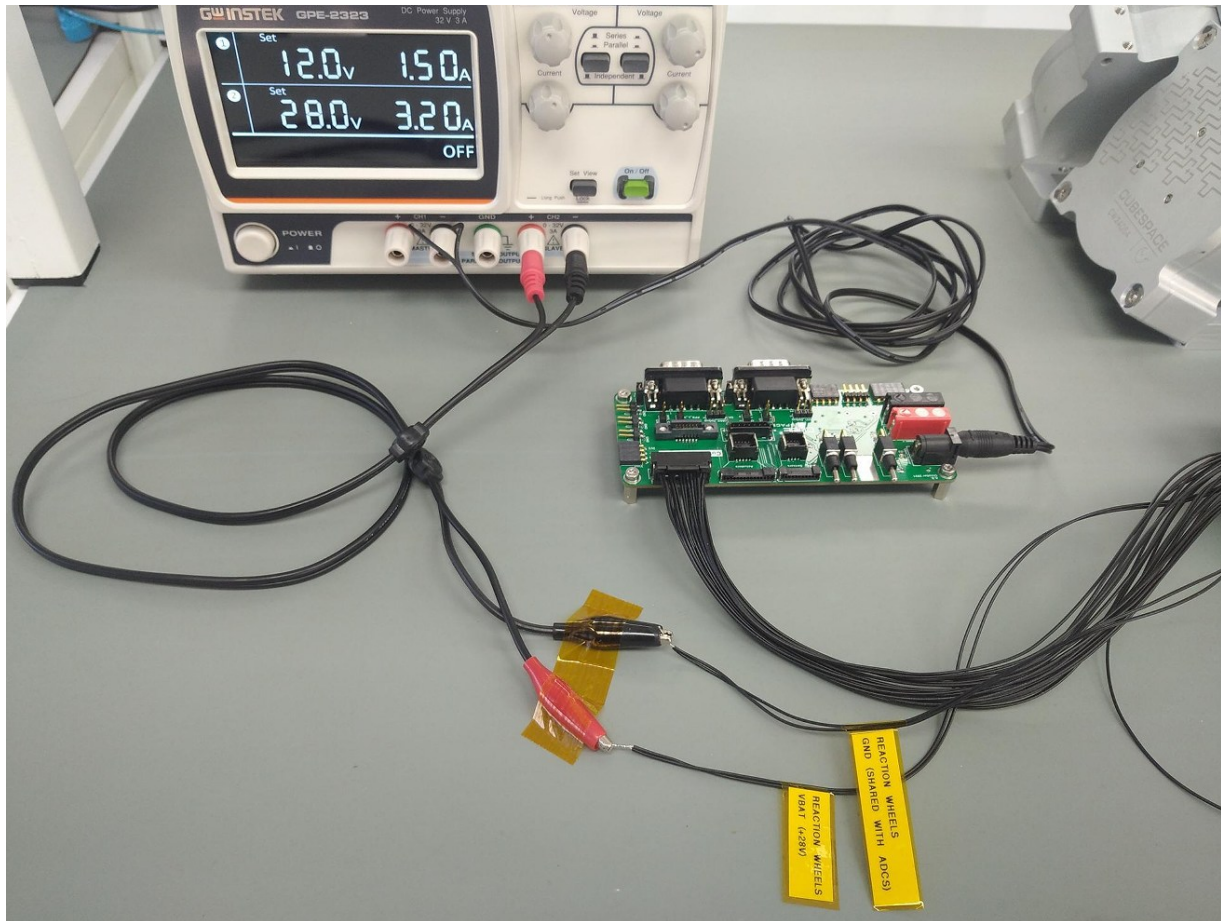


Figure 48: Setup of two independent Power Supply channels. Channel 1 = 12V for CubeSupport, Channel 2 = 28V for V_{Bat}

Once the setup is complete, enable power in the following sequence:

- 1) Switch on 12V power to the CubeSupport board.
- 2) Toggle on the “VBAT On” switch on CubeSupport board.
- 3) Toggle on the “Enable” switch on the CubeSupport Board.
- 4) Enable the supply for “REACTION WHEELS VBAT”.
- 5) Once bundle is powered, proceed with the steps below as usual.



6.5.2 Common health-check steps for all CubeWheel variants

In the steps below, commands will be sent to the CubeWheels via CubeComputer. After sending each command with the  button, refresh the field to ensure that the command has been received correctly. Follow the steps below to perform the CubeWheel health check via CubeComputer.

- 1) Power on the wheels as per Table 22. After each respective wheel has been powered on, verify and note the current increase caused by powering on each wheel using the “Idle CubeWheel MCU / Battery current” in Table 26 before powering on the next wheel.

Table 22: CubeWheel Health Check: Power On

Field	Expected Result	Actual result	Comments	P / F
Power State tab → cube-computer-common - PowerState				
RWL0 power state	PowerOn		Set, then Get to confirm receipt	
RWL1 power state	PowerOn		Set, then Get to confirm receipt	
RWL2 power state	PowerOn		Set, then Get to confirm receipt	
RWL3 power state	PowerOn		Set, then Get to confirm receipt	

To command the open-loop wheel speed, CubeComputer must be in control mode "ConNone". Verify this by navigating to the “Control mode” setting located at ADCS tab → cube-computer-5-control-program-8 - Commands → Control mode . If this is not true, select “ConNone” and send it to the CubeComputer.

- 2) After setting the control mode, refresh the telemetry and confirm that it has been registered.
- 3) Retrieve the wheel health telemetry and record the reported values for non-spinning wheels.

Table 23: CubeWheel Health Check: Expected (additional) Current Consumption

Field	Expected Result	Actual result	Comments	P / F
Health tab → cube-computer-5-control-program-8 – Health → Health Telemetry for Reaction Wheels				
Idle CubeWheel MCU current – POWER ON, not spinning	CW0017: ~25 mA CW0057: ~25 mA CW0162: ~25 mA CW0500: ~28 mA CW1200: ~25mA CW5000: ~28 mA Others: TBC (just record values for these and other rows please)	RWL0: RWL1: RWL2: RWL3:	Failed if > 40 mA (all variants)	
Idle CubeWheel Battery current – POWER ON, not spinning	CW0017: ~5 mA CW0057: ~5 mA CW0162: ~5 mA CW0500: ~10mA CW1200: ~10mA CW5000: ~15 mA	RWL0: RWL1: RWL2: RWL3:	Failed if > 20mA (all variants)	

- 4) Now command the wheel speeds as shown in Table 24. After each respective wheel has been commanded to the specified speed, verify and note the current increase caused by the wheel using “Active CubeWheel Battery current” in Table 26.



Note that for larger reaction wheels for example CW5000, a Power Supply current limit of 3.2A might be insufficient to spin up all four wheels simultaneously. In this case, don't spin up all four wheels simultaneously, but rather one after the other. Alternatively, more 28V Power Supply channels could be added in parallel.

Table 24: CubeWheel Health Check: Command Wheel Speed

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 – Commands → OpenLoopCommandRwl				
RWL open-loop speed command	2000 rpm	RWL0: RWL1: RWL2: RWL3:	Set, then Get to confirm receipt	

5) Verify that the wheels measure approximately the commanded speed, as shown in Table 25.

Table 25: CubeWheel Health Check: Expected Actual speeds

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 – AdcsTelemetrySensorRaw → Raw RWL sensor telemetry				
Timestamp	Incrementing			
RWL measured speed	2000 rpm	RWL0: RWL1: RWL2: RWL3:	±5 rpm	
RWL0 valid flag	Checked			
RWL1 valid flag	Checked			
RWL2 valid flag	Checked			
RWL3 valid flag	Checked			

6) Ensure that the current consumption values in Table 26 below are completed. If this is not the case, power off the wheels and repeat steps 1 to 5.

Table 26: CubeWheel Health Check: Expected (additional) Current Consumption

Field	Expected Result	Actual result	Comments	P / F
Health tab → cube-computer-5-control-program-8 – Health → Health Telemetry for Reaction Wheels				
Active CubeWheel Battery current – POWER ON, RWL Spinning at 2000 rpm	CW0017: ~12 mA CW0057: ~20mA CW0162: ~20mA CW0500: ~43mA CW1200: ~68mA CW5000: ~100 mA	RWL0: RWL1: RWL2: RWL3:	After allowing the wheel to reach constant speed.	

7) Command all the wheels' speed back to 0 (ADCS tab → cube-computer-5-control-program-8 - Commands → OpenLoopCommandRwl)

8) Switch off all the wheels (Power State tab → cube-computer-common → PowerState)



Once the values in the above tables have been confirmed, the CubeWheels are considered to be in working condition. If any result obtained does not match the expected result, please contact CubeSpace.

6.6 CubeSense Sun health check

NOTE: Due to the nature of the health check sequence, only one CubeSense Sun is tested at a time. Therefore, this section will refer to “FSSn”, where the “n” denotes the number of the CubeSense Sun being currently tested.

The gain and exposure settings for the CubeSense Sun must temporarily be adjusted for ground testing. Note that after power cycling the CubeSense Sun, these temporary settings will revert back to the default values, suitable for sun detection in orbit. Follow the steps below to temporarily adjust the gain and exposure settings.

Follow the steps below to perform the CubeSense Sun health check via CubeComputer:

- 1) Power on the CubeSense Sun as per Table 27.

Table 27: CubeSense Sun Health Check: Power On

Field	Expected Result	Actual result	Comments	P / F
Power State tab → cube-computer-common - PowerState				
FSSn power state	PowerOn	FSS0: FSS1: FSS2: FSS3:	Set, then Get to confirm receipt	

- 2) Verify that the current drawn from a bench power supply set to **12.0V** increases by the values shown in Table 28 below for each unit tested.

Table 28: CubeSense Sun Health Check: Expected Current Consumption

Field	Expected Result	Actual result	Comments	P / F
Active CubeSense Sun current – POWER ON	Consistently varying between ~15 mA and ~45 mA with a ~ 1 second period	FSS0: FSS1: FSS2: FSS3:	Failed if > 80 mA, or constant current	

- 3) At the top of the “PassThrough” tab, select the correct FSS from the Node dropdown menu , for example NodeFss0. If the correct FSS does not appear in the Node dropdown menu, click the refresh button .

Note: If the “PassThrough” tab is not visible due to a narrow display window, access it from the drop-down menu shown below:

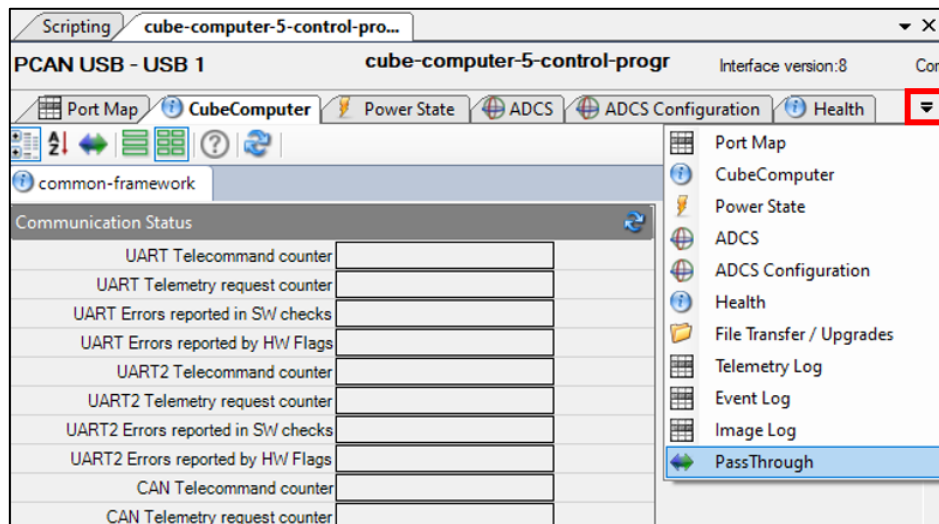


Figure 49: Accessing the Passthrough tab from the drop-down menu.

- 4) From the Program dropdown menu , select ProgramTypeControl.
- 5) Click the Connect button  to connect to the CubeSense Sun control application.
- 6) Scroll down the new view that appears and make the temporary settings in the camera settings fields (PassThrough tab  cube-sense-common  Set Camera Settings). Use the values shown in Table 29.
- 7) Verify that the settings were correctly sent by retrieving the configuration from the correct field (PassThrough tab  cube-sense-common  Configuration).
- 8) Don't power down the CubeSense Sun yet, but continue with the next steps starting with Step 1).

Table 29: CubeSense Sun Health Check: Adjusting settings to temporary values for ground testing

Field	Expected Result	Actual result	Comments	P / F
Passthrough tab → cube-sense-common – Set Camera Settings				
FSSn Camera Settings	Exposure: 200 Gain Control: 0 Blue Gain: 128 Red Gain: 128	FSS0: FSS1: FSS2: FSS3:	Can only Set, retrieve settings from the Configuration field.	

Follow the steps below to perform the CubeSense Sun health check via CubeComputer:

- 1) Remove any lens cap / protective tape from the lens and place it nearby. Note: The lens covering is intended to keep the lens protected from scratches, dust, and other foreign debris. This step should only be performed in a clean, dust-free area like a cleanroom. If the lens is contaminated or scratched, it can compromise the operation of the CubeSense Sun.
- 2) Using a small but bright light source (such as a mobile phone flashlight), illuminate the lens by holding the light source near the boresight of CubeSense at approximately 10 cm from the lens.
- 3) Request  the CubeSense Sun Raw telemetry and verify that the raw telemetry packet is as expected as per Table 30.

Note: If the angles aren't updating, ensure that the relevant FSS is in "PowerOn" state, not "PowerOnPass". This can be verified and changed in the Power State tab.



Table 30: CubeSense Sun Health Check: Expected Results

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 – AdcsTelemetrySensorRaw → Raw CubeSense Sun telemetry				
Timestamp	Incrementing			
FSSn alpha angle	Within viewing angle. This could be quite large, up to 20 – 30, as this measurement is highly dependent on the angle at which the light source is held relative to the camera boresight.	FSS0: FSS1: FSS2: FSS3:	Depends on which CubeSense Sun is currently being tested. Values of the units that were already tested might persist.	
FSSn beta angle		FSS0: FSS1: FSS2: FSS3:		
FSSn capture result	CaptureSuccessNormal OR CaptureSuccessFlipped Or: SunDetStartup (when not powered on)	FSS0: FSS1: FSS2: FSS3:		
FSSn detection result	Success Or: SunDetStartup (when not powered on)	FSS0: FSS1: FSS2: FSS3:		

- 4) Capture and download an image for each CubeSense Sun of the bundle as follows:
- 5) Store an image by navigating to the “Image Log” tab, selecting the relevant NodeFssx unit from the drop-down menu  , and subsequently clicking the Store button  . The image taken can be the same mobile phone flashlight used in the previous step.
- 6) **Note:** The CubeSense Sun’s current consumption will quickly increase by about 20 mA while the image is captured. When the current consumption returns to normal, the flashlight can be put away while the data transfer finishes.
- 7) Wait for the saving of the image to the CubeComputer’s non-volatile memory to finish.
- 8) The stored images on the CubeComputer can be refreshed by clicking the Request button  below the “Storage” heading in the “Image Log” tab. An image can be selected, and subsequently downloaded by clicking the Download button  .
- 9) Put the lens cap / protective film back, in order to protect the lens.
- 10) Verify that the downloaded image is as expected.
- 11) Repeat these steps for all additional CubeSense Sun units.
- 12) Capture the result in Table 31.

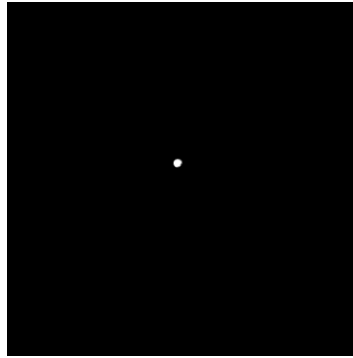


Figure 50: Example of captured CubeSense Sun image as pointed at a mobile phone flashlight.

Table 31: CubeSense Sun Health Check: Image download

Field	Expected Result	Actual result	Comments	P / F
Successful image capture and download for all CubeSense Sun units	Successful image captured and downloaded for all CubeSense Sun units	FSS0: FSS1: FSS2: FSS3:		

13) Power off the CubeSense Sun (Power State tab → cube-computer-common → PowerState).

14) Repeat this test for each individual CubeSense Sun unit.

Once the values in the above tables have been confirmed, the CubeSense Sun is considered to be in working condition. If any result obtained does not match the expected result, please contact CubeSpace.

6.7 CubeMag health check

NOTE: This section applies to both CubeMag Compact and CubeMag Deployable.

Follow the steps below to perform the CubeMag health check via CubeComputer:

- 1) Power on the CubeMag(s) as per Table 32, noting the current consumption of each respective CubeMag in Table 33 before powering on the next.

Table 32: CubeMag Health Check: Power On

Field	Expected Result	Actual result	Comments	P / F
Power State tab → cube-computer-common - PowerState				
MAG0 power state	PowerOn		Set, then Get to confirm receipt	
MAG1 power state	PowerOn		Set, then Get to confirm receipt	

- 2) Ensure the current consumption values in Table 33 below is completed. If this is not the case, power off the units and repeat Step 1).

Table 33: CubeMag Health Check: Expected Current Consumption

Field	Expected Result	Actual result	Comments	P / F
Active CubeMag current – POWER ON	CubeMag Compact: ~15 mA CubeMag Deployable: ~25 mA	MAG0: MAG1:	Failed if: CubeMag Compact: > 40 mA CubeMag Deployable: > 40 mA	

- 3) Request the CubeMag Raw telemetry and verify that the raw telemetry packet is as expected as per Table 34.



- 4) Slowly rotate each CubeMag about all axes and confirm that the magnitude of the raw vector is in the expected range (15 uT to 65 uT).
- 5) Slowly rotate each CubeMag about all axes and confirm that each axis displays both positive and negative values when rotating.

Table 34: CubeMag Health Check: Expected Results

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 – AdcsTelemetrySensorRaw → Raw Mag sensor telemetry				
Timestamp	Incrementing			
MAG0 raw vector X component	Varying with movement. Absolute Vector magnitude: 15-65 uT		If present	
MAG0 raw vector Y component				
MAG0 raw vector Z component				
MAG1 raw vector X component	Varying with movement. Absolute Vector magnitude: 15-65 uT		If present	
MAG1 raw vector Y component				
MAG1 raw vector Z component				
MAG0 valid flag	Checked		If present.	
MAG1 valid flag	Checked		If present.	

- 6) Switch off the CubeMag unit(s) (Power State tab → cube-computer-common → PowerState).

Once the values in the above tables have been confirmed, the CubeMag(s) is considered to be in working condition. If any result obtained does not match the expected result, please contact CubeSpace.

6.8 CubeSense Earth health check

Follow the steps below to perform the CubeSense Earth health check via CubeComputer:

- 1) Power on the CubeSense Earth unit(s) as per Table 35, noting the current consumption of each respective CubeSense Earth in Table 36 before powering on the next.

Table 35: CubeSense Earth Health Check: Power On

Field	Expected Result	Actual result	Comments	P / F
Power State tab → cube-computer-common - PowerState				
HSS0 power state	PowerOn		Set, then Get to confirm receipt	
HSS1 power state	PowerOn		Set, then Get to confirm receipt	

- 2) Ensure the current consumption values in Table 36 below is completed. If this is not the case, power off the units and repeat step 1.



Table 36: CubeSense Earth Health Check: Expected Current Consumption

Field	Expected Result	Actual result	Comments	P / F
Active CubeSense Earth current – POWER ON	Consistently varying between ~35 mA and ~50 mA with a ~ 1 second period		Failed if > 70 mA	

- 3) Request the raw CubeSense Earth telemetry and verify that the raw telemetry packet is as expected as per Table 37.

Table 37: CubeSense Earth Health Check: Telemetry

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 – AdcsTelemetrySensorRaw → Raw CubeSense Earth telemetry				
Timestamp	Incrementing			
HSS0 raw elevation angle	Near 0			
HSS0 raw rotation angle	Near 0			
HSS0 capture result	EarthCapSuccess			
HSS0 detection result	EarthNoHorizon		Could also indicate “Success” or “Too few edges”	
HSS1 raw elevation angle	Near 0			
HSS1 raw rotation angle	Near 0			
HSS1 capture result	EarthCapSuccess			
HSS1 detection result	EarthNoHorizon		Could also indicate “Success” or “Too few edges”	

- 4) Capture and download an image for each CubeSense Earth of the bundle as follows:
- 5) Remove any lens cap / protective tape from the lens and place it nearby. **Note: The lens covering is intended to keep the lens protected from scratches, dust, and other foreign debris. This step should only be performed in a clean, dust-free area like a cleanroom. If the lens is contaminated or scratched, it can compromise the operation of the CubeSense Earth.**
- 6) Store an image by navigating to the “Image Log” tab, selecting the relevant NodeHssx unit from the drop-down menu , and subsequently clicking the Store button . The image taken might be of a roof light or warm surface like a human face.
- 7) Wait for the saving of the image to the CubeComputer’s non-volatile memory to finish. It will take a short while.
- 8) Put the lens cap / protective film back in, order to protect the lens.
- 9) The stored images on the CubeComputer can be refreshed by clicking the Request button  below the “Storage” heading in the “Image Log” tab. An image can be selected, and subsequently downloaded by clicking the Download button .
- 10) Verify that the downloaded image is as expected.
- 11) Repeat these steps for all additional CubeSense Earth units.
- 12) Capture the result in Table 38.

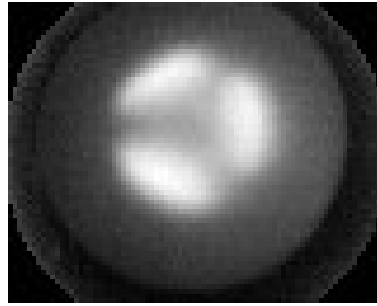


Figure 51: Example of captured CubeSense Earth image as pointed at a roof light.

Table 38: CubeSense Earth Health Check: Image download

Field	Expected Result	Actual result	Comments	P / F
Successful image capture and download for all CubeSense Earth units	Successful image captured and downloaded for all CubeSense Earth units	HSS0: HSS1:		

13) Switch off the CubeSense Earth unit(s) (Power State tab → cube-computer-common → PowerState).
Once the values in the above tables have been confirmed, the CubeSense Earth unit(s) is considered to be in working condition. If any result obtained does not match the expected result, please contact CubeSpace.

6.9 CubeStar health check

Follow the steps below to perform the CubeStar health check via CubeComputer:

1) Power on the CubeStar as per Table 39.

Table 39: CubeStar Health Check: Power On

Field	Expected Result	Actual result	Comments	P / F
Power State tab → cube-computer-common - PowerState				
STR0 power state	PowerOn		Set, then Get to confirm receipt	
STR1 power state	PowerOn		Set, then Get to confirm receipt	

2) Verify that the current drawn from a bench power supply set to **12.0V** increases by the values shown in Table 40 below for each unit tested.

Table 40: CubeStar Health Check: Expected Current Consumption

Field	Expected Result	Actual result	Comments	P / F
Active CubeStar current – POWER ON	Varying around ~20 - 60 mA with 1-second intervals	STR0: STR1:	Failed if > 80 mA	

3) Request  the CubeStar Health telemetry and verify that the Health telemetry packet is as expected as per Table 41.



Table 41: CubeStar Health Check: Expected Results

Field	Expected Result	Actual result	Comments	P / F
Health tab → cube-computer-5-control-program-8 – Health → Health telemetry for CubeStar				
STR0 MCU Temperature	~ 25 °C		Depends on ambient temperature	
STR0 MCU Current	~ 17 mA			
STR0 MCU Voltage	~ 3300 mV			
STR0 Current	~ 1 - 10 mA			
STR0 CAM Overcurrent	Not Checked			
STR0 Error Code	0			
STR1 MCU Temperature	~ 25 °C		Depends on ambient temperature	
STR1 MCU Current	~16 mA			
STR1 MCU Voltage	~ 3300 mV			
STR1 Current	~ 1 - 10 mA			
STR1 CAM Overcurrent	Not Checked			
STR1 Error Code	0			

4) Capture an image from the CubeStar, first with the **lens cap on**, as follows:

- 4.1) Temporarily halt the periodic capture of images by placing the relevant STR in a “PowerOnPass” state (Power State tab → cube-computer-common – PowerState → PowerState).
- 4.2) Store an image by navigating to the “Image Log” tab, selecting the relevant NodeStrx unit from the drop-down menu , and subsequently clicking the Store button .

Note: The CubeStar’s current consumption will quickly increase by about 40 mA while the image is captured. When the current consumption returns to normal, the image capture is complete.

Wait for the saving of the image to the CubeComputer’s non-volatile memory to finish. This image will be downloaded during a later step.

5) Capture a second image from the CubeStar, with the lens cap removed, as follows:

- 5.1) Remove any lens cap / protective tape from the lens and place it nearby. **Note: The lens covering is intended to keep the lens protected from scratches, dust, and other foreign debris. This step should only be performed in a clean, dust-free area like a cleanroom. If the lens is contaminated or scratched, it can compromise the operation of the CubeStar.**
- 5.2) Store an image by navigating to the “Image Log” tab, selecting the relevant NodeStrx unit from the drop-down menu , and subsequently clicking the Store button .

Note: The CubeStar’s current consumption will quickly increase by about 40 mA while the image is captured. When the current consumption returns to normal, the image capture is complete.

6) Put the lens cap / protective film back, in order to protect the lens.

7) Wait for the saving of the image to the CubeComputer’s non-volatile memory to finish.



8) Retrieve the two captured CubeStar images:

8.1) The stored images on the CubeComputer can be refreshed by clicking the Request button  below the “Storage” heading in the “Image Log” tab. An image can be selected, and subsequently downloaded by clicking the Download button .

8.2) Download the two images and verify that the downloaded images are as expected.



Figure 52: An image taken with CubeStar.

9) Switch off the CubeStar unit (Power State tab  cube-computer-common  PowerState).

10) Repeat these steps for all additional CubeStar units.

11) Capture the result in Table 42.

Table 42: CubeStar Health Check: Image download

Field	Expected Result	Actual result	Comments	P / F
Successful image capture and download for all CubeStar units	Successful image captured and downloaded for all CubeStar units	STR0: STR1:	There is a known issue in the CubeSupport Application at the time of writing this document, that prevents the saving of the downloaded CubeStar bitmap correctly. The displayed image can be saved by taking a screenshot of the CubeSupport tab.	

12) Switch off the CubeStar unit(s) (Power State tab  cube-computer-common  PowerState).

Once the values in the above tables have been confirmed, the CubeStar unit(s) is considered to be in working condition. If any result obtained does not match the expected result, please contact CubeSpace.



6.10 CubeAuriga health check

6.10.1 Preparation

The Auriga optical head(s) that will interface with CubeAuriga, is required in order to perform the following tests. Connect each Auriga optical head (OH) to CubeAuriga using the supplied harness(es).



NOTE: Correct mating of the Micro-D connectors is crucial to avoid contact damage. Each jackscrew should be tightened alternately 1 turn per side until the recommended mating torque is achieved on both sides. The manufacturer's recommended mating torque for both the 15W and 21W connectors' jackscrews is 28 cNm.

CubeAuriga requires an optical head configuration binary file to be present in its flash memory. This file is uploaded by CubeSpace to the CubeAuriga unit prior to shipping, and this procedure is not normally expected to be performed by clients. The procedure is detailed in Section 6.10.3, in case a client needs to perform this.



NOTE: The CubeSpace Final AIT team must first perform the steps in Section 6.10.3 before proceeding.

6.10.2 Health Check

Follow the steps below to perform the CubeAuriga health check via CubeComputer.

1) Power on the CubeAuriga as per Table 43.

Table 43: CubeAuriga Health Check: Power On

Field	Expected Result	Actual result	Comments	P / F
Power State tab → cube-computer-common - PowerState				
STR0 power state	PowerOn		Set, then Get to confirm receipt	
STR1 power state	PowerOn		Set, then Get to confirm receipt	

2) Verify that the current drawn from a bench power supply set to **12.0V** increases by the values shown in Table 44 below for each unit tested.

Table 44: CubeAuriga Health Check: Expected Current Consumption

Field	Expected Result	Actual result	Comments	P / F
Active CubeAuriga current – POWER ON	CubeAuriga typically draws up to 270 mA with 1x OH (110 for CG + 160 for 1x OH) or up to 450 mA with 2x OHs (130 for CG + 320 for 2x OHs).	STR0: STR1:	Fail if > 330 mA (single OH) Fail if > 540 mA (2x OH)	

3) Ensure the correct OH calibration binary has been uploaded to CubeAuriga as follows:



NOTE: This step is a crucial part of the client-side health check. If it fails, follow the steps in Section 6.10.3 and retry. In the case of a repeated failure, stop the checks and please contact CubeSpace.

- 4) At the top of the “PassThrough” tab (see Figure 49), select the correct STR from the Node dropdown menu , for example NodeStr0. If the correct STR does not appear in the Node dropdown menu, click the refresh button .
- 5) From the Program dropdown menu , select ProgramTypeControl.
- 6) Click the Connect button  to connect to the CubeAuriga Control Program.
- 7) Scroll down the new view that appears and retrieve the Identification telemetry. Confirm that Node type identifier equals “NodeTypeLegacyCubeAuriga” and check that Runtime advances when the refresh button  is clicked.
- 8) Retrieve the ID of each Optical Head’s calibration parameters by clicking the refresh button  in the Auriga Status TM field (PassThrough tab  cube-auriga-1-control-program-1 – Attitude  Auriga Status TM).
- 9) The field “OHx Calib Param Id” for each OH should correspond to the last four digits of the serial number on the physical optical head label as shown below:

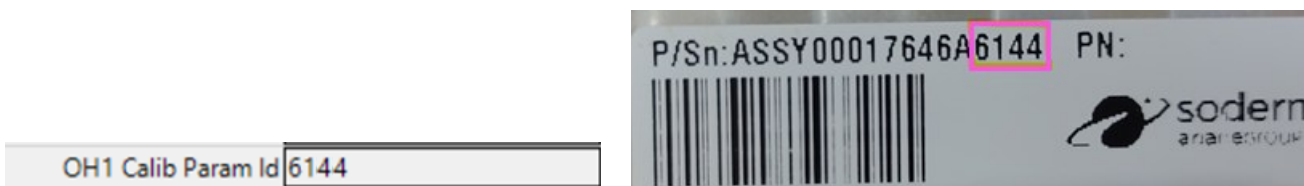


Figure 53: Confirming the correct calibration binary uploaded to the CubeAuriga.



Table 45: Optical Head Calibration binary Param IDs

Field	Expected Result	Actual result	Comments	P / F
Passthrough tab → cube-auriga-1-control-program-1 – Attitude → Auriga Status TM				
P/Sn on Optical Head 1 label			Record the full number from the label as shown in Figure 53	
Reported OH1 Calib Param Id	Last four digits must match with P/Sn in the row above		Fail if not matching. Follow Section 6.10.3 and retry.	
P/Sn on Optical Head 2 label			Record the full number from the label as shown in Figure 53	
Reported OH2 Calib Param Id	Last four digits must match with P/Sn in the row above		Fail if not matching. Follow Section 6.10.3 and retry.	

10) While still in Passthrough mode, perform the following self-tests:

10.1) Send a command to perform a self-test of the SRAM (PassThrough tab → cube-auriga-1-control-program-1 – Health → Trigger External SRAM Test) with the Magic Number value set to 591751049 (0x23456789).

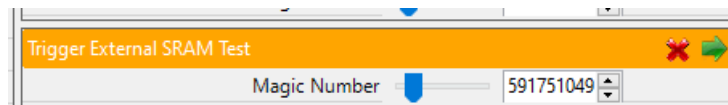


Figure 54: Setting the correct Magic Number to perform the SRAM self-test.

10.2) Refresh the Health → SRAM Test Result field until “SRAM Test State” equals “Idle” (it takes around 3 seconds). Note down result in Table 46.

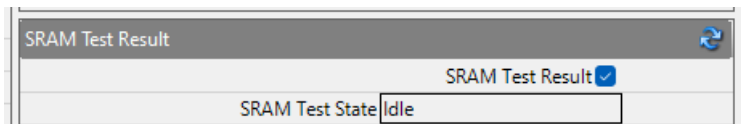


Figure 55: SRAM self-test result.

10.3) Send a command to perform a self-test of the Auriga optical head (PassThrough tab → cube-auriga-1-control-program-1 – Health → Trigger Auriga OH Self Test) with the Magic Number value set to 305419896 (0x12345678).



Figure 56: Setting the correct Magic Number to perform the OH self-test.

10.4) Refresh the Health → Auriga OH Self Test Results field until “Self-test State” equals “Idle” (it takes around 20 seconds). Note down result in Table 46.

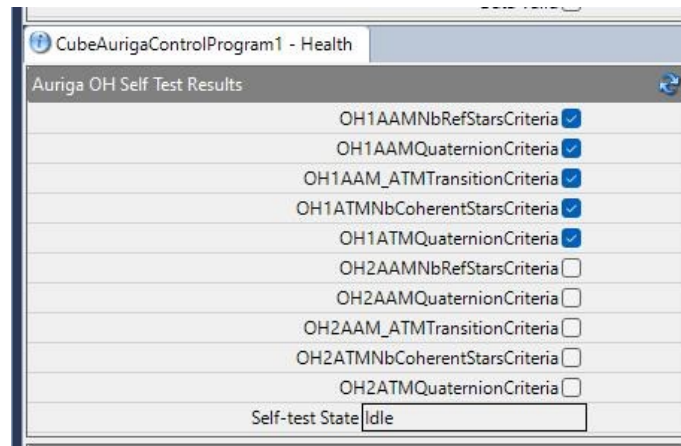


Figure 57: Auriga optical head self-test result.

Table 46: CubeAuriga Test results

Field	Expected Result	Actual result	Comments	P / F
Passthrough tab → cube-auriga-1-control-program-1 – Health				
External SRAM Test	Box is checked		Fail if box not checked	
Auriga OH Self Test	5 boxes checked for each OH connected. Only one OH is tested at a time. Repeat test if more than one OH connected.		Fail if all 5 boxes not checked	

11) The Passthrough connection can now be disconnected.

12) Request (🔍) the CubeAuriga Health telemetry and verify that the Health telemetry packet is as expected as per Table 47.

Table 47: CubeAuriga Health Telemetry: Expected Results

Field	Expected Result	Actual result	Comments	P / F
Health tab → cube-computer-5-control-program-8 – Health → Health telemetry for CubeAuriga				
STR0 MCU Voltage	~ 30 °C		Depends on ambient temperature	
STR0 MCU Current	~ 50 mA		Fail if > 60 mA	
STR0 MCU Voltage	~ 3150 - 3350 mV		Fail if outside expected range	
STR0 OH1 Current	~ 150 mA ² (180 – 300 mA if using loopback tester)		Fail if <120 or > 190	
STR0 OH2 Current	~ 150 mA ^{1, 2} (0 mA if no OH2 connected) (180 – 300 mA if using loopback tester)		Fail if <120 or > 190 ¹	
STR0 FPGA1 Voltage	~1200 mV		Fail if <1100 or > 1250	
STR0 FPGA2 Voltage	~1200 mV		Fail if <1100 or > 1250	
STR0 FPGA1 Current	~40 mA		Fail if <30 or > 50	



Field	Expected Result	Actual result	Comments	P / F
STR0 FPGA2 Current	~40 mA ¹ (0 – 3 mA if no OH2 connected)		Fail if <30 or > 50 ¹	

Notes: 1. Only if OH2 is also connected to CubeAuriga. Without OH2, the expected value is 0 – 3 mA.
2. If the SpaceWire loopback tester is used, expected OH1 and OH2 currents will be 250 mA.

13) Request the Raw CubeAuriga telemetry and verify as per Table 48:

Table 48: Raw CubeAuriga Telemetry expected values

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → Raw CubeAuriga telemetry				
STRx OH1 State	"OhStateAcqFF"			
STRx OH2 State	"OhStateAcqFF" if OH is connected, otherwise "OhStateOff"			
STRx Data Valid	Box is checked		Fail if box not checked	

Note: STRx → the x depends on whether CubeAuriga is configured as STR0 or STR1. (Likely STR0).

14) Switch off the CubeAuriga unit (Power State tab → cube-computer-common → PowerState).

Once the values in the above tables have been confirmed, the CubeAuriga is considered to be in working condition. If any result obtained does not match the expected result, please contact CubeSpace.

6.10.3 Uploading the Optical Head Configuration binary file

CubeAuriga requires a binary configuration file which contains the calibration info of the optical heads that will be connected to it. Please request this file from CubeSpace.

- 1) Power off the CubeSupport PCB and disconnect the harness connected to the CubeADCS.
- 2) Plug the CubeAuriga into the CubeSupport PCB's Sensors port, using its supplied harness.
- 3) Swop the PCAN cable from the "CAN2_CC" port (See Figure 19) and ensure it is instead plugged into the "CAN1_Nodes" port.
- 4) Power on the CubeSupport PCB and establish a CAN connection to the CubeAuriga bootloader within 5 seconds.
- 5) Ensure that the "Use CubeSat Space Protocol (CSP)" box remains **deselected**.
- 6) The CAN address scan range  can be widened from "1 to 10" to "1 to 100".
- 7) If it takes longer than 5 seconds to establish the CAN connection, the Control Program might be running instead of the bootloader. In that case, perform a "Reset to Bootloader" procedure (Advanced tab → common-framework - Commands → Reset → ToBootloader) as in Section 6.2.
- 8) Once the Bootloader view opens, browse the Local Files to select the configuration file to upload and send it. When complete, the file list will refresh. It should now show 3 files:



The screenshot shows a software interface titled 'CAN BUS' with a sub-header 'cube-common-base-bootloader-6'. It includes tabs for 'Firmware/Config Upload', 'Advanced', 'Raw Memory Access', and 'Upgrade (CubeComputer ...)'. Below these is a 'Remote Files' section with a table listing files. The table has columns: File Han..., Corr..., Type, Program, Application Version, System Version, Owner, Address, and Size. Three files are listed: File Handle 0 (Control, 0.0.0), File Handle 1 (Control, 1.1.0), and File Handle 2 (Cfg, 1.0.5).

File Han...	Corr...	Type	Program	Application Version	System Version	Owner	Address	Size
☐ 0	False	Bin	Control	0.0.0	0.0.0	0	8020000	3104
☐ 1	False	Bin	Control	1.1.0	4.7.0	0	8040000	1271324
☐ 2	False	Cfg	Control	1.0.5	4.7.0	CG24001	81C0000	577

Figure 58: File list showing the three required files.

- 9) Next, configure CubeAuriga’s bootloader to jump to the Control Program by default, instead of the newly uploaded configuration binary file (which will be at File Handle 0). Go to the ‘Advanced’ tab and look for ‘Default application target’. Set the File Handle value to “1” and click Send:

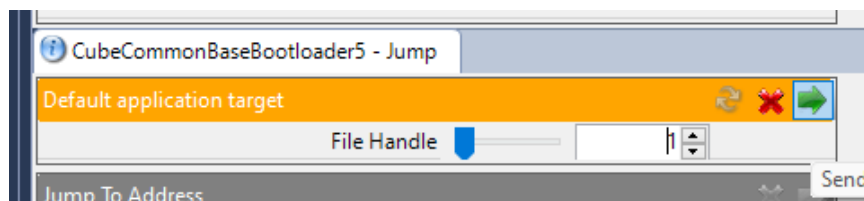


Figure 59: Reconfiguring the default jump target.

- 10) The CubeAuriga Control Program can now be run, by either of the following:
- 10.1) Command a “Jump to Application” as shown in Figure 41, ensuring that the correct “Bin” file at File Handle 1 is selected first, or
 - 10.2) Close the communication connection, power cycle the CubeSupport PCB by toggling the VBAT switch, and wait 5 seconds for the bootloader to jump to the Control Program. Reestablish the communication connection.
- 11) From the updated view, or by requesting the Identification telemetry, confirm that the CubeAuriga bootloader has successfully jumped to the Control Program.
- 12) Restore the original setup:
- 12.1) Close the CAN connection.
 - 12.2) Power off the CubeSupport PCB.
 - 12.3) Unplug the CubeAuriga from the CubeSupport PCB and plug it back into the CubeADCS.
 - 12.4) Switch the PCAN cable back to the “CAN2_CC” port.
 - 12.5) Plug the CubeADCS back into the CubeSupport PCB using the supplied harness.

6.11 CSS health check

NOTE: This section assumes that 10 sensors will be used for the health check.

Follow the steps below to perform the CSS health check via CubeComputer.

- 1) Connect the sensors to the CSS Ports on CubeConnect.
Note: Each sensor will be automatically given an ID based on the position of the sensor in the array and the port it is connected to.
 - Sensors connected to “Coarse Sun Sensors 1-5” port will be labelled CSS0 to CSS4.
 - Sensors connected to “Coarse Sun Sensors 6-10” port will be labelled CSS5 to CSS9.
- 2) Request raw CSS telemetry and take note that:
 - 2.1) The sensor readings are valid (between 0 and 255),



- 2.2) The valid flag is checked,
- 2.3) The Time in seconds and nano seconds advances.
- 3) Take a light source such as a torch, cell phone torch, lamp etc. and shine a light directly onto the sensors.
- 4) Request raw CSS telemetry again and take note that:
 - 4.1) The valid flag is still checked,
 - 4.2) The CSS readings have increased.



NOTE: Coarse Sun Sensor photodiodes are chosen to have a good response to sunlight, which is very bright in orbit. An LED from a cell phone might not be bright enough unless pressed directly against the photodiode.

- 5) Capture the results in Table 49.

Table 49: CSS Health Check: Expected Results

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 – AdcsTelemetrySensorRaw → Raw CSS sensor telemetry				
Timestamp	Incrementing			
CSS0 raw measurement	0 – 255		The result of each sensor will be a positive integer, increasing with light intensity.	
CSS1 raw measurement	0 – 255			
CSS2 raw measurement	0 – 255			
CSS3 raw measurement	0 – 255			
CSS4 raw measurement	0 – 255			
CSS5 raw measurement	0 – 255			
CSS6 raw measurement	0 – 255			
CSS7 raw measurement	0 – 255			
CSS8 raw measurement	0 – 255			
CSS9 raw measurement	0 – 255			
CSS valid flag	Checked (True) or unchecked (False)		Indicates whether the combined readings are within expected range. For this ground test, this checkbox can be ignored.	

If all the steps have been followed and the associated expected results achieved, the CSS health check can be considered successful, and the CSS circuit is working as expected. If any result obtained does not match the expected result, please contact CubeSpace.

6.12 GNSS interface health check

This section discusses the testing of the GNSS interface. While the CubeADCS supports the transmitting of GNSS data from the OBC to CubeADCS via telecommand, this procedure must happen every second at < 100 ms latency. It is thus preferred that the GNSS_UART_RX line of the CubeADCS is connected to the Data Output pin of the GNSS module to directly receive the data. Both these methods of updating the GNSS data is tested as discussed below.



6.12.1 GNSS data update via telecommand

To perform the GNSS data update via telecommand, follow the steps described below. Unsolicited telemetry is first set up to assist with checking of the results.

- 1) Using the CubeSupport software, connect to the CubeADCS. This might be over CAN, UART, RS422 or RS485. **Note:** Unsolicited Telemetry over I2C is not supported at the time of writing this document.
- 2) Locate the settings at CubeComputer tab → cube-computer-common - Logging → Unsolicited Telemetry Message Setup .
- 3) If connected over CAN, the settings in the “CAN Config” section needs to be changed.
- 4) If connected over UART, the settings in the “UART Config” section needs to be changed.
- 5) If connected over RS422 / RS485, the settings in the “UART2 Config” section needs be changed.
- 6) From the drop-down menu, select “interval1s”, make a check in the box next to “Raw GNSS sensor telemetry”, and send the command to the CubeComputer.
- 7) A new tab, Unsolicited Telemetry, will now have a container for Raw GNSS telemetry which will continuously be updated with the CubeComputer’s latest information.
- 8) A telecommand can be sent via ADCS tab → cube-computer-5-control-program-8 - Commands → Commanded GNSS measurements to update the CubeComputer’s GNSS data. Example data is shown in the table below.

Field	Value	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 – Commands → Commanded GNSS measurements			
GNSS Time integer seconds	Within 1 year of current Unix time, in integer format. This can be retrieved by first retrieving the Raw GNSS telemetry (ADCS tab → control-program – AdcsTelemetrySensorRaw → Raw GNSS sensor telemetry)	s	
GNSS Time nanoseconds	0	ns	
Satellite position vector X component (GNSS frame)	1500000	cm	
Satellite position vector Y component (GNSS frame)	-1600000	cm	
Satellite position vector Z component (GNSS frame)	1700000	cm	
Satellite velocity vector X component (GNSS frame)	-60000	cm/s	
Satellite velocity vector Y component (GNSS frame)	70000	cm/s	
Satellite velocity vector Z component (GNSS frame)	-80000	cm/s	
Sync time	Checked		

- 9) From the Unsolicited Telemetry tab, verify that the results match the data that was sent.



Ensure that the RTC Time (s) value in the Commanded GNSS measurements packet does not differ from the current RTC time by more than 32000000 seconds (1 year), otherwise the command will get rejected. The current RTC Time can easily be determined by retrieving the Raw GNSS telemetry first.



If no GNSS update is received by the CubeComputer for 1.5 seconds, the position and velocity vectors will revert back to 0. It is thus required that after sending commanded GNSS measurements over telecommand, that the Unsolicited Telemetry tab is checked very quickly. Doing this check can be made easier with the CubeSupport Application's scripting functionality, but this is not within the scope of this document.

10) Fill in Table 50 below.

Table 50: GNSS Update over telecommand check: Expected Results

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → control-program – AdcsTelemetrySensorRaw → Raw GNSS sensor telemetry				
RTC Time (s)	Equal to GNSS-supplied +/- 1		RTC time (PPS corrected)	
RTC Time (ns)	0 – 999'999'999		RTC time (PPS corrected)	
RTC Drift (ns)	0 – 999'999'999		RTC clock drift	
Satellite ECEF position vector X component	Identical to set values. Note that units of the commanded values are in cm, however the units of the Raw Telemetry packet are in m and will therefore be 100 times smaller than the commanded values.		m	
Satellite ECEF position vector Y component			m	
Satellite ECEF position vector Z component			m	
Satellite ECEF velocity vector X component			m/s	
Satellite ECEF velocity vector Y component			m/s	
Satellite ECEF velocity vector Z component			m/s	
GNSS-supplied unix time integer seconds	Equal to GNSS-supplied		GNSS time	
GNSS-supplied unix time nanoseconds	0 – 999'999'999		N/A (always zero)	
GPS week number	Identical to set values. These could be 0.			
GPS Time of Week				
GPS UTC Offset				
GNSS data is valid	Checked		The displayed data is valid	
GNSS has lock	Checked		The displayed data is recent	
GNSS PPS detected	Checked or Unchecked		GNSS PPS signal input detected	



6.12.2 GNSS data update via UART interface

CubeADCS is usually configured to accept GNSS data over the GNSS UART interface. It can be tested by either directly connecting the actual GNSS module, or emulating the module by way of the FTDI UART cable that forms part of the supplied Ground Support Equipment. CubeSpace does not have or supply GNSS modules, and therefore tests the UART interface using the FTDI cable during in-house health checking. The GNSS UART input pin is usually supplied in an open-ended form on the CubeSupport harness. Figure 60 and Figure 61 shows this setup. Note that the client can substitute the FTDI UART cable in these figures with the actual GNSS module.

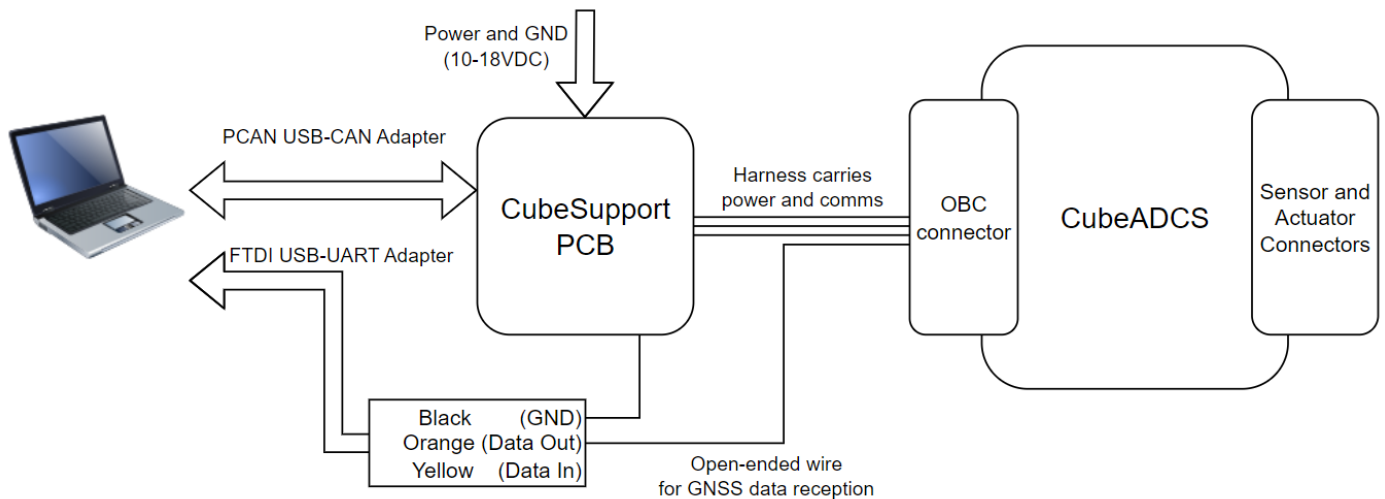


Figure 60: Diagram of GNSS UART interface test.

Shown below is the setup for injecting GNSS UART packets into the CubeADCS. Note that the primary CAN communications channel is used, and the USB-UART cable included in the Ground Support Equipment is connected to the CC_LPUART_RX wire of the CubeSupport harness using the middle (orange) data output pin. The black (GND) pin is connected to one of the several GND connections on the CubeSupport PCB, shown in Figure 62.

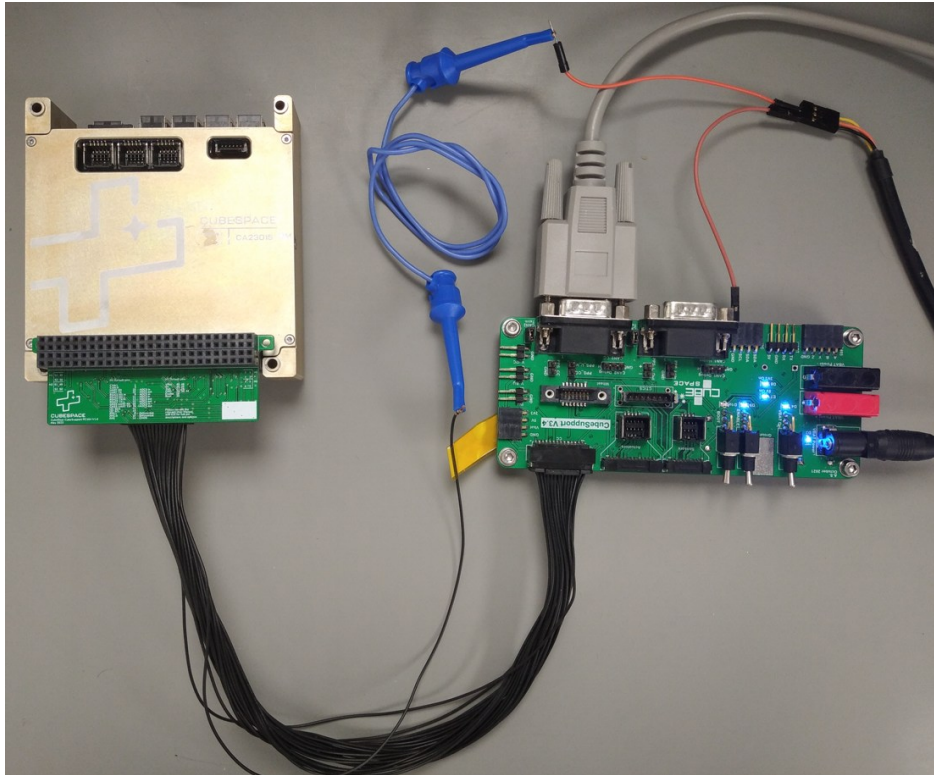


Figure 61: GNSS UART test setup.

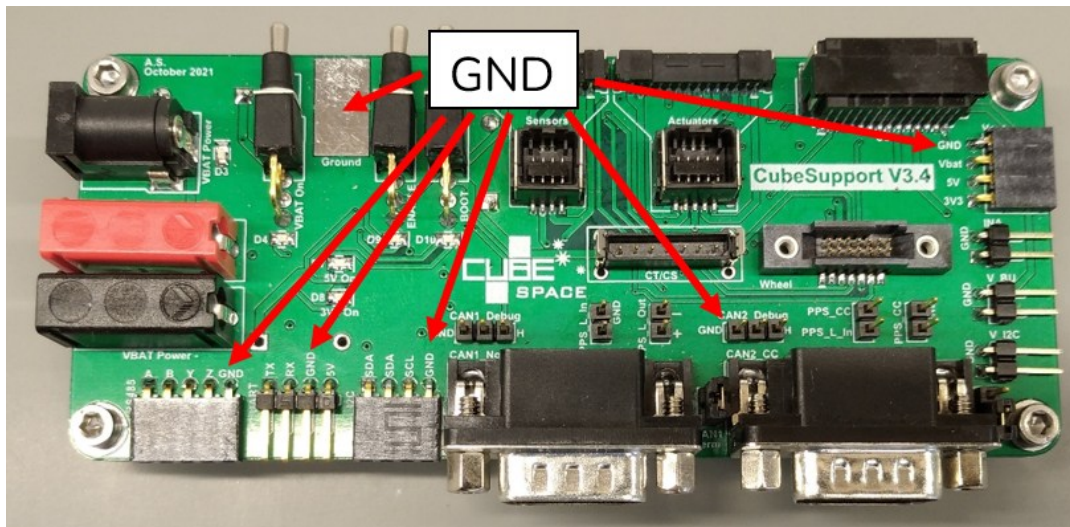


Figure 62: GND points on the CubeSupport PCB.

Case 1: Testing with the actual GNSS module

If not testing with the actual GNSS module, this paragraph can be skipped.

- 1) Power off the CubeADCS by moving the “VBAT on” switch on the CubeSupport PCB to the OFF position.
- 2) Make the required connections as shown in Figure 60 and Figure 61, with the FTDI UART cable replaced by the GNSS module.
- 3) Power on the CubeADCS by moving the “VBAT on” switch on the CubeSupport PCB to the ON position.



- 4) Power on and set up the GNSS module. Ensure that its antenna is properly connected.



CubeADCS doesn't power or set up the GNSS module. It is the responsibility of the client to own, power, and setup the GNSS module for binary protocol communication. Ensure that the GNSS module is correctly set up as required.

- 5) After the CubeADCS has finished booting into the Control-Program, the GNSS data sent by the module can be retrieved by refreshing the ADCS tab → control-program - AdcsTelemetrySensorRaw → Raw GNSS sensor telemetry window. Ensure that these results are realistic and fill in Table 52.

Case 2: Testing using the FTDI UART cable as GNSS emulator

If not testing with the FTDI UART cable, this paragraph can be skipped.

- 1) Power off the CubeADCS by moving the “VBAT on” switch on the CubeSupport PCB to the OFF position.
- 2) Make the required connections as shown in Figure 60 and Figure 61.
- 3) Power on the CubeADCS by moving the “VBAT on” switch on the CubeSupport PCB to the ON position.
- 4) Using the CubeSupport software, connect to the CubeADCS. This will likely be over CAN, since the supplied UART cable is in use.
- 5) After the CubeADCS has finished booting into the Control-Program, set up Unsolicited Telemetry again by following Steps 2) to 7) from the previous section.
- 6) The actual data to be sent over the UART interface depends on the configured GNSS protocol. Consult Table 51 to obtain the data to be sent. Note that some bytes are ignored by the CubeADCS and are set to 0x00 for this test, but this will not be the case if the data came from an actual GNSS module.

Table 51: GNSS interface testing: UART data

[illegible]



The data retrieved from the table above, can be sent to the CubeADCS using the Python code shown in Figure 63. The single packet required by the SkyTraq protocol is shown, being transmitted out of COM4, using the following settings:

- Baudrate: 115200
- Data bits: 8
- Stop bits: 1
- Parity: None
- Flow control: None

```
import time
import serial

if __name__ == '__main__':
    skyTraq_BinaryNMEA_msg = b'\xA0\xA1\x00\x3B\xA8\x02\x08\x04\xD2\x00\xBC\x61\x4D\x00\x00\x00'
    skyTraq_BinaryNMEA_msg += b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
    skyTraq_BinaryNMEA_msg += b'\x00\x00\x00\x00\x00\x00\x00\xF8\xA4\x33\x44\xC5\x21\x97\x64\x07'
    skyTraq_BinaryNMEA_msg += b'\x5B\xCC\xBC\x00\x12\xD6\x44\xFF\xAD\x1C\xDC\xFF\xA9\x58\xCC\x24'
    skyTraq_BinaryNMEA_msg += b'\x0D\x0A'

    ser = serial.Serial("COM4", 115200)
    ser.write(skyTraq_BinaryNMEA_msg)
```

Figure 63: Python code for sending GNSS data.

Alternatively, the data can be sent using a terminal program like “Realterm” or “Terminal by Br@y++”. In the case of these two programs, a hex editor program like “HxD” will need to be used to create a binary file first (see Figure 64), before sending it out of the serial port.

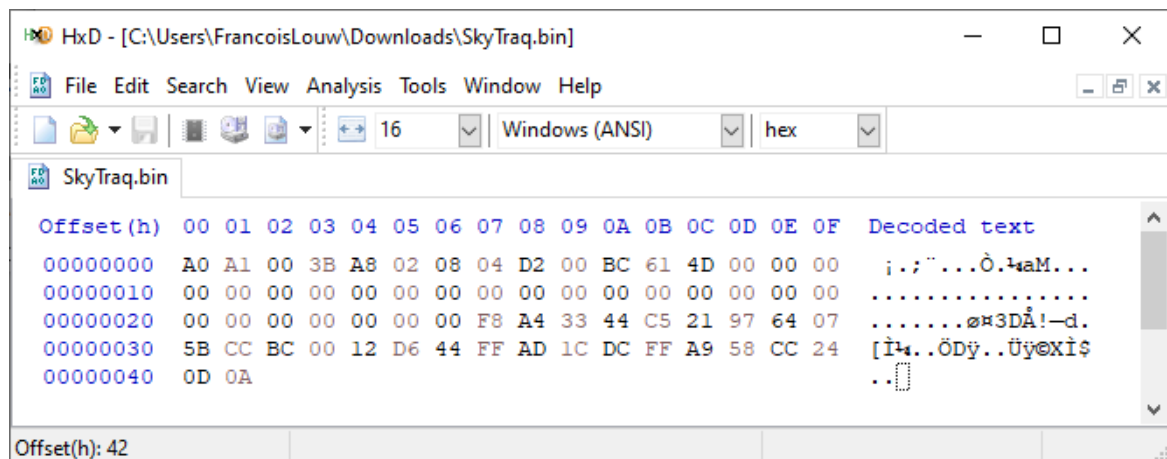


Figure 64: Creating a binary file using HxD.

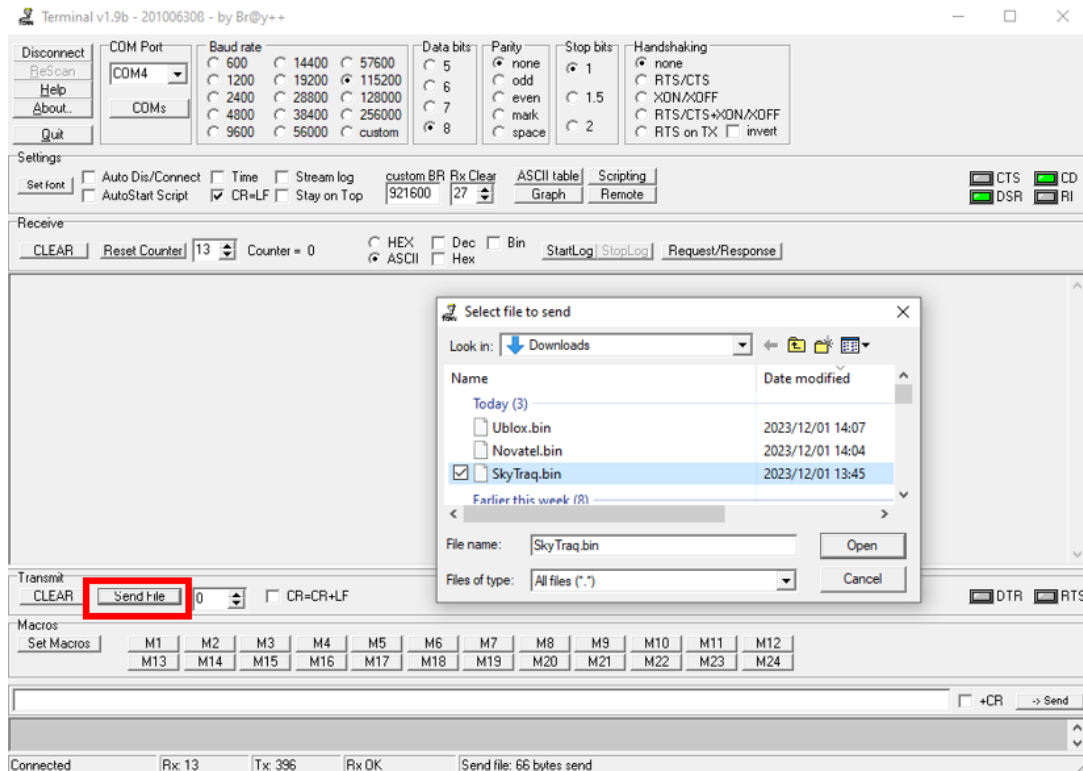


Figure 65: Sending the created binary file using Terminal by Br@y++.

The GNSS data sent over the UART cable can be verified by checking the retrieved data from the Unsolicited Telemetry tab. It can also be verified by refreshing the ADCS tab  control-program - AdcsTelemetrySensorRaw  Raw GNSS sensor telemetry window, however this must be done within 1.5 seconds of sending the data, or the results will revert back to 0.

The Position and Velocity vectors must match what is shown in Figure 66 to verify that the GNSS interface is healthy. The RTC and GNSS supplied time, and UTC offset could vary. Verify that the results match the expected values and fill in Table 52 below. Note that some values in Figure 66 have a yellow background, indicating they are outside of expected ranges, and that the “GNSS has lock” checkbox is thus not set.

Raw GNSS sensor telemetry		
RTC Time (s)	3029549370	s
RTC Time (ns)	312591552	ns
RTC Drift (ns)	0	ns
Satellite ECEF position vector X compon	-1 234 567.00	m
Satellite ECEF position vector Y compon	-9 876 543.00	m
Satellite ECEF position vector Z compon	1 234 567.00	m
Satellite ECEF velocity vector X compon	12 345.00000	s
Satellite ECEF velocity vector Y compone	-54 321.00000	m/s
Satellite ECEF velocity vector Z compon	-56 789.00000	m/s
GNSS-supplied unix time integer secon	1062411456	s
GNSS-supplied unix time nanoseconds	0	ns
GPS week number	1234	
GNSS Time of Week	123456780	ms
GNSS UTC Offset	0	s
GNSS data is valid ☒		
GNSS has lock ☐		
GNSS PPS Detected ☐		

Figure 66: Expected GNSS retrieved data.



Table 52: GNSS Interface Health Check: Expected Results

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → control-program - AdcsTelemetrySensorRaw → Raw GNSS sensor telemetry				
RTC Time (s)	Equal to GNSS-supplied		RTC time (PPS corrected)	
RTC Time (ns)	0 – 999'999'999		RTC time (PPS corrected)	
RTC Drift (ns)	0 – 999'999'999		RTC clock drift	
Satellite ECEF position vector X component	If using antenna: Realistic values, remaining ±constant		m	
Satellite ECEF position vector Y component			m	
Satellite ECEF position vector Z component	If using UART cable: As shown in Figure 66		m	
Satellite ECEF velocity vector X component			m/s	
Satellite ECEF velocity vector Y component			m/s	
Satellite ECEF velocity vector Z component			m/s	
GNSS-supplied unix time integer seconds	Incrementing and realistic		GNSS time	
GNSS-supplied unix time nanoseconds	0 – 999'999'999		N/A (always zero)	
GPS week number	If using antenna: Realistic values, remaining ±constant			
GPS Time of Week				
GPS UTC Offset	If using UART cable: As shown in Figure 66			
GNSS data is valid	Checked		The displayed data is valid	
GNSS has lock	Checked or Unchecked		Checked if values are within expected ranges	
GNSS PPS detected	Variable, will test later		GNSS PPS signal input detected	

Once the values in the above tables have been confirmed, the interface from the GNSS, via the client OBC, is considered to be in working condition. If any result obtained does not match the expected result, please contact CubeSpace.



6.13 PPS health check

NOTE: This section will use a different method depending on if the CubeComputer has been configured for receiving a PPS signal over a differentially signalled bus or single-ended 0 – 3.3V LVTTTL. Figure 67 shows the pin that drives the CubeSupport PCB's LVDS driver. Figure 68 shows the CubeSupport PCB's single-ended 0 – 3.3V LVTTTL PPS input pin. The CubeSupport PCB's GND pins are shown in Figure 62.

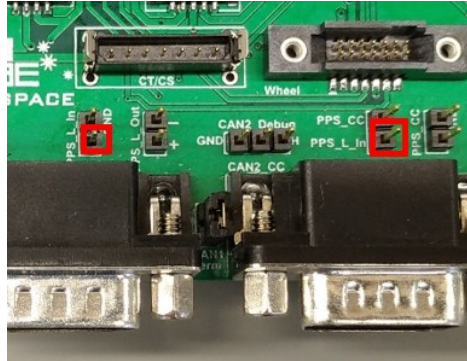


Figure 67: CubeSupport PCB “PPS_L_In” header pins indicated. Either of these can be used.

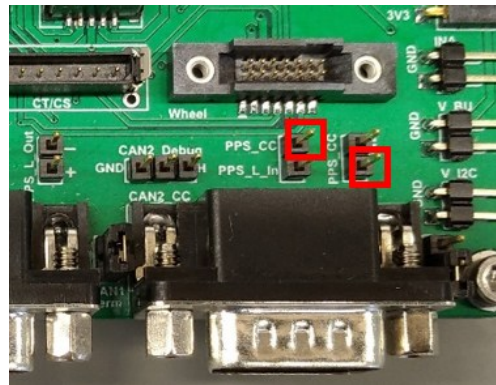


Figure 68: CubeSupport PCB “PPS_CC” header pins indicated. Either of these can be used.

Follow the steps below to perform the PPS health check via CubeComputer.

- 1) By requesting the raw GNSS telemetry, confirm that the “GNSS PPS Detected” checkbox is not checked and note the result in Table 53.
- 2) Apply a rising 3.3V edge to the CubeSupport PCB's PPS input pin.
- 3) For a differentially signalled PPS input signal, this would be one of the “PPS_L_In” pins as indicated in Figure 67. These two pins are connected together on the CubeSupport PCB.
OR
- 4) For a single-ended 0-3.3V LVTTTL PPS input signal, this would be one of the “PPS_CC” pins as indicated in Figure 68. These two pins are connected together on the CubeSupport PCB.
- 5) Use one of the GND pins on the CubeSupport PCB as shown in Figure 62 for the signal return.
- 6) Request the GNSS telemetry again and confirm that the “GNSS PPS Detected” checkbox is now checked. Note the result in Table 53.
- 7) Wait at least 5 seconds. The CubeComputer will lose track of the PPS input state after this time has elapsed.
- 8) Request the GNSS telemetry again and confirm that the “GNSS PPS Detected” checkbox is now cleared again. Note the result in Table 53.



Table 53: GNSS PPS Health Check: Power On

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 – AdcsTelemetrySensorRaw → Raw GNSS sensor telemetry				
GNSS PPS Detected	Unchecked		At the start of the test, when the PPS input pin has not changed state for at least 5 seconds.	
GNSS PPS Detected – 3.3V rising edge applied to PPS input pin	Checked		This telemetry result must be obtained within 5 seconds of the rising 3.3V edge being applied.	
GNSS PPS Detected – after waiting at least 5 seconds	Unchecked		This telemetry result must be obtained at least 5 seconds after the rising 3.3V edge being applied.	

The CubeComputer uses the PPS input to generate either a single-ended or differential PPS output signal for use by certain sensor devices which might require such a signal. The voltage levels on the PPS output pins are simply recorded in the next steps because the expected values are highly dependent on the specific client configuration. See Figure 69 and Figure 70 for specific pin numbers on the PPS Output connectors of the two CubeADCS variants.

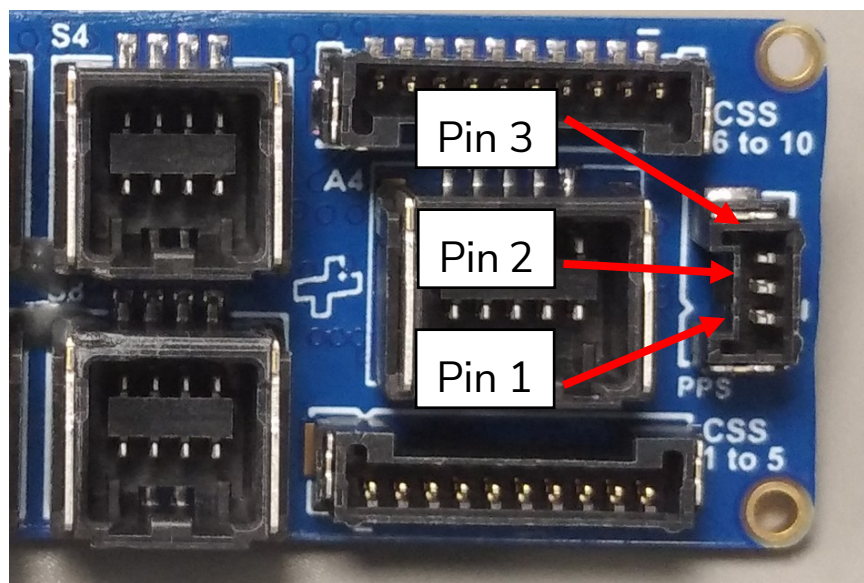


Figure 69: Pinout of the PPS output header, PC104 variant



Note that Figure 70 shows one of the existing CubeConnect pinouts of the Backplane variant, identified by the lettering “v4.3” on the silkscreen outline. Newer revisions (identified by “v4.4” on the silkscreen) have the same pinout as for the PC104 variants, with the same three signals appearing on two headers. See [RD2] for further detail.

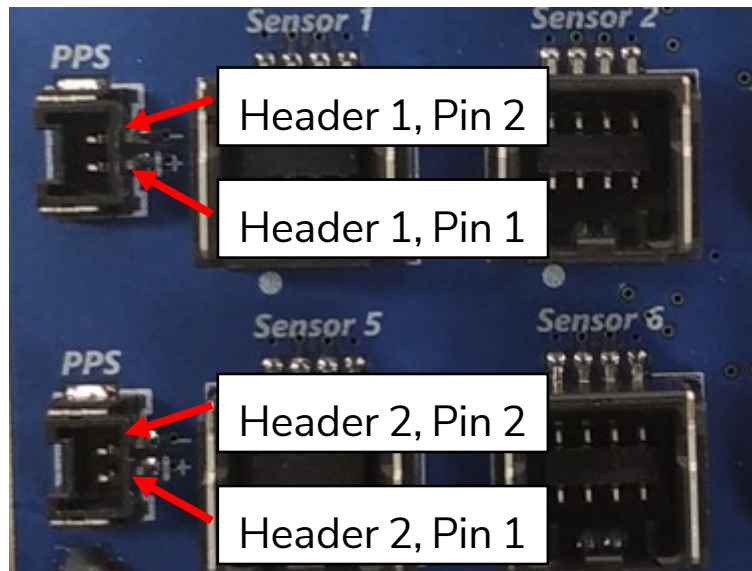


Figure 70: Pinout of the PPS output headers, Backplane variant

- 9) Apply a constant 3.3 V level to the CubeSupport PCB's PPS input pin as discussed in Step 2).
- 10) Using a multimeter, record the voltage levels between GND and all pins indicated in Figure 69 or Figure 70 in Table 54.
- 11) Apply a constant 0 V level to the CubeSupport PCB's PPS input pin as discussed in Step 2).
- 12) Using a multimeter, record the voltage levels between GND and all pins indicated in Figure 69 or Figure 70 in Table 54.

Table 54: GNSS PPS Health Check: Output header pin voltage level check

Field	Expected Result	For PC104 and newer backplane (v4.4) variants	For existing Backplane Variant (v4.3)
3.3 V applied to PPS input	Value on the Output header(s) between 0 and 3.3 V.	Output header, Pin 1: Output header, Pin 2: Output header, Pin 3:	Output header 1 Pin 1: Output header 1 Pin 2: Output header 2 Pin 1: Output header 2 Pin 2:
0 V applied to PPS input	Highly dependent on client configuration, some pins might even be not connected. Simply record the values.	Output header, Pin 1: Output header, Pin 2: Output header, Pin 3:	Output header 1 Pin 1: Output header 1 Pin 2: Output header 2 Pin 1: Output header 2 Pin 2:



6.14 Third-party Reaction Wheel health check

A high-level diagram of the setup for testing a third-party reaction wheel is shown in Figure 71. Specifically, NSS T2 Reaction Wheels are shown, but performing the health checks of other third-party reaction wheels will follow the same procedure as discussed below. A power switch board, not manufactured or supplied by CubeSpace, is also shown in Figure 71. In some configurations, this board or module is required to switch the power to the reaction wheel on or off, controlled by a 5V GPIO output of the CubeNode plugged into one of the Actuator headers on the CubeADCS core. The exact configuration of the test setup will depend on the client's selected options.

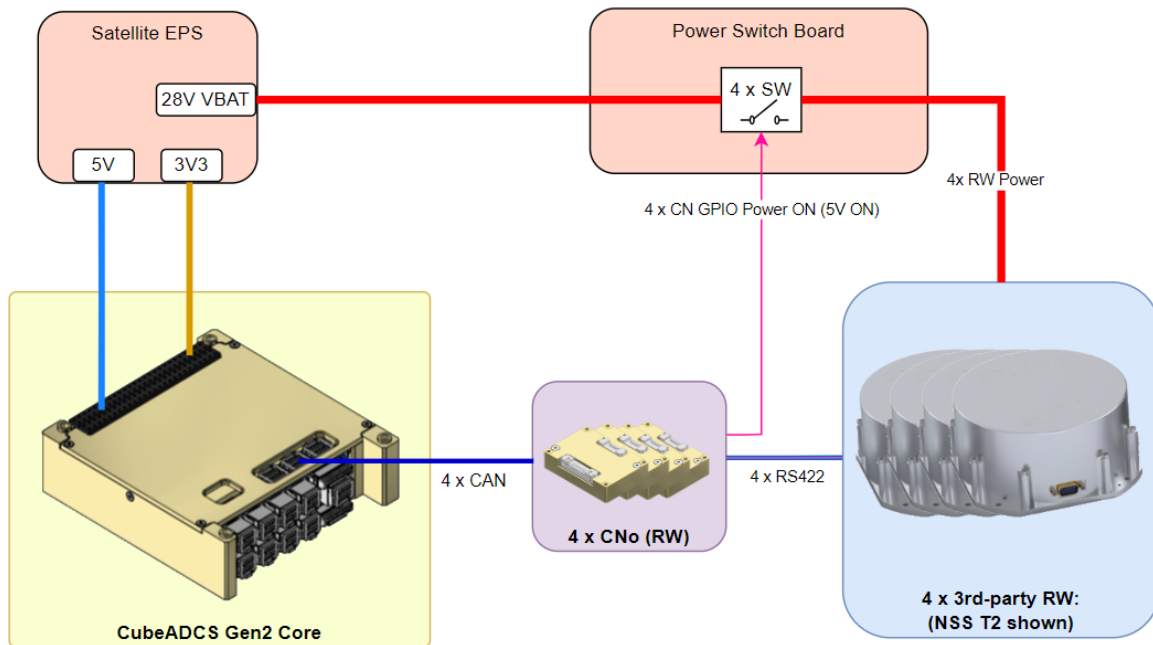


Figure 71: High-level diagram of CubeADCS, CubeNodes, Power Switch Board, and reaction wheels

An actual test setup of this arrangement is shown in Figure 72, with no other sensors or actuators plugged in, for clarity. While the CubeSupport PCB is still supplied with **12.0 V** during ground testing, the power for the Power Switch Board must be supplied from another bench PSU set to **28 V, 1.5 A**. Each additional connected reaction wheel will require an additional **1.5 A** of current (for example, for four wheels, the required power supply is **28 V, 6 A**). If one PSU channel cannot provide this current, this can instead be achieved by connecting more PSU channels in parallel.



Note that it is important that the correct CubeNode is used when connecting to the third-party product. For example, a CubeNode intended for a third-party star tracker will not function with a third-party reaction wheel. It should be indicated on the packaging what serial number CubeNode to use for which third-party product, but this might not always be the case.

Verifying that the correct CubeNode is used, is done in the following steps.

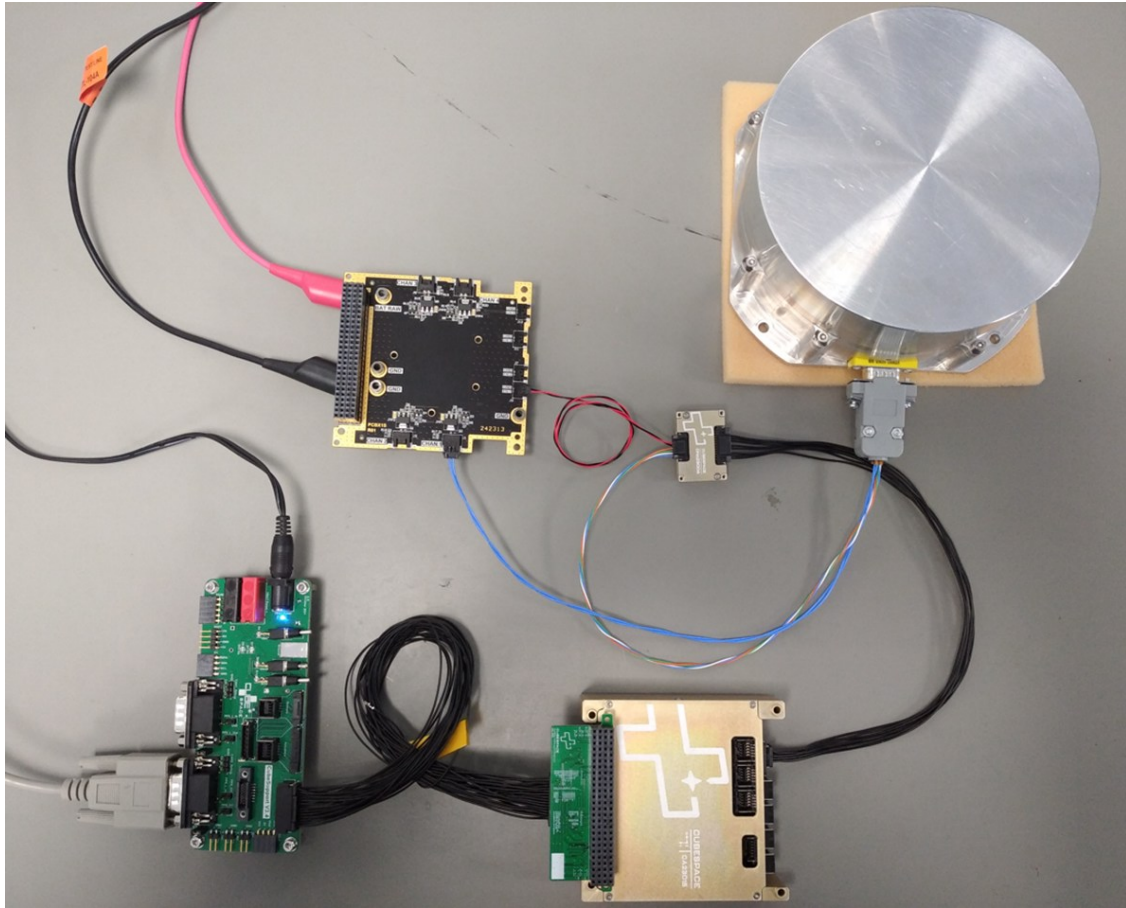


Figure 72: CubeADCS, CubeNode, Power Switch Board, and T2 reaction wheel

First determine that the correct CubeNode is used before connecting the third-party product:

- 1) Connect the CubeADCS to the CubeNode(s) and to the CubeSupport board as shown in Figure 72, **but do not connect anything else to the CubeNode output(s) yet**. Connect the CubeSupport PCB to a 12.0V power supply.
- 2) Verify that the correct CubeNode is used:
 - 2.1) At the top of the “PassThrough” tab, select the correct RWL from the Node dropdown menu , for example NodeRwl0. If the correct RWL does not appear in the Node dropdown menu, click the refresh button .
 - 2.2) From the Program dropdown menu , select ProgramTypeControl.
 - 2.3) Click the Connect button  to connect to the CubeNode control application.
 - 2.4) Scroll down the new view that appears and retrieve the Identification and Serial Number telemetry by clicking the refresh button  in the Identification2 and Serial Number fields (PassThrough tab  common-framework - Identification  Identification2 and PassThrough tab  common-framework - Identification  Serial Number).
- 3) Ensure that the retrieved values match the Expected Result as per Table 55, and fill in the results in the table.
- 4) Repeat for the remaining CubeNodes intended for use with the third-party reaction wheels.



Table 55: CubeNode-NSSRWL Health Check: CubeNode identifier result

Field	Expected Result	Actual result	Comments	P / F
PassThrough tab → common-framework - Identification → Identification2 and PassThrough tab → common-framework - Identification → Serial Number				
CubeNode for Rwl0	Identification2 → Node type identifier: NodeTypeCubeNodeNssrwL or NodeTypeCubeNodeQuadNssrwL	Node type, Program type, and Serial Integer matches: Serial integer:	“Node type identifier” can not be anything other than NodeTypeCubeNodeNssrwL or NodeTypeCubeNodeQuadNssrwL. “Program type identifier” can not be anything other than ProgramTypeControl. For some configurations, only 3 reaction wheels might be used, so Rwl3 will not be applicable.	
CubeNode for Rwl1	Identification2 → Program type identifier: ProgramTypeControl	Node type, Program type, and Serial Integer matches: Serial integer:		
CubeNode for Rwl2	Serial Number → Serial integer: As engraved or otherwise indicated on the CubeNode mechanics	Node type, Program type, and Serial Integer matches: Serial integer:		
CubeNode for Rwl3		Node type, Program type, and Serial Integer matches: Serial integer:		

- Once ensured that the correct CubeNode(s) are used, power down the CubeADCS by toggling the VBAT switch on the CubeSupport PCB to the OFF position.
 - Proceed with performing the remaining connections between the CubeNode(s) and the third-party devices, as shown in Figure 71 and in Figure 72.
 - Connect the CubeSupport PCB to a 12.0V power supply and the PDM to a 28V, 1.5A power supply (for one connected reaction wheel) or 28V, 6A (for four connected reaction wheels).
 - Power on the CubeADCS core via the VBAT On and ENABLE switches on the CubeSupport PCB
- Follow the next steps to perform the Reaction Wheel health check. After sending each command with the  button, refresh the field to ensure that the command has been received correctly. In the steps below, commands will be sent to the third-party reaction wheel via the CubeNode connected to CubeComputer.
- Power on the wheels as per Table 56.

Table 56: Reaction Wheel Health Check: Power On

Field	Expected Result	Actual result	Comments	P / F
Power State tab → cube-computer-common – PowerState				
RWL0 power state	PowerOn		Set, then Get to confirm receipt	
RWL1 power state	PowerOn		Set, then Get to confirm receipt	
RWL2 power state	PowerOn		Set, then Get to confirm receipt	
RWL3 power state	PowerOn		Set, then Get to confirm receipt	

- To command the open-loop wheel speed, CubeComputer must be in control mode “ConNone”. Verify this by navigating to the “Control mode” setting located at ADCS tab → cube-computer-5-control-program-8 – Commands → Control mode . If this is not true, select “ConNone” and send it to the CubeComputer.
- After setting the control mode, refresh the telemetry and confirm that it has been registered.
- Now command the wheel speeds as shown in Table 57.



Note that the NSS reaction wheels draw a lot of power while spinning up. Approximately 2 seconds after the wheel speed has been commanded, the wheel will start spinning, drawing approximately 1.2 ampere each until settling at the commanded speed. If the power supply is unable to supply this current, a voltage dip could occur that resets the wheel and CubeNode, requiring the user to power off the wheels (as explained in Steps 15) and 16)) and power them on again (as explained in Step 9)).

Table 57: Reaction Wheel Health Check: Command Wheel Speed

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 – Commands → OpenLoopCommandRwl				
RWL0 open-loop speed command	500		Set, then Get to confirm receipt	
RWL1 open-loop speed command	0		Set, then Get to confirm receipt	
RWL2 open-loop speed command	0		Set, then Get to confirm receipt	
RWL3 open-loop speed command	0		Set, then Get to confirm receipt	

13) Verify that the wheels measure approximately the commanded speed, as shown in Table 58.

Table 58: Reaction Wheel Health Check: Expected Actual speed

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 – AdcsTelemetrySensorRaw → Raw NSSRWL sensor telemetry				
Times integer seconds	Incrementing			
Time nanoseconds	0 – 999999999			
RWL0 target value	500 rpm			
RWL0 measured speed	500 rpm		±5 rpm	
RWL0 measured current	~30 mA		After settling at target speed	
RWL0 temperature	~ Room temp.			
RWL1 target value	0			
RWL1 measured speed	0			
RWL1 measured current	0			
RWL1 temperature	0			
RWL2 target value	0			
RWL2 measured speed	0			
RWL2 measured current	0			
RWL2 temperature	0			
RWL3 target value	0			
RWL3 measured speed	0			
RWL3 measured current	0			



Field	Expected Result	Actual result	Comments	P / F
RWL3 temperature	0			
RWL0 valid flag	Checked			
RWL0 wheel mode	Checked			
RWL0 current-control mode	Unchecked			
RWL0 speed-control mode	Checked			
RWL0 torque-control mode	Unchecked			
RWL0 CRC error	Unchecked			
RWL0 wrong low-level command	Unchecked			
RWL0 wrong high-level command	Unchecked			
RWL0 Timeout SCI1 main	Unchecked			
RWL1 valid flag	Unchecked		If using a CubeNode Quad, some of these values could be checked. The checked values must reflect those of RWL0.	
RWL1 wheel mode	Unchecked			
RWL1 current-control mode	Unchecked			
RWL1 speed-control mode	Unchecked			
RWL1 torque-control mode	Unchecked			
RWL1 CRC error	Unchecked			
RWL1 wrong low-level command	Unchecked			
RWL1 wrong high-level command	Unchecked			
RWL1 Timeout SCI1 main	Unchecked			
RWL2 valid flag	Unchecked			
RWL2 wheel mode	Unchecked			
RWL2 current-control mode	Unchecked			
RWL2 speed-control mode	Unchecked			
RWL2 torque-control mode	Unchecked			
RWL2 CRC error	Unchecked			
RWL2 wrong low-level command	Unchecked			
RWL2 wrong high-level command	Unchecked			
RWL2 Timeout SCI1 main	Unchecked			
RWL3 valid flag	Unchecked			
RWL3 wheel mode	Unchecked			
RWL3 current-control mode	Unchecked			
RWL3 speed-control mode	Unchecked			
RWL3 torque-control mode	Unchecked			
RWL3 CRC error	Unchecked			
RWL3 wrong low-level command	Unchecked			
RWL3 wrong high-level command	Unchecked			
RWL3 Timeout SCI1 main	Unchecked			



- 14) Further testing can also be done, for example spinning the wheels at -1000, 1500, and -2000 rpm.
(Note that the EM T2 wheel is rated for a wheel speed up to 2600 rpm).
- 15) Command all the wheels' speed back to 0 (ADCS tab → cube-computer-5-control-program-8 – Commands → OpenLoopCommandRwl).
- 16) Switch off all the wheels (Power State tab → cube-computer-common → PowerState).

Once the values in the above tables have been confirmed, the third-party Reaction Wheel(s) and the corresponding CubeNode(s) are considered to be in working condition. If any result obtained does not match the expected result, please contact CubeSpace.

6.15 PST3S Star Tracker health check

The setup for the health checking of a TY-Space PST3S Star Tracker is shown in Figure 73, with no other sensors or actuators plugged in, for clarity. Note that the CubeNode-CubeADCS connection requires the correct harness, with the additional PPS header. Also shown is a 1PPS signal supplied from a Function Generator and connected to the CubeSupport PCB as discussed in Section 6.13, specifically Step 2).



Note that it is important that the correct CubeNode is used when connecting to the third-party product. For example, a CubeNode intended for a third-party reaction wheel will not function with a third-party star tracker. It should be indicated on the packaging what serial number CubeNode to use for which third-party product, but this might not always be the case.

Verifying that the correct CubeNode is used, is done in the following steps.

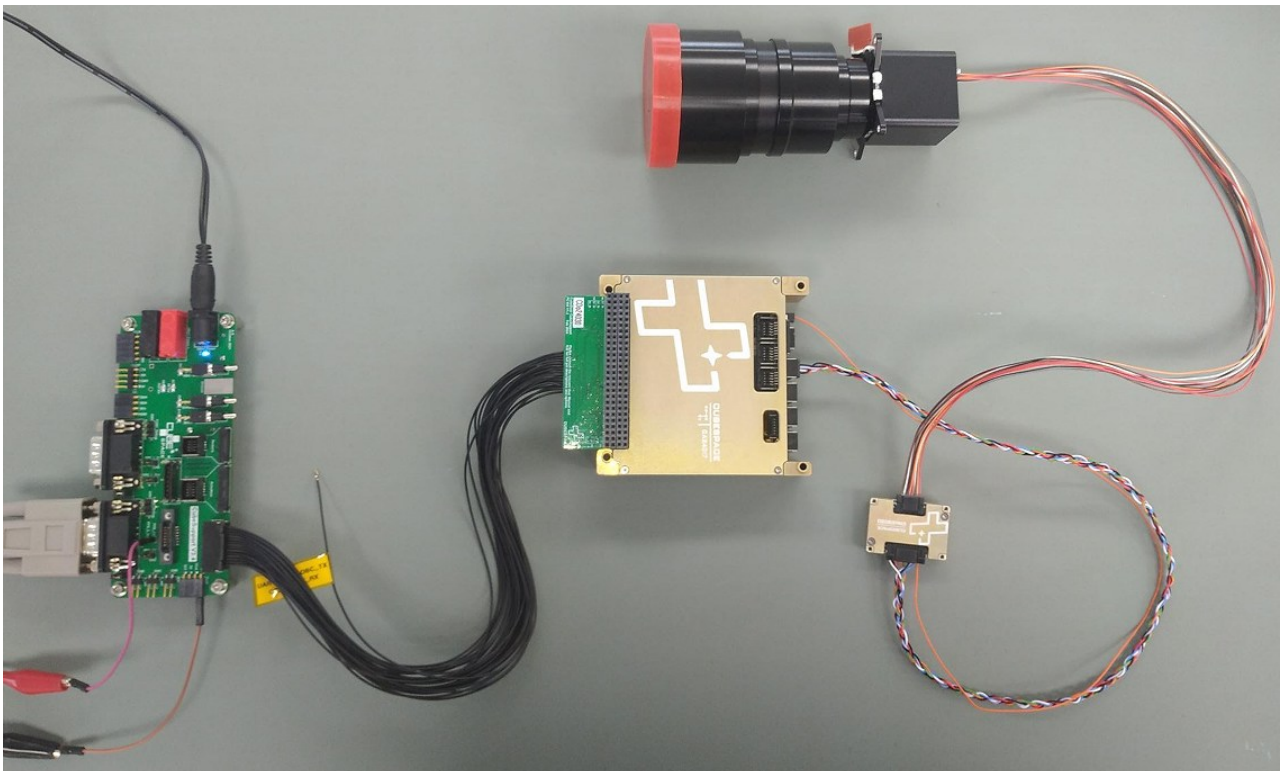


Figure 73: TY-Space Star Tracker connected to the CubeADCS via CubeNode

First determine that the correct CubeNode is used before connecting the third-party product:



- 1) Connect the CubeADCS to the CubeNode(s) and to the CubeSupport board as shown in Figure 73, **but do not connect anything else to the CubeNode output(s) yet**. Connect the CubeSupport PCB to a 12.0V power supply.
- 2) Verify that the correct CubeNode is used:
 - 2.1) At the top of the “PassThrough” tab, select the correct STR from the Node dropdown menu , for example NodeStr0. If the correct STR does not appear in the Node dropdown menu, click the refresh button .
 - 2.2) From the Program dropdown menu , select ProgramTypeControl.
 - 2.3) Click the Connect button  to connect to the CubeNode control application.
 - 2.4) Scroll down the new view that appears and retrieve the Identification2 and Serial Number telemetry by clicking the refresh button  in the Identification and Serial Number fields (PassThrough tab  common-framework - Identification  Identification2 and PassThrough tab  common-framework - Identification  Serial Number).
- 3) Ensure that the retrieved values match the Expected Result as per Table 55, and fill in the results in the table.
- 4) Repeat for the remaining CubeNodes intended for use with the third-party star trackers.

Table 59: CubeNode-PST3S Health Check: CubeNode identifier result

Field	Expected Result	Actual result	Comments	P / F
PassThrough tab → common-framework – Identification → Identification2 and PassThrough tab → common-framework – Identification → Serial Number				
CubeNode for Str0	Identification2 → Node type identifier: NodeTypeCubeNodePst3s Identification2 → Program type identifier: ProgramTypeControl	Node type, Program type, and Serial Integer matches: Serial integer:	“Node type identifier” can not be anything other than NodeTypeCubeNodePst3s. “Program type identifier” can not be anything other than ProgramTypeControl.	
CubeNode for Str1	Serial Number → Serial integer: As engraved or otherwise indicated on the CubeNode mechanics	Node type, Program type, and Serial Integer matches: Serial integer:	For some configurations, only 1 star tracker might be used, so Str1 will not be applicable.	

- 6) Once ensured that the correct CubeNode(s) are used, power down the CubeADCS by toggling the VBAT switch on the CubeSupport PCB to the OFF position.
- 7) Proceed with performing the remaining connections between the CubeNode(s) and the third-party devices, as shown in Figure 73.
- 8) Power on the CubeADCS core via the VBAT On and ENABLE switches on the CubeSupport PCB

Follow the next steps to perform the PST3S Star Tracker health check. After sending each command with the  button, refresh the field to ensure that the command has been received correctly. In the steps below, commands will be sent to the third-party star tracker via the CubeNode connected to CubeComputer.



Note that the following tests were performed with an H2 variant of the PST3S Star Tracker, but the results are still valid for the H5 model.

9) Power on the third-party Star Tracker.

Table 60: PST3S Star Tracker Health Check: Power On

Field	Expected Result	Actual result	Comments	P / F
Power State tab → cube-computer-common – PowerState				
STR0 power state	PowerOn		Set, then Get to confirm receipt	

10) Verify that the current drawn from a bench power supply set to **12.0V** increases by values similar to those indicated in Table 61 for each unit assessed.

Table 61: PST3S Star Tracker Health Check: Expected Current Consumption

Field	Expected Result	Actual result	Comments	P / F
Idle CubeNode current – POWER ON	Without PST3S Star Tracker connected: ~20 mA With PST3S Star Tracker connected: ~200 mA		Failed if > 40 mA with no components attached to the CubeNode. Failed if > 250 mA with a TY Space Star Tracker attached to the CubeNode.	

11) Request (📡) the Raw PST3S telemetry and verify that the raw telemetry packet is as per Table 62.



Note that the results in Table 62 were obtained without any TY-Space PST3S Star Tracker nor simulator connected to the CubeNode PST3S, allowing for standalone CubeNode PST3S health checking.

Table 62: PST3S Star Tracker Health Check: Expected Results without PST3S Star Tracker connected.

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 - AdcsTelemetrySensorRaw → Raw PST3S star tracker telemetry				
Timestamp	Incrementing			
Star tracker TCTLM instance	Str0			
PST3S Version	Pst3sVersionG2			
Quaternion 1	-1 to +1			
Quaternion 2	-1 to +1			
Quaternion 3	-1 to +1			



Field	Expected Result	Actual result	Comments	P / F
Quaternion 4	-1 to +1			
Angular Velocity X	Near 0			
Angular Velocity Y	Near 0			
Angular Velocity Z	Near 0			
Rate Quality	RateInvalid			
Unix Seconds	0			
Unix Seconds Decimal	0.000000			
Temperature	0			
Image Exposure	0			
Image Threshold	0			
Background Values	0			
Internal Process Code	0			
CMOS Chip ID Self-test Result	Correct			
FPGA MRAM Self-test Result	Correct			
Number of Extracted Stars	0			
Operating Mode	Normal Mode or ModelInvalid			
Imaging Gain	0			
Minimum Navigation Stars	0			
Data Valid	GestureValid			
External Image State	ImageInvalid			
Number of Identified Stars	0			
Device Number	0			
Software Version	0			
EDAC Error Count	0			
Image Frame Number	0			
Dynamic Mode	DynamicModelInvalid			
SAA Mode	SaaInvalid			
SAA Threshold	0			
Quaternion Filtering State	QFilteringGetInvalid			
Four Star Capture Threshold	0			
Tracking Threshold	0			
EDAC Switch	Unchecked			



Note that the Expected Results in Table 63 were captured with the TY-Space Star Tracker connected to a TY-Space Star Pattern Simulator, which the client might not have access to. Therefore, two columns of “Expected Results” are supplied.



Figure 74: TY-Space Star Pattern Simulator

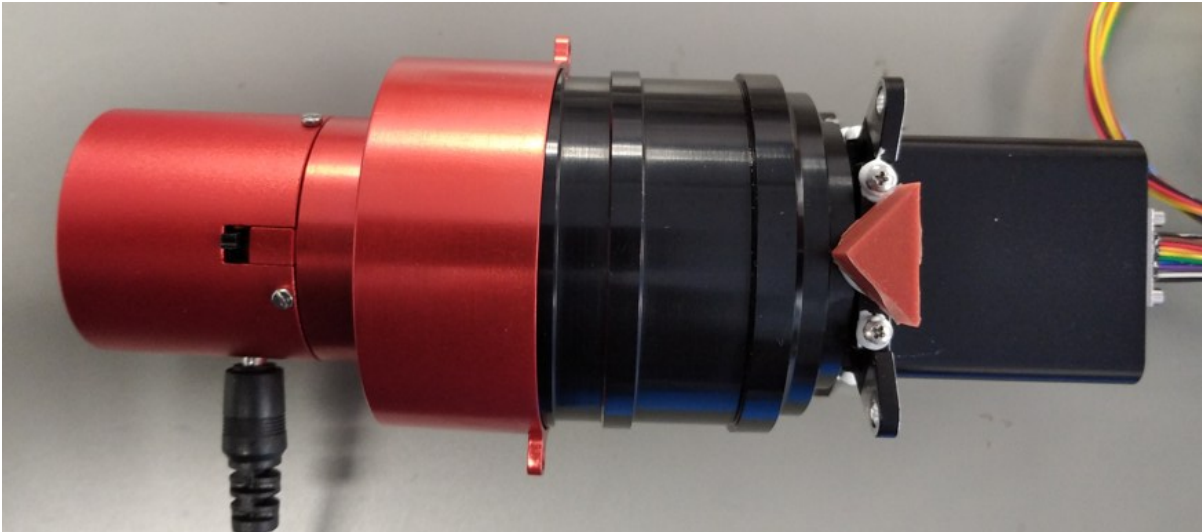


Figure 75: TY-Space Star Pattern Simulator mounted on the TY-Space Star Tracker

Table 63: PST3S Star Tracker Health Check: Expected Results with PST3S Star Tracker connected.

Field	Expected Result: Without Star Simulator	Expected Result: With Star Simulator	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 - AdcsTelemetrySensorRaw → Raw PST3S star tracker telemetry					
Timestamp	Incrementing	Incrementing			
Star tracker TCTLM instance	Str0	Str0			



Field	Expected Result: Without Star Simulator	Expected Result: With Star Simulator	Actual result	Comments	P / F
PST3S Version	Pst3sVersionH3	Pst3sVersionH3		CubeNode protocol version is the same for H3 and H5 models	
Quaternion 1	-1 to +1	-1 to +1		This could vary slightly if the Simulator is used and carefully moved / rotated.	
Quaternion 2	-1 to +1	-1 to +1			
Quaternion 3	-1 to +1	-1 to +1			
Quaternion 4	-1 to +1	-1 to +1			
Angular Velocity X	Near 0	Near 0			
Angular Velocity Y	Near 0	Near 0			
Angular Velocity Z	Near 0	Near 0			
Rate Quality	RateInvalid	Precise		1PPS signal is supplied through the CubeSupport PCB.	
Unix Seconds	Incrementing	Incrementing		PST3S Time	
Unix Seconds Decimal	0 – 999'999	0 – 999'999		PST3S Time	
Temperature	+ - Room Temperature	+ - Room Temperature			
Image Exposure	80	80			
Image Threshold	11	11			
Background Values	0	0			
Internal Process Code	0	1			
CMOS Chip ID Self-test Result	Correct	Correct		Not relevant to H3/H5	
FPGA MRAM Self-test Result	Correct	Correct			
Number of Extracted Stars		15		Variable	
Operating Mode	Normal Mode	Normal Mode			
Imaging Gain	26	26			
Minimum Navigation Stars	5	5			
Data Valid	GestureValid	AttitudeValid		Could be "GestureValid" if the minimum number of identified stars is not met.	
External Image State	ImageInvalid	ImageInvalid			
Number of Identified Stars	0	9		Variable	
Device Number	Refer to the PST3S Manual	Refer to the PST3S Manual		Serial number	
Software Version	Refer to the PST3S Manual	Refer to the PST3S Manual			
EDAC Error Count	0	0			
Image Frame Number	Incrementing	Incrementing			
Dynamic Mode	DynamicModeLow	DynamicModeLow			



Field	Expected Result: Without Star Simulator	Expected Result: With Star Simulator	Actual result	Comments	P / F
SAA Mode	SaaTracking	SaaTracking			
SAA Threshold	10	10			
Quaternion Filtering State	QFilteringGetInvalid	QFilteringGetInvalid			
Four Star Capture Threshold	10	10			
Tracking Threshold	10	10			
EDAC Switch	Checked	Checked			

12) Power off the Star Tracker (control-program - Commands  Power state).

13) Repeat these steps for all additional third-party star trackers if applicable. Fill in Table 64.

Table 64: PST3S Star Tracker Health Check: Checks passed.

Field	Expected Result	Actual result	Comments	P / F
STR0	All checks passed			
STR1	All checks passed			

Once the values in the above tables have been confirmed, the PST3S Star tracker(s) and corresponding CubeNode(s) are considered to be in working condition. If any result obtained does not match the expected result, please contact CubeSpace.

6.16 LITEF uFORS Fibre Optic Gyro Health check

Due to the limited amount of sensor and actuator ports of the CubeADCS, the uFORS Gyros are powered and operated via a CubeNode Quad. Each external fibre-optic gyro measures one of the X, Y, or Z axes, and three of these are plugged into the CubeNode Quad, which in turn is plugged into the CubeADCS. Figure 76 shows this arrangement.

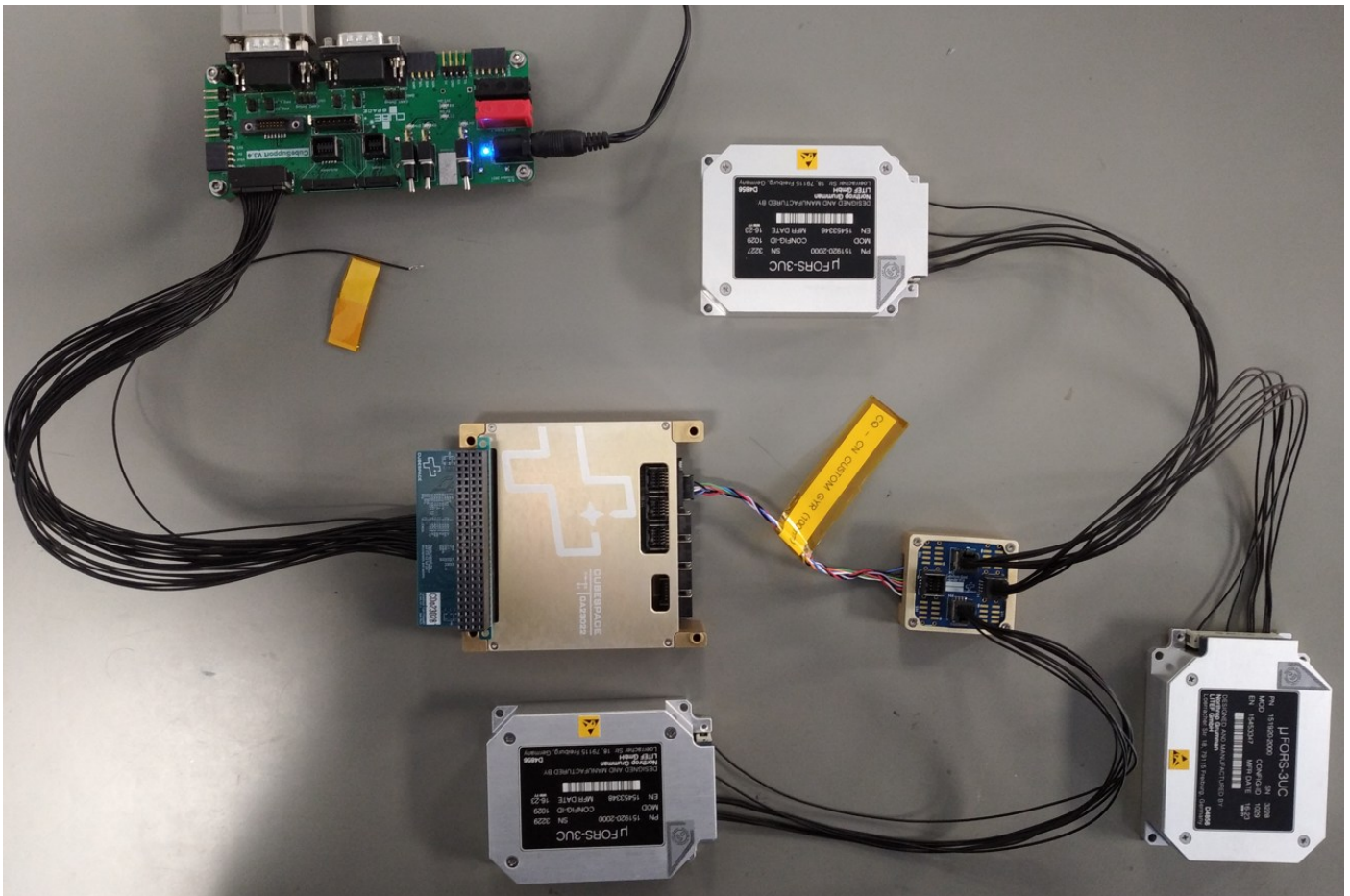


Figure 76: Three uFORS Gyros connected to the CubeADCS via CubeNode Quad



Note in Figure 76 that the three gyros are connected to the CubeNode Quad ports labelled H2 (X-Axis, Port 2), H3 (Y-Axis, Port 3), and H4 (Z-Axis, Port 4). This configuration is currently hardcoded in the CubeNode Quad firmware. If a client wishes to change this arrangement, please contact CubeSpace.

First determine that the correct CubeNode Quad is used before connecting the third-party products:

- 1) Connect the CubeADCS to the CubeNode Quad and to the CubeSupport board as shown in , **but do not connect anything else to the CubeNode Quad output(s) yet**. Connect the CubeSupport PCB to a 12.0V power supply.
- 2) Verify that the correct CubeNode Quad is used:
- 3) At the top of the “PassThrough” tab, select the correct EXTGYR from the Node dropdown menu  , for example NodExtGyr0. If the correct EXTGYR does not appear in the Node dropdown menu, click the refresh button  .
- 4) From the Program dropdown menu  , select ProgramTypeControl.
- 5) Click the Connect button  to connect to the CubeNode Quad control application.
- 6) Scroll down the new view that appears and retrieve the Identification and Serial Number telemetry by clicking the refresh button  in the Identification2 and Serial Number fields (PassThrough tab  common-framework - Identification  Identification2 and PassThrough tab  common-framework - Identification  Serial Number).
- 7) Ensure that the retrieved values match the Expected Result as per Table 65, and fill in the results in the table.



Table 65: uFORS Gyros Health Check: CubeNode Quad identifier result

Field	Expected Result	Actual result	Comments	P / F
PassThrough tab → common-framework – Identification → Identification2 and PassThrough tab → common-framework – Identification → Serial Number				
CubeNode Quad for ExtGyr0	Identification2 → Node type identifier: NodeTypeCubeNodeQuadLiteufors Identification2 → Program type identifier: ProgramTypeControl Serial Number → Serial integer: As engraved or otherwise indicated on the CubeNode Quad mechanics	Node type, Program type, and Serial Integer matches: Serial integer:	“Node type identifier” can not be anything other than NodeTypeCubeNodeQuadLiteufors. “Program type identifier” can not be anything other than ProgramTypeControl.	

- 8) Once ensured that the correct CubeNode Quad(s) are used, power down the CubeADCS by toggling the VBAT switch on the CubeSupport PCB to the OFF position.
- 9) Proceed with performing the remaining connections between the CubeNode(s) and the third-party devices, as shown in Figure 76.
- 10) Power on the CubeADCS core via the VBAT On and ENABLE switches on the CubeSupport PCB

Follow the next steps to perform the uFORS Gyros health check. After sending each command with the  button, refresh the field to ensure that the command has been received correctly. In the steps below, commands will be sent to the third-party gyros via the CubeNode Quad connected to CubeComputer.

- 1) Power on the CubeNode Quad and connected uFORS Gyros.

Table 66: uFORS Gyros Health Check: Power On

Field	Expected Result	Actual result	Comments	P / F
Power State tab → cube-computer-common – PowerState				
EXTGYR0 power state	PowerOn		Set, then Get to confirm receipt	

- 2) Verify that the current drawn from a bench power supply set to **12.0V** increases by values similar to those indicated in Table 67 for each set of gyros assessed.



Note that the current for the CubeNode Quad is drawn exclusively from the 3.3 V rail, while the power for the uFORS Gyros is drawn exclusively from the 5 V rail. The CubeNode Quad simply switches this 5 V supply to each gyro on or off.

Table 67: uFORS Gyros Health Check: Expected Current Consumption

Field	Expected Result	Actual result	Comments	P / F
Idle CubeNode Quad current – POWER ON	Without any uFORS Gyros connected: ~30 mA Additional current drawn by each connected uFORS Gyro: ~220 mA		Failed if > 50 mA with no components attached to the CubeNode Quad. Failed if > 800 mA with three uFORS Gyros connected.	



- 3) Request (🔍) the Raw uFORS gyros telemetry and verify that the raw telemetry packet is as per Table 68.

Table 68: uFORS Gyros Health Check: Expected rates

Field	Expected Result	Actual result	Comments	P / F
ADCS tab → cube-computer-5-control-program-8 – AdcsTelemetrySensorRaw → RAW LITEF uFORS sensor telemetry				
Time integer seconds	Incrementing			
Time nanoseconds	0 – 999 999 999			
EXTGYR0 Rate uFORS X	-0.005 to +0.005 deg/s		These values must be near zero, but varying slightly, when the uFORS Gyros are not being rotated. If the gyros are carefully rotated, these values will indicate the measured rate of rotation around the specific axis in degrees per second. Note that each connected uFORS gyro only measures one axis, and must be rotated around its indicated axis of rotation. Rotating it around one of the other two axis will yield no result for that specific gyro.	
EXTGYR0 Rate uFORS Y	-0.005 to +0.005 deg/s			
EXTGYR0 Rate uFORS Z	-0.005 to +0.005 deg/s			
All other EXTGyro checkboxes unchecked	Unchecked			
All EXTGyro1 Rate values	0.000			
All other EXTGyro1 checkboxes unchecked	Unchecked			

Once the values in the above table have been confirmed, the LITEF uFORS fibre optic gyro(s) and corresponding CubeNode Quad are considered to be in working condition. If any result obtained does not match the expected result, please contact CubeSpace.

6.17 Operation at high and low ends of expected VBAT range

The CubeADCS must be powered on, and all connected sensors and actuators switched on so that the worst-case current consumption can be tested. This has to be repeated at both the low and high end of the expected VBAT range.

The exact results from these tests are not recorded but are nevertheless checked to be within the limits specified earlier in this document. Fill in Table 69 below.

- 4) Based on the actuators used, determine the minimum and maximum values for the battery voltage supply to be used in this test, from the table below.



Table 69: VBAT extremes for different product configurations

Actuators used	Minimum VBAT	Maximum VBAT	Requires separate CubeSupport and VBAT supplies (Section 6.5.1)?
CubeWheels: CW0017, CW0057, or CW0162	10.0 V	17.0 V	No
CubeWheels: CW0500 or CW1200	10.0 V	24.0 V	Yes
CubeWheels: CW2500 or CW5000	24.0 V	36.0 V	Yes
Other third-party reaction wheels	Consult CubeSpace Engineering team, who will populate Table 70		Yes
No reaction wheels included in shipment	10.0 V	17.0 V	No

Note: The voltage supply to the **CubeSupport PCB** itself can be 10.0V – 17.0V, but ensure it is never supplied with **more than 17.6V**.

5) Record the determined minimum and maximum battery voltage to be used for this test in the table below.

Note: If the maximum voltage available from the Power Supply for the VBAT supply is only for example 30V or 32V, then use the maximum available, just make a note in Table 70 below.

Table 70: Extremes to be used for VBAT test

Field	Actuators used	Minimum VBAT	Maximum VBAT
Record the values determined from Table 69 above.			

6) If third-party reaction wheels are used and powered by a client-supplied Power Switch, ensure that the independent power supply is correctly set up as discussed in Section 6.14, Step 7), and proceed with the steps below.

7) Perform the tests from, and fill in the results into, the table below.

Table 71: Operation against VBAT extremes

Field	Expected Result	Actual result	Comments	P / F / NA
Set the bench PSU to the maximum battery voltage determined in Table 70				
Power cycle the CubeADCS by toggling the ENABLE switch on the CubeSupport PCB	Current consumption of the CubeADCS falls away. When restarting, normal operation resumes after booting into the Control Program.			
Send a software reset command	When restarting, normal operation resumes after booting into the Control Program.			



Field	Expected Result	Actual result	Comments	P / F / NA
Switch all connected sensors and actuators on			Refer to previous sections if required	
Set all three magnetorquers to maximum allowed duty cycle			Software will limit this to no more than 80%	
Simultaneously spin up all connected reaction wheels to 2000 rpm				
Record current consumption from the bench PSU set to maximum VBAT			After reaction wheels have settled at 2000 rpm.	
Retrieve the CubeComputer Health telemetry as in Table 15 and record the Battery voltage	~24000 mV for CW0500 or CW1200 ~28000 mV for CW2500 or larger, ~ 16300 mV otherwise (after accounting for the voltage drop of the diode on the CubeSupport PCB)			
Spin all connected reaction wheels down to 0 rpm, switch off all magnetorquers and connected sensors and actuators.				
Set the bench PSU to the minimum battery voltage determined in Table 70				
Power cycle the CubeADCS by toggling the ENABLE switch on the CubeSupport PCB	Current consumption of the CubeADCS falls away. When restarting, normal operation resumes after booting into the Control Program.			
Send a software reset command	When restarting, normal operation resumes after booting into the Control Program.			
Switch all connected sensors and actuators on			Refer to previous sections if required	
Set all three magnetorquers to maximum allowed duty cycle			Software will limit this to no more than 80%	
Simultaneously spin up all connected reaction wheels to 2000 rpm			Large actuators might require additional VBAT power supply channels in parallel. Otherwise, spinning up the wheels progressively is acceptable.	
Record current consumption from the bench PSU set to minimum VBAT			After reaction wheels have settled at 2000 rpm	



Field	Expected Result	Actual result	Comments	P / F / NA
Retrieve the CubeComputer Health telemetry as in Table 15 and record the Battery voltage	~10000 mV for CW0500 or CW1200 (CubeSupport PCB bypassed) ~24000 mV for CW2500 or larger ~ 9300 mV otherwise (after accounting for the voltage drop of the diode on the CubeSupport PCB)			
Spin all connected reaction wheels down to 0 rpm, switch off all magnetorquers and connected sensors and actuators.				



6.18 Known issues

This section is for recording known issues that were encountered during the Health Check but were not considered to be blockers for proceeding with the order. **Note:** This section might be left blank if no issues were encountered.

- 1) At the time of writing this revision of this document, an issue exists in the CubeSupport Application that prevents an image downloaded from a CubeStar to be saved to the local PC. A screenshot can be taken of the CubeSupport view if a record of the downloaded image is required.
- 2) At the time of writing this revision of this document, an issue exists in the CubeSupport Application that prevents the saving of downloaded CubeStar bitmaps correctly (see Table 42).



7. Appendix: CubeProduct Upgrades via CubeSupport

This appendix will describe how to upgrade CubeProducts connected to the CubeADCS using the CubeSupport application. The implementation of the CubeSupport application follows section 4.8.

This section does not describe how to upgrade the CubeComputer control program. This upgrade is performed by the bootloader program and is described in [RD3].

This appendix assumes that the user has successfully completed the health check/acceptance test, and that the CubeSupport application is currently connected to the CubeComputer control program.

Navigate to the “File Transfer / Upgrades” tab. The user interface (UI) is shown in Figure 77.

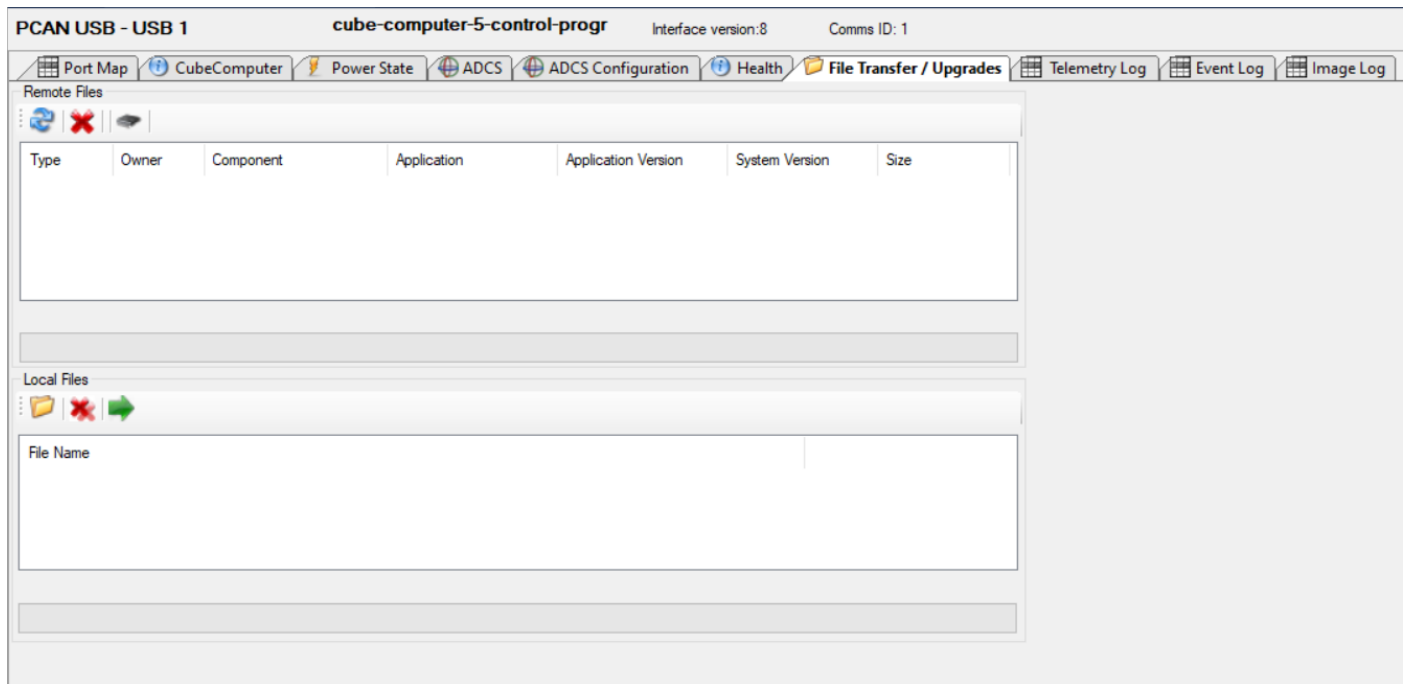


Figure 77: File Transfer / Upgrades UI

The UI consists of a “Remote Files” group (top), and a “Local Files” group (bottom).

The local files group is used to load local CubeSpace (.cs) files and stage them for upload. Once staged, the files that the user wants to upload can be selected and then uploaded. The progress bar at the bottom of the group shows the progress of the upload of each individual file. When the batch of uploads is complete a message box will appear indicating a successful upload of all files, or individual message box(s) for each failed upload. When the user clicks the “Load File” button, a dialogue box will appear, shown in Figure 78.

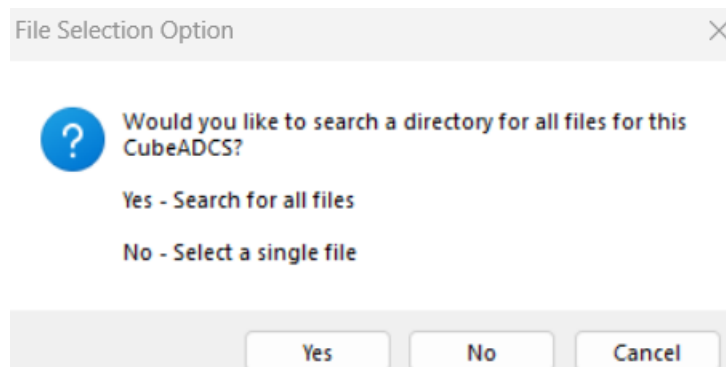


Figure 78: File Load Option



If the user selects the “Yes” option, the ExpectedNodes telemetry will be requested from the CubeADCS. The user can then select a directory to search. All the firmware and configuration files for the expected nodes in the selected directory will be loaded and staged for upload. This option is typically used when a new software bundle is provided, and the user wishes to upgrade the entire CubeADCS system. Note that bootloader files are not included in the search as they are not required under normal operating conditions. If the user wishes to upload bootloader files, select the “No” option and search for the specific bootloader file.

The remote files group is used to view files stored on the CubeComputer. The refresh button can be used to request file information for all files. The file list is automatically refreshed after every batch of file uploads. Once the user has confirmed that the required files are present on the CubeComputer, the user can then select the configuration file(s) of the CubeProduct(s) to be upgraded, then click the upgrade button. Note that although both the CubeProduct firmware binary and configuration are both upgraded, only the configuration file should be selected. This is because the parameters required to initiate an upgrade, as described in section 4.8.3, are only listed in the file information for configuration files, since the firmware binary is shared between all CubeProducts of the same type and cannot be used to target a specific CubeProduct. This requires user discretion to ensure that the firmware binary for the selected configuration file is also present. The progress bar at the bottom of the group shows the progress of each individual upgrade. When the batch of upgrades is complete a message box will appear indicating a successful upgrade of all CubeProducts, or individual message box(s) for each failed upgrade.

Note that upgrading the bootloader of a connected CubeProduct is not possible from the remote files group. If the bootloader firmware binary files have been uploaded, they will appear in the list of files, however, the bootloader application does not have a configuration file and so does not fit the program flow of this user control. Under normal conditions, the bootloader should not be upgraded, however if the user requires it, raw telecommands/telemetry must be used from the “CubeComputer” tab. The process for a bootloader upgrade is the same as described in section 4.8.3.

The user may hover the mouse cursor over the buttons to see the tool-tip text on what the button is used for.

For information on how to upgrade the CubeComputer control program via CubeSupport, refer to [RD3].